

DSA

Data Structures & Algorithms

Apuntes para entrevistas Google-level

 Big-O	 Arrays	 Listas	 Árboles
 Grafos	 Búsqueda	 Sorting	 DP

Aldo Zepeda Montoya

zetinaa3@gmail.com

github.com/AldoZM

30 de junio de 2026

Índice general

I	 Fundamentos	5
1.	 Big-O Notation	6
2.	 Complejidad Espacial (Space Complexity)	10
II	 Estructuras Lineales	12
3.	 Arrays (Vectores)	13
4.	 Linked Lists (Listas Enlazadas)	16
5.	 Fast & Slow Pointers (Tortuga y Liebre)	20
6.	 Patrones de Linked List (Reverse, Merge, Reorder)	24
7.	 Stacks (Pilas)	27
8.	 Queues (Filas / Colas)	30
III	 Estructuras No Lineales	33
9.	 Binary Search Trees (Árboles Binarios de Búsqueda)	34
10.	 AVL Trees (Árboles Binarios Balanceados)	38
11.	 Recorridos de Árboles (Tree Traversals)	42
12.	 LCA y Serialización de Árboles	46
13.	 Propiedades de Árboles (Altura, Diámetro, Balance)	50
14.	 Heaps (Montículos)	54
15.	 Segment Tree (Árbol de Segmentos)	57
16.	 Fenwick Tree / BIT (Binary Indexed Tree)	61
17.	 Hash Tables (Tablas Hash / Mapas)	65
18.	 Hashing a Fondo: Colisiones e Implementación	68

19.	 Patrones con Hash Map (Frecuencias, Prefix-Sum, Agrupación)	72
20.	 LRU Cache (Hash Map + Lista Doble)	76
21.	 Tries (Árboles de Prefijos)	80
22.	 Union-Find (Disjoint Set Union - DSU)	84
 IV  Grafos		88
23.	 Representación de Grafos	89
24.	 BFS y DFS (Recorridos de Grafos)	92
25.	 BFS Avanzado (Grid, Multi-source y 0-1 BFS)	96
26.	 Algoritmo de Dijkstra (Camino más Corto)	100
27.	 Bellman-Ford (Camino más Corto con Pesos Negativos)	104
28.	 Floyd-Warshall (Todos los Pares)	108
29.	 Topological Sort (Ordenamiento Topológico)	112
30.	 MST: Kruskal y Prim (Árbol de Expansión Mínima)	116
31.	 Detección de Ciclos en Grafos	120
32.	 Componentes Fuertemente Conexas (SCC · Kosaraju)	124
33.	 Grafo Bipartito (2-Coloring)	128
34.	 Puntos de Articulación y Puentes (Tarjan)	132
35.	 Flujo Máximo (Ford-Fulkerson / Edmonds-Karp)	136
 V  Algoritmos de Ordenamiento		140
36.	 Ordenamientos Básicos (Bubble, Insertion, Selection)	141
37.	 Merge Sort (Ordenamiento por Mezcla)	145
38.	 Quick Sort (Ordenamiento Rápido)	148
39.	 Heap Sort (Ordenamiento por Montículo)	151
40.	 Counting, Radix y Bucket Sort (No Comparación)	155

VI	 Algoritmos de Búsqueda	159
41.	 Linear Search (Búsqueda Lineal)	160
42.	 Binary Search (Búsqueda Binaria)	163
VII	 Patrones de Entrevista	167
43.	 Two Pointers (Dos Punteros)	168
44.	 Sliding Window (Ventana Deslizante)	171
45.	 Introducción a Dynamic Programming (DP)	174
46.	 DP - Memoization (Top-Down)	177
47.	 DP - Tabulation (Bottom-Up)	180
48.	 DP: Knapsack, Subset Sum y Coin Change	183
49.	 DP en Cadenas: LCS y Edit Distance	187
50.	 DP: Longest Increasing Subsequence (LIS)	191
51.	 Greedy (Algoritmos Golosos)	195
52.	 Backtracking (Vuelta Atrás)	199
53.	 Bit Manipulation (Manipulación de Bits)	202
54.	 Recursión y Memoization	205
VIII	 Cadenas y Matemáticas	208
55.	 Búsqueda de Patrones: KMP y Rabin-Karp	209
56.	 Matemáticas: GCD, Criba y Exponenciación Rápida	213



Fundamentos

Capítulo 1

Big-O Notation

¿Qué es?

Imagínate que tienes una caja de chocolates y quieres encontrar uno de sabor fresa. ¿Cuánto tiempo tardarías?

- **Caso 1:** Si sabes que el de fresa SIEMPRE está en la posición 1 → Lo agarras directo. No importa si hay 10 o 1,000,000 chocolates. → Siempre es 1 paso. ¡Rápidísimo!
- **Caso 2:** Si los chocolates están desordenados y tienes que revisar uno por uno hasta encontrar el de fresa → Si hay 10 chocolates, revisas hasta 10. → Si hay 1,000,000 chocolates, revisas hasta 1,000,000. → El tiempo crece igual que la cantidad de chocolates.
- **Caso 3:** Si por cada chocolate que revisas, tienes que revisar TODOS los demás para compararlos → Si hay 10 chocolates → $10 \times 10 = 100$ comparaciones → Si hay 100 chocolates → $100 \times 100 = 10,000$ comparaciones → ¡Crece muy rápido y se pone muy lento muy pronto!

Big-O Notation es la forma que tienen los programadores para describir qué tan rápido o lento crece el tiempo que tarda un algoritmo cuando le das más y más datos.

La "n" siempre representa la cantidad de datos de entrada. Big-O siempre describe el PEOR caso posible (lo más lento que puede ser).

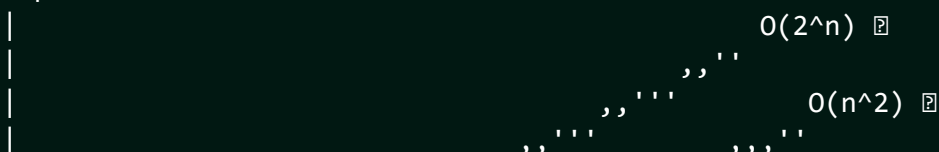
Dicho de otra forma

Big-O es la métrica estándar para comparar la eficiencia de algoritmos independientemente del hardware. Describe cómo escala el tiempo de ejecución (o uso de memoria) en función del tamaño del input "n", siempre en el worst case.

En interviews, SIEMPRE tienes que dar el Big-O de tu solución. Es tan esperado como que el código funcione.

Diagrama

Tiempo





$O(1)$	→ Constante	→ IDEAL
$O(\log n)$	→ Logarítmico	→ MUY BUENO
$O(n)$	→ Lineal	→ ACEPTABLE
$O(n \log n)$	→ Linealítmico	→ BUENO para sorting
$O(n^2)$	→ Cuadrático	→ MALO (evitar si puedes)
$O(2^n)$	→ Exponencial	→ FATAL (solo para n muy pequeño)

⚙️ ¿Cómo funciona?

$O(1)$ — Tiempo Constante Sin importar cuántos datos tengas, siempre tarda lo mismo. Ejemplo: acceder a `arr[5]` directamente por índice.

$O(\log n)$ — Tiempo Logarítmico Cada paso **DIVIDE** el problema a la mitad. Si tienes 1,024 elementos → necesitas máximo 10 pasos ($2^{10} = 1024$). Ejemplo: Binary search.

$O(n)$ — Tiempo Lineal Si duplicas el input, el tiempo se duplica. Ejemplo: recorrer todos los elementos de una lista una vez.

$O(n \log n)$ — Tiempo Linealítmico Un poco peor que lineal. La mayoría de buenos algoritmos de sorting. Ejemplo: Merge Sort.

$O(n^2)$ — Tiempo Cuadrático Dos loops anidados sobre el mismo input. Si duplicas el input, el tiempo se **CUADRUPLICA**. Ejemplo: Bubble Sort.

$O(2^n)$ — Tiempo Exponencial Cada elemento nuevo **DUPLICA** el trabajo. $n=30$ → más de 1,000,000,000 operaciones. Ejemplo: Fibonacci sin memoization.

```

1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 // O(1) - Constante
6 int obtenerPrimero(vector<int>& arr) {
7     return arr[0]; // acceso directo por índice → O(1)
8 }
9
10 // O(n) - Lineal
11 bool buscarLineal(vector<int>& arr, int objetivo) {
12     for (int i = 0; i < arr.size(); i++) {

```

```

13     if (arr[i] == objetivo) return true;
14 }
15 return false;
16 }
17
18 // O(n^2) - Cuadrático
19 bool tieneDuplicados(vector<int>& arr) {
20     int n = arr.size();
21     for (int i = 0; i < n; i++) {
22         for (int j = i + 1; j < n; j++) {
23             if (arr[i] == arr[j]) return true;
24         }
25     }
26     return false;
27 }
28
29 // O(log n) - Logarítmico
30 int busquedaBinaria(vector<int>& arr, int objetivo) {
31     int izq = 0;
32     int der = arr.size() - 1;
33     while (izq <= der) {
34         int medio = izq + (der - izq) / 2;
35         if (arr[medio] == objetivo) return medio;
36         if (arr[medio] < objetivo) izq = medio + 1;
37         else der = medio - 1;
38     }
39     return -1;
40 }

```

Complejidad Big-O

Notación & Nombre & Ejemplo real & ¿Usable?
$O(1)$ & Constante & Array access, hash lookup & Ideal
$O(\log n)$ & Logarítmico & Binary search, BST ops & Muy bueno
$O(n)$ & Lineal & Loop simple, linear search & Aceptable
$O(n \log n)$ & Linealítmico & Merge sort, Heap sort & Bueno
$O(n^2)$ & Cuadrático & Bubble sort, nested loops & Malo
$O(2^n)$ & Exponencial & Subsets, recursion fatal & Terrible

En entrevistas

- Big-O ignora constantes y términos menores: $O(3n + 5) \rightarrow O(n)$.
- Siempre menciona tiempo Y espacio.
- **Regla de oro:** Si ves que tu solución tiene $O(n^2)$ y el problema involucra búsqueda, piensa si un hash map te da $O(n)$ en su lugar.

 Resumen rápido

- Big-O = qué tan rápido crece el tiempo al crecer el input.
- Siempre describe el PEOR caso.
- $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$.
- Hash maps convierten muchos $O(n^2)$ en $O(n)$.

Capítulo 2

Complejidad Espacial (Space Complexity)

¿Qué es?

Imagina que tu programa es una receta de cocina. Big-O en tiempo mide cuánto TARDAS en cocinar. Complejidad espacial mide cuántas OLLAS Y SARTENES necesitas mientras cocinas.

Si para preparar una ensalada de 10 ingredientes necesitas una tabla de picar pequeña, pero para 100 ingredientes necesitas una mesa gigante, eso significa que el espacio crece con el número de ingredientes.

En programación, la complejidad espacial mide cuánta MEMORIA EXTRA (RAM) utiliza tu algoritmo para resolver un problema a medida que el input se hace más grande.

Dicho de otra forma

La Space Complexity analiza el uso de recursos de memoria. Es crucial distinguir entre:

1. **Input Space:** La memoria para guardar los datos de entrada.
2. **Auxiliary Space:** La memoria EXTRA o temporal que el algoritmo crea.

En la mayoría de las entrevistas, cuando preguntan "Space Complexity", se refieren al Auxiliary Space. Si tu algoritmo modifica el input original sin crear estructuras nuevas, se dice que es "In-place" y su complejidad espacial es $O(1)$.

DIAGRAMA

Cuando usas recursión, cada llamada ocupa un "piso" en el stack:

```
factorial(4)
|-- factorial(3)
    |-- factorial(2)
        |-- factorial(1) <-- Tope del stack
```

Cada nivel de recursión consume memoria adicional en el Call Stack.

⚙️ ¿Cómo funciona?

$O(1)$ — Espacio Constante Solo usas unas cuantas variables fijas sin importar el input. Ejemplo: Hacer un swap de dos números usando una variable temporal.

$O(n)$ — Espacio Lineal Creas una estructura (como un array o vector) proporcional al input. Ejemplo: Copiar una lista de n elementos a una nueva lista.

$O(n^2)$ — Espacio Cuadrático Creas una estructura bidimensional (matriz) de $n \times n$. Ejemplo: Una tabla de adyacencia para un grafo denso.

📊 Complejidad Big-O

Algoritmo & Espacio & Razón

Acceso a Array & $O(1)$ & No requiere memoria extra

Búsqueda Lineal & $O(1)$ & Solo usa un par de índices

Búsqueda Binaria iter. & $O(1)$ & Solo usa izq/der/mid

Búsqueda Binaria recur. & $O(\log n)$ & $\log(n)$ llamadas en el stack

Merge Sort & $O(n)$ & Necesita un array auxiliar

Quick Sort & $O(\log n)$ & Promedio de llamadas stack

Crear matriz $n \times n$ & $O(n^2)$ & Espacio bidimensional

🎯 En entrevistas

- En Google preguntan SIEMPRE espacio Y tiempo. No esperes a que pregunten.
- **"In-place algorithm"**: Significa que el algoritmo tiene $O(1)$ espacio auxiliar porque modifica el input original directamente.
- **Trade-off clásico**: A veces puedes hacer que un programa sea más rápido (tiempo) si estás dispuesto a usar más memoria (espacio). Ejemplo: Memoization en Dynamic Programming.

📋 Resumen rápido

- Space Complexity = cuánta RAM EXTRA usa el algoritmo.
- No cuenta el espacio del input original (Auxiliary Space).
- Estructuras de datos (arrays, maps) creadas $\rightarrow O(\text{tamaño})$.
- Recursión $\rightarrow$ cada nivel cuenta como $O(1)$ en el Call Stack.
- $O(1)$ espacio = "In-place", es lo más eficiente en memoria.

Estructuras Lineales

Capítulo 3

Arrays (Vectores)

¿Qué es?

Los vagones de un tren numerados en orden. El vagón 0, el vagón 1, el vagón 2... Cada vagón tiene exactamente UN pasajero (dato).

Para subir a cualquier vagón, solo dices el número (el índice) y ¡listo! No tienes que pasar por todos los vagones anteriores. Esto es posible porque todos los vagones están pegaditos uno tras otro en la vía.

Un array es una colección de elementos del mismo tipo guardados en posiciones de memoria CONTIGUAS (una tras otra).

```
1 El Array es la estructura de datos más básica y fundamental. Su
2 característica principal es el acceso aleatorio (Random Access) en
3 tiempo constante  $O(1)$ .
```

```
4
5 En lenguajes modernos como C++, solemos usar "Dynamic Arrays"
6 (como `std::vector`), que pueden crecer automáticamente si se llenan,
7 aunque por debajo siguen siendo un bloque de memoria contigua.
```

Diagrama

```
Índice: [0] [1] [2] [3] [4]
Valor:  [ 5][ 2][ 8][ 1][ 9]
```

↑

Dirección de memoria contigua →→→→→→→→→→

Insertar 99 en el índice 2:

ANTES: [5][2][8][1][9]

↓ ↓ ← desplazar →

DESPUÉS: [5][2][99][8][1][9] ← $O(n)$ porque movimos 8, 1 y 9

- ```
1 1. Acceso: Vas directo al índice `i`. Es instantáneo.
2 2. Búsqueda: Tienes que revisar uno por uno hasta encontrar el valor.
3 3. Inserción:
4 - Al final: Si hay espacio, es instantáneo.
5 - Al inicio/medio: Tienes que "empujar" a todos los de la derecha
6 un lugar para hacer espacio.
7 4. Eliminación:
8 - Al final: Borrás y ya.
9 - Al inicio/medio: Tienes que "jalar" a todos los de la derecha
```

un lugar a la izquierda para cerrar el hueco.

## PSEUDOCÓDIGO

---

*// Insertar en medio*

función insertar(**array**, valor, posicion):

**PARA** i desde tamaño hasta posicion + 1:

**array**[i] = **array**[i-1] *// desplazar a la derecha*

**array**[posicion] = valor

*// Eliminar en medio*

función eliminar(**array**, posicion):

**PARA** i desde posicion hasta tamaño - 2:

**array**[i] = **array**[i+1] *// desplazar a la izquierda*

## CÓDIGO C++

---

#include <iostream>

#include <vector>   *// En C++ usamos vector para arrays dinámicos*

#include <algorithm>

using namespace std;

int main() {

*// 1. Declaración e inicialización*

**vector**<int> arr = {5, 2, 8, 1, 9};

*// 2. Acceso por índice -> O(1)*

    cout << "Elemento en idx 2: " << arr[2] << endl; *// 8*

*// 3. Insertar al final -> O(1) amortizado*

    arr.push\_back(10); *// [5, 2, 8, 1, 9, 10]*

*// 4. Insertar en medio -> O(n)*

*// Insertamos el 99 en la posición 2*

    arr.insert(arr.begin() + 2, 99); *// [5, 2, 99, 8, 1, 9, 10]*

*// 5. Eliminar en medio -> O(n)*

*// Borramos el elemento en la posición 4*

    arr.erase(arr.begin() + 4); *// [5, 2, 99, 8, 9, 10]*

*// 6. Tamaño del array*

    cout << "Tamaño: " << arr.size() << endl;

*// 7. Recorrer el array -> O(n)*

**for**(int x : arr) {

        cout << x << " ";

    }

    cout << endl;

```

60 return 0;
61 }
62

```

63 COMPLEJIDAD (Big-O)

| Operación       | Tiempo      | Espacio |
|-----------------|-------------|---------|
| Access          | $O(1)$      | $O(1)$  |
| Search unsorted | $O(n)$      | $O(1)$  |
| Search sorted   | $O(\log n)$ | $O(1)$  |
| Insert end      | $O(1)^*$    | $O(1)$  |
| Insert middle   | $O(n)$      | $O(1)$  |
| Delete end      | $O(1)$      | $O(1)$  |
| Delete middle   | $O(n)$      | $O(1)$  |

64 \*amortizado en dynamic array (vector)



## En entrevistas

Arrays son la estructura de datos más común en entrevistas. Domínalos.

Patrones clave: - "Two Pointers": Usar un índice al inicio y otro al final. - "Sliding Window": Una ventana que se desliza para encontrar subarrays. - "Prefix Sum": Pre-calcular sumas para responder rangos en  $O(1)$ .

Off-by-one errors: El error más común es salirte del rango (acceder a `arr[n]` cuando el tamaño es `n`). Siempre usa '`i < n`'.

Pregunta clave: "¿El array está ordenado?". Si está ordenado, casi siempre la respuesta involucra Binary Search  $O(\log n)$ .

- 
- Array = bloque de memoria contigua.
- Acceso instantáneo  $O(1)$  si conoces el índice.
- Insertar/Eliminar en medio es LENTO  $O(n)$  por los desplazamientos.
- En C++: `std::vector` es el rey.
- Muy eficiente en espacio (mínimo overhead).
- Ideal para cuando conoces el tamaño o solo agregas/quitas al final.

# Capítulo 4

## Linked Lists (Listas Enlazadas)

### ¿Qué es?

Una cacería del tesoro . Cada pista (node) te dice dónde está la SIGUIENTE pista.

No sabes dónde está la pista #5 sin seguir la cadena desde el inicio: pista 1 → pista 2 → pista 3 → pista 4 → pista 5.

A diferencia de los arrays (vagones pegaditos), los nodos de una Linked List pueden estar en cualquier parte de la memoria. Lo único que los mantiene unidos es que cada uno sabe la dirección (pointer) del siguiente.

### Dicho de otra forma

Una Linked List es una estructura lineal donde los elementos no se guardan en posiciones contiguas de memoria. Cada elemento es un objeto llamado "Node", que contiene: 1. El valor (data). 2. Un puntero (next) al siguiente nodo.

Esto permite insertar y eliminar elementos en los extremos en  $O(1)$  sin tener que desplazar a nadie, pero sacrificas el acceso aleatorio (no puedes ir directo al índice 'i').

### Diagrama

Singly Linked List:

HEAD

↓

[3|•] → [7|•] → [1|•] → [9|NULL]  
node0   node1   node2   node3

Doubly Linked List:

HEAD

TAIL

↓

↓

[NULL|3|•] <-> [•|7|•] <-> [•|1|•] <-> [•|9|NULL]

Insertar 5 después del node1 (valor 7):

ANTES: [3] → [7] → [1] → [9]

↓

DESPUÉS: [3] → [7] → [5] → [1] → [9] ←  $O(1)$  si ya tienes el puntero

1. Acceso/Búsqueda: Tienes que empezar en el HEAD e ir saltando de



```

2 nodo en nodo. $O(n)$.
3 2. Inserción:
4 - Al Head: Creas nodo, apuntas su `next` al viejo head, actualizas
5 head. $O(1)$.
6 - En medio: Llegas a la posición, ajustas los punteros para incluir
7 el nuevo nodo.
8 3. Eliminación:
9 - Del Head: Mueves el head al `head->next`. $O(1)$.
10 - En medio: Ajustas el puntero del nodo anterior para que "salte"
11 al nodo que quieres borrar.

```

### PSEUDOCÓDIGO

---

```

14
15
16 // Insertar al inicio
17 función insertarAlInicio(valor):
18 nuevoNodo = crear Nodo(valor)
19 nuevoNodo.next = head
20 head = nuevoNodo
21
22 // Buscar valor
23 función buscar(valor):
24 actual = head
25 MIENTRAS actual != NULL:
26 SI actual.valor == valor: return VERDADERO
27 actual = actual.next
28 return FALSO
29

```

### CÓDIGO C++

---

```

31
32
33 #include <iostream>
34 using namespace std;
35
36 // Definición del Nodo
37 struct Node {
38 int data;
39 Node* next;
40 Node(int val) : data(val), next(nullptr) {} // Constructor
41 };
42
43 class LinkedList {
44 public:
45 Node* head;
46 LinkedList() : head(nullptr) {}
47
48 // Insertar al inicio -> $O(1)$
49 void insertHead(int val) {
50 Node* newNode = new Node(val);
51 newNode->next = head;

```

```
52 head = newNode;
53 }
54
55 // Insertar al final -> O(n) (sin tail pointer)
56 void insertTail(int val) {
57 Node* newNode = new Node(val);
58 if (!head) {
59 head = newNode;
60 return;
61 }
62 Node* temp = head;
63 while (temp->next) temp = temp->next; // Recorrer hasta el final
64 temp->next = newNode;
65 }
66
67 // Buscar -> O(n)
68 bool search(int val) {
69 Node* temp = head;
70 while (temp) {
71 if (temp->data == val) return true;
72 temp = temp->next;
73 }
74 return false;
75 }
76
77 // Imprimir lista
78 void print() {
79 Node* temp = head;
80 while (temp) {
81 cout << temp->data << " -> ";
82 temp = temp->next;
83 }
84 cout << "NULL" << endl;
85 }
86 };
87
88 int main() {
89 LinkedList list;
90 list.insertHead(10);
91 list.insertHead(20);
92 list.insertTail(5);
93
94 list.print(); // 20 -> 10 -> 5 -> NULL
95
96 cout << "¿Está el 10? " << (list.search(10) ? "Sí" : "No") << endl;
97
98 return 0;
99 }
```

COMPLEJIDAD (Big-O)

---

| Operación         | Tiempo   | vs Array                                |
|-------------------|----------|-----------------------------------------|
| Access por índice | $O(n)$   | Array: $O(1)$ ← <b>array</b> gana       |
| Search            | $O(n)$   | Igual                                   |
| Insert head       | $O(1)$   | Array: $O(n)$ ← linked <b>list</b> gana |
| Insert tail       | $O(1)^*$ | Array: $O(1)$ amort. → Igual            |
| Insert medio      | $O(n)$   | Igual (pero no desplaza datos)          |
| Delete head       | $O(1)$   | Array: $O(n)$ ← linked <b>list</b> gana |
| *con tail pointer |          |                                         |

## En entrevistas

Patrón "Fast & Slow Pointers" (Tortoise and Hare): - Sirve para detectar ciclos (si se encuentran, hay ciclo). - Sirve para encontrar el medio (cuando fast llega al final, slow está en el medio).

Dummy Node: Crear un nodo falso al inicio ayuda a simplificar casos borde (como insertar en una lista vacía o borrar el head).

Memory Management: En C++, recuerda que cada 'new' necesita un 'delete' para evitar memory leaks (fugas de memoria).

Pregunta clave: "¿Es singly o doubly?". Si es doubly, puedes ir hacia atrás, lo que facilita borrar un nodo si ya tienes el puntero.

## Resumen rápido

- Cadena de nodos dispersos en memoria.
- Acceso secuencial (no puedes saltar).
- Excelente para insertar/eliminar al inicio  $O(1)$ .
- No desperdicia memoria (solo usa lo que necesita).
- Invertir una Linked List es un problema clásico (usar 3 punteros).
- Usa más memoria por elemento que un array (por el puntero next).

# Capítulo 5

## Fast & Slow Pointers (Tortuga y Liebre)

### ¿Qué es?

Dos corredores en una pista circular : uno lento (tortuga, avanza 1 paso) y uno rápido (liebre, avanza 2 pasos). Si la pista tiene un bucle, tarde o temprano el rápido le da la vuelta y ALCANZA al lento por detrás. Si la pista termina (hay una meta), el rápido llega al final y nunca se encuentran.

Ese simple truco resuelve un montón de problemas de listas enlazadas: detectar ciclos, encontrar el nodo del medio y hallar dónde empieza el bucle, todo sin usar memoria extra.

### Dicho de otra forma

La técnica Fast & Slow (algoritmo de la tortuga y la liebre de Floyd) usa dos punteros que recorren la lista a velocidades distintas: uno avanza de 1 en 1 y otro de 2 en 2.

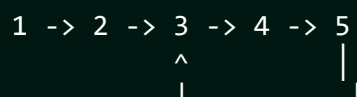
Sus tres usos clásicos:

- **Detectar ciclo:** si en algún momento `fast == slow`, hay ciclo. Si `fast` llega a NULL, no lo hay.
- **Encontrar el medio:** cuando `fast` llega al final, `slow` está justo a la mitad.
- **Inicio del ciclo:** tras el encuentro, mueve un puntero al `head` y avanza ambos de 1 en 1; se cruzan en el inicio del bucle.

Todo en  $O(n)$  tiempo y  $O(1)$  espacio: ahí está su valor.

### Diagrama

Lista con ciclo:



slow (1 paso): 1 2 3 4 5 3 4 ...  
fast (2 pasos): 1 3 5 4 3 5 4 ...

encuentro en el nodo 4 => HAY CICLO

Encontrar el medio (lista sin ciclo):

1 -> 2 -> 3 -> 4 -> 5 -> NULL

fast llega a NULL => slow queda en 3 (el medio)

```

1 DETECTAR CICLO
2 - slow = head, fast = head.
3 - Avanza slow 1 y fast 2 mientras fast y fast->next existan.
4 - Si slow == fast en algún punto -> hay ciclo.
5
6 INICIO DEL CICLO (después del encuentro)
7 - Deja un puntero en el punto de encuentro, mueve otro al head.
8 - Avanza ambos de 1 en 1; donde se crucen es el inicio del ciclo.
9
10 PSEUDOCÓDIGO (detectar + inicio de ciclo)

11
12
13 función detectarCiclo(head):
14 slow = head; fast = head
15 MIENTRAS fast Y fast.next:
16 slow = slow.next
17 fast = fast.next.next
18 SI slow == fast: // hay ciclo
19 p = head
20 MIENTRAS p != slow:
21 p = p.next; slow = slow.next
22 return p // inicio del ciclo
23 return NULL // no hay ciclo
24
25 CÓDIGO C++

26
27
28 struct Node { int val; Node* next; };
29
30 // ¿Tiene ciclo?
31 bool hasCycle(Node* head) {
32 Node *slow = head, *fast = head;
33 while (fast && fast->next) {
34 slow = slow->next;
35 fast = fast->next->next;
36 if (slow == fast) return true; // se alcanzaron
37 }
38 return false; // fast llegó a NULL
39 }
40
41 // Nodo del medio
42 Node* middleNode(Node* head) {
43 Node *slow = head, *fast = head;
44 while (fast && fast->next) {

```

```

45 slow = slow->next;
46 fast = fast->next->next;
47 }
48 return slow; // mitad exacta (o 2da mitad si
 par)
49 }

```

50  
51 COMPLEJIDAD (Big-O)

| Problema         | Tiempo | Espacio |
|------------------|--------|---------|
| Detectar ciclo   | $O(n)$ | $O(1)$  |
| Encontrar medio  | $O(n)$ | $O(1)$  |
| Inicio del ciclo | $O(n)$ | $O(1)$  |

59  
60 \* La gran ventaja es el  $O(1)$  de memoria (sin HashSet de nodos vistos).



Otros usos del patrón:

- Palíndromo de lista: encuentra el medio, invierte la 2da mitad y compara.
- Happy Number: aplica fast/slow sobre la secuencia de sumas de cuadrados de dígitos para detectar el ciclo.
- Eliminar el N-ésimo desde el final: separa dos punteros por N nodos y avánzalos juntos.



## En entrevistas

”Linked List Cycle” (LeetCode 141/142): 141 pide detectar, 142 pide el nodo donde empieza. Ambos son Floyd puro.

¿Por qué funciona lo del inicio? La distancia del head al inicio del ciclo es igual a la distancia del punto de encuentro al inicio del ciclo (avanzando en el bucle). Sabértelo justifica el algoritmo.

Cuida los NULL: siempre valida **fast != NULL** Y **fast->next != NULL** antes de hacer **fast->next->next**, o provocas un segfault.

Alternativa con HashSet: guardar nodos visitados también detecta ciclos, pero usa  $O(n)$  memoria. Floyd es superior por su  $O(1)$ .



## Resumen rápido

- Dos punteros: lento (1 paso) y rápido (2 pasos).
- Se encuentran = hay ciclo; fast llega a NULL = no hay.
- fast al final -> slow queda en el medio.
- Inicio del ciclo: un puntero al head, avanzar ambos de 1 en 1.
- $O(n)$  tiempo,  $O(1)$  espacio (mejor que Hash-)

---

Set). • Clásicos: cycle detection, medio, palíndromo, happy number.

---

---

# Capítulo 6

## Patrones de Linked List

### ¿Qué es?

La mayoría de los problemas de listas enlazadas en entrevistas se reducen a "manipular punteros con cuidado". Tres patrones cubren casi todo:

- Invertir: darle la vuelta a las flechas, una por una.
- Fusionar dos listas ordenadas: como cerrar un cierre, tomando cada vez el elemento más pequeño de las dos.
- Reordenar: partir, invertir e intercalar.

El secreto siempre es el mismo: usa un **dummy node** (nodo falso al inicio) y muévete con punteros temporales para no perder el resto de la cadena.

### Dicho de otra forma

Los patrones más rentables sobre listas enlazadas:

- **Reverse**: tres punteros (**prev**, **cur**, **next**). En cada paso, apunta **cur->next** hacia atrás y avanza.  $O(n)$  tiempo,  $O(1)$  espacio.
- **Merge Two Sorted Lists**: un dummy y un puntero **tail**; vas enganchando el menor de las dos cabezas. Base del Merge Sort sobre listas.
- **Reorder / intercalar**: encuentra el medio (fast & slow), invierte la segunda mitad y entrelaza ambas.
- **Remove N-th from end**: dos punteros separados por N; cuando el de adelante llega al final, el de atrás está justo antes del objetivo.

El **dummy node** evita casos borde al modificar la cabeza.

### Diagrama

REVERSE (3 punteros):

```
NULL <- 1 2 -> 3 -> NULL prev=NULL cur=1
 ^ ^
 prev cur
```

guarda next, apunta **cur->next=prev**, avanza

Resultado: NULL <- 1 <- 2 <- 3 (head ahora es 3)

MERGE dos listas ordenadas:

```
L1: 1 -> 3 -> 5 dummy -> 1 -> 2 -> 3 -> 4 -> 5 -> 6
```



L2: 2 -> 4 -> 6      (toma el menor de cada cabeza en cada paso)

```

1 REVERSE (iterativo, 3 punteros)
2 - prev=NULL, cur=head. En cada paso: guarda next, cur->next=prev,
3 prev=cur, cur=next. Al final prev es la nueva cabeza.
4
5 MERGE (con dummy)
6 - dummy y tail. Mientras ambas listas tengan nodos, engancha el menor
7 y avanza esa lista. Al final pega la que sobró.
8
9 PSEUDOCÓDIGO (reverse)

10
11
12 función reverse(head):
13 prev = NULL; cur = head
14 MIENTRAS cur != NULL:
15 sig = cur.next // guarda el resto
16 cur.next = prev // voltea la flecha
17 prev = cur // avanza prev
18 cur = sig // avanza cur
19 return prev // nueva cabeza
20
21 CÓDIGO C++

22
23
24 struct Node { int val; Node* next; };
25
26 // Invertir -> O(n) tiempo, O(1) espacio
27 Node* reverseList(Node* head) {
28 Node* prev = nullptr;
29 while (head) {
30 Node* next = head->next;
31 head->next = prev;
32 prev = head;
33 head = next;
34 }
35 return prev;
36 }
37
38 // Fusionar dos listas ordenadas
39 Node* mergeTwoLists(Node* a, Node* b) {
40 Node dummy{0, nullptr};
41 Node* tail = &dummy;
42 while (a && b) {
43 if (a->val <= b->val) { tail->next = a; a = a->next; }
44 else { tail->next = b; b = b->next; }
45 tail = tail->next;
46 }
47 tail->next = a ? a : b; // engancha lo que sobró
48 return dummy.next;

```

```

49 }
50
51 // Eliminar el N-ésimo desde el final (two pointers)
52 Node* removeNthFromEnd(Node* head, int n) {
53 Node dummy{0, head};
54 Node *fast = &dummy, *slow = &dummy;
55 for (int i = 0; i < n; i++) fast = fast->next; // separa N
56 while (fast->next) { fast = fast->next; slow = slow->next; }
57 slow->next = slow->next->next; // salta el
 objetivo
58 return dummy.next;
59 }

```

60 COMPLEJIDAD (Big-O)

| Patrón          | Tiempo   | Espacio |
|-----------------|----------|---------|
| Reverse         | $O(n)$   | $O(1)$  |
| Merge 2 sorted  | $O(n+m)$ | $O(1)$  |
| Reorder         | $O(n)$   | $O(1)$  |
| Remove N-th end | $O(n)$   | $O(1)$  |



### En entrevistas

"Reverse Linked List" (206), "Merge Two Sorted Lists" (21), "Reorder List" (143), "Remove Nth Node From End" (19): el póker de listas. Domínalos y cubres la mayoría de preguntas de listas.

Dummy node siempre: cuando la operación puede cambiar la cabeza (borrar el primer nodo, fusionar), un nodo falso al inicio elimina el caso especial y limpia el código.

No pierdas la referencia: antes de reasignar `cur->next`, SIEMPRE guarda el siguiente en una variable temporal, o pierdes el resto de la lista.

Merge K listas: usa una min-heap con las K cabezas  $\rightarrow O(N \log K)$ . Es la generalización de "Merge Two Lists" que suele venir después.



### Resumen rápido

- Reverse: 3 punteros (prev, cur, next); voltea flechas;  $O(1)$  espacio.
- Merge ordenadas: dummy + tail, engancha el menor cada vez.
- Reorder: medio (fast/slow) + invertir 2da mitad + intercalar.
- Remove N-th: dos punteros separados por N.
- Usa dummy node cuando la cabeza pueda cambiar.
- Guarda el 'next' antes de reasignar punteros.

# Capítulo 7

## Stacks (Pilas)

### ¿Qué es?

Una pila de platos en el fregadero. Solo puedes poner platos encima (push) y solo puedes agarrar el de hasta arriba (pop).

Si quieres el plato de abajo, debes quitar todos los de encima primero. Es una estructura "LIFO" (Last In, First Out): el último en entrar es el primero en salir.

Imagina un tubo de papas Pringles : la última papa que metieron en la fábrica es la primera que te comes al abrir el tubo.

### Dicho de otra forma

Un Stack es una estructura de datos lineal de tipo ADT (Abstract Data Type) que restringe el acceso a un solo extremo: el "Top".

Es fundamental para el funcionamiento de las computadoras, ya que el "Call Stack" de los lenguajes de programación usa esta lógica para gestionar las llamadas a funciones y la recursión.

### Diagrama

```
push(A) → push(B) → push(C)
```

```
[C] ← TOP (C es el último en entrar)
[B]
[A] ← BOTTOM
```

```
pop() → devuelve C y lo quita, queda:
```

```
[B] ← TOP (ahora B es el nuevo tope)
[A]
```

1. `push(x)`: Agregas el elemento `x` al tope de la pila.  $O(1)$ .
2. `pop()`: Quitas el elemento del tope.  $O(1)$ .
3. `peek()` / `top()`: Miras qué hay en el tope sin quitarlo.  $O(1)$ .
4. `isEmpty()`: Verificas si la pila está vacía.  $O(1)$ .
5. `size()`: Devuelves cuántos elementos hay.  $O(1)$ .

6  
7

PSEUDOCÓDIGO

```
8
9
10 // Validar paréntesis balanceados usando un Stack
```

```
11 función esValido(cadena):
```

```
12 pila = crear Stack
```

```
13 PARA cada caracter EN cadena:
```

```
14 SI caracter == '(':
```

```
15 pila.push('(')
```

```
16 SINO SI caracter == ')':
```

```
17 SI pila.isEmpty(): return FALSO
```

```
18 pila.pop()
```

```
19 return pila.isEmpty()
```

```
20
21 CÓDIGO C++
```

```
22
23
24 #include <iostream>
```

```
25 #include <stack> // STL Stack
```

```
26 #include <vector>
```

```
27 using namespace std;
```

```
28
29 // Implementación manual simple usando un vector
```

```
30 class MyStack {
```

```
31 private:
```

```
32 vector<int> data;
```

```
33 public:
```

```
34 void push(int x) { data.push_back(x); }
```

```
35 void pop() { if(!data.empty()) data.pop_back(); }
```

```
36 int top() { return data.back(); }
```

```
37 bool isEmpty() { return data.empty(); }
```

```
38 };
```

```
39
40 int main() {
```

```
41 // 1. Usando STL Stack
```

```
42 stack<int> s;
```

```
43
44 s.push(10);
```

```
45 s.push(20);
```

```
46 s.push(30);
```

```
47
48 cout << "Tope actual: " << s.top() << endl; // 30
```

```
49
50 s.pop(); // Quita el 30
```

```
51 cout << "Nuevo tope: " << s.top() << endl; // 20
```

```
52
53 // 2. Recorrer (vaciando)
```

```
54 while(!s.empty()) {
```

```
55 cout << s.top() << " ";
```

```
56 s.pop();
```

```
57 }
```

```
58 cout << endl;
```

|    |                             |        |         |
|----|-----------------------------|--------|---------|
| 59 |                             |        |         |
| 60 | <code>return 0;</code>      |        |         |
| 61 | <code>}</code>              |        |         |
| 62 |                             |        |         |
| 63 | COMPLEJIDAD (Big-O)         |        |         |
| 64 |                             |        |         |
| 65 |                             |        |         |
| 66 | Operación                   | Tiempo | Espacio |
| 67 |                             |        |         |
| 68 | <code>push(x)</code>        | $O(1)$ | $O(1)$  |
| 69 | <code>pop()</code>          | $O(1)$ | $O(1)$  |
| 70 | <code>peek() / top()</code> | $O(1)$ | $O(1)$  |
| 71 | <code>isEmpty()</code>      | $O(1)$ | $O(1)$  |
| 72 | Total (n datos)             | -      | $O(n)$  |

### En entrevistas

Problema de Oro: "Valid Parentheses". Si ves paréntesis, corchetes o llaves en un problema, ¡usa un stack!

Recursión vs Stack: Cualquier problema recursivo se puede resolver usando un Stack de forma iterativa. A veces esto evita el Stack Overflow.

Stack Underflow: Intentar hacer '`pop()`' o '`top()`' en una pila vacía. SIEMPRE verifica '`!s.empty()`' antes de acceder al tope.

Patrón "Monotonic Stack": Un stack donde los elementos siempre están en orden (creciente o decreciente). Clave para problemas como "Next Greater Element" o "Daily Temperatures".

### Resumen rápido

- LIFO: Last In, First Out.
- Todas las operaciones básicas son  $O(1)$  (instantáneas).
- Usos: Historial del navegador (Undo/Redo), procesar expresiones matemáticas, DFS (Depth First Search).
- Muy eficiente pero limitado: solo ves el tope.
- La base de la recursión en cualquier lenguaje.

# Capítulo 8

## Queues (Filas / Colas)

🤔 ¿Qué es?

La fila para entrar al cine o al banco . El primero que llega es el primero en ser atendido. No puedes "colarte" al frente; llegas al final y esperas tu turno.

Es una estructura "FIFO" (First In, First Out): el primero que entra es el primero en salir. Es lo opuesto al Stack.

 Dicho de otra forma

Una Queue es una estructura lineal que restringe el acceso: - Solo puedes AGREGAR (Enqueue) por el final (Back/Rear). - Solo puedes QUITAR (Dequeue) por el frente (Front).

Existen variaciones importantes: 1. Deque: Puedes insertar/eliminar por ambos extremos. 2. Priority Queue: Los elementos salen según su importancia, no solo por orden de llegada (implementada usualmente con un Heap).

## Diagrama

enqueue(A) → enqueue(B) → enqueue(C)

FRONT  
↓  
[A] → [B] → [C]

dequeue() → devuelve A y lo quita, queda:

FRONT                      BACK

↓                                      ↓

[B] → [C]

1. enqueue(x): Agregas el elemento `x` al final de la fila. O(1).
2. dequeue(): Quitas el primer elemento del frente. O(1).
3. front() / peek(): Miras quién es el primero sin quitarlo. O(1).
4. isEmpty(): Verificas si hay alguien en la fila. O(1).
5. size(): Devuelves cuántas personas hay esperando. O(1).

## PSEUDOCÓDIGO

```

9
10 // BFS (Breadth First Search) básico usando una Queue
11 función BFS(inicio):
12 fila = crear Queue
13 fila.enqueue(inicio)
14 marcar inicio como visitado
15
16 MIENTRAS fila NO esté vacía:
17 actual = fila.dequeue()
18 PARA cada vecino de actual:
19 SI vecino NO visitado:
20 fila.enqueue(vecino)
21 marcar vecino como visitado
22
23 CÓDIGO C++
24
25
26 #include <iostream>
27 #include <queue> // STL Queue
28 #include <deque> // STL Deque (Double-ended queue)
29 using namespace std;
30
31 int main() {
32 // 1. Queue estándar (FIFO)
33 queue<int> q;
34
35 q.push(10); // Enqueue
36 q.push(20);
37 q.push(30);
38
39 cout << "Primero en la fila: " << q.front() << endl; // 10
40 cout << "Último en la fila: " << q.back() << endl; // 30
41
42 q.pop(); // Dequeue (quita el 10)
43 cout << "Nuevo primero: " << q.front() << endl; // 20
44
45 // 2. Deque (Doble extremo)
46 deque<int> dq;
47 dq.push_back(100);
48 dq.push_front(50); // [50, 100]
49 dq.pop_back(); // [50]
50
51 cout << "Frente de deque: " << dq.front() << endl;
52
53 return 0;
54 }
55
56 COMPLEJIDAD (Big-O)

```

|    |                  |        |         |
|----|------------------|--------|---------|
| 59 | Operación        | Tiempo | Espacio |
| 60 |                  |        |         |
| 61 | enqueue(x)       | $O(1)$ | $O(1)$  |
| 62 | dequeue()        | $O(1)$ | $O(1)$  |
| 63 | front() / peek() | $O(1)$ | $O(1)$  |
| 64 | isEmpty()        | $O(1)$ | $O(1)$  |
| 65 | Total (n datos)  | -      | $O(n)$  |



### En entrevistas

Sin Queue no hay BFS: Si el problema pide "camino más corto" en un grafo sin pesos o explorar un árbol por niveles, vas a usar una Queue.

Priority Queue: Úsala cuando necesites el "máximo" o "mínimo" elemento de forma constante mientras insertas nuevos datos.

Implementación con Array: Cuidado, si implementas una Queue con un Array simple, el 'dequeue' podría ser  $O(n)$  si tienes que mover todos los elementos. Por eso se usan "Circular Queues".

"Sliding Window Maximum": Un problema clásico de Google que se resuelve de forma óptima usando un Deque.



### Resumen rápido

- FIFO: First In, First Out.
- Ideal para: Procesamiento de tareas, buffers, algoritmos de grafos como BFS.
- Deque: Permite meter/sacar por ambos lados (muy flexible).
- Priority Queue: Saca al que tiene más "rango", no al que llegó primero.



## Estructuras No Lineales

# Capítulo 9

## 🌳 Binary Search Trees (Árboles Binarios de Búsqueda)

### 🤖 ¿Qué es?

Un árbol genealógico de cabeza. La raíz (root) está hasta arriba. Cada persona (nodo) puede tener máximo dos hijos.

Pero es un árbol inteligente con una regla de oro: - Todo lo que está a la IZQUIERDA de un nodo es MENOR. - Todo lo que está a la DERECHA de un nodo es MAYOR.

Es como una biblioteca donde los libros están tan bien ordenados que puedes encontrar cualquiera en segundos, sin revisar todos los estantes.

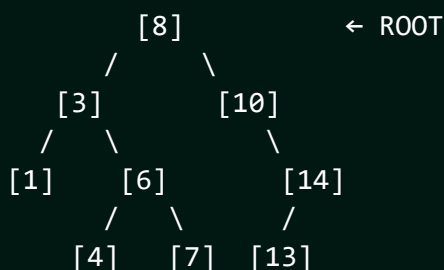
### 💬 Dicho de otra forma

Un Binary Search Tree (BST) es una estructura de datos no lineal formada por nodos. Cada nodo contiene un valor y punteros a sus hijos izquierdo y derecho.

La propiedad de orden ( $\text{izq} < \text{raíz} < \text{der}$ ) permite que las operaciones de búsqueda, inserción y eliminación se realicen en tiempo logarítmico  $O(\log n)$ , siempre y cuando el árbol esté balanceado.

### 🧐 Diagrama

BST de ejemplo:



- Leaf Nodes (hojas): 1, 4, 7, 13 (no tienen hijos).
- Height (altura): 3 (niveles desde la raíz).

1. Search: Comparas con la raíz. ¿Es menor? Vas a la izquierda.
2. ¿Es mayor? Vas a la derecha. Repites hasta encontrarlo o llegar a un callejón sin salida (NULL).
- 3.

- 4 2. Insert: Buscas el lugar donde debería estar el valor siguiendo la
- 5 regla de orden y lo agregas como una nueva hoja.
- 6 3. Delete: Es la más compleja. Tiene 3 casos:
- 7 - Nodo es una hoja: Solo lo borras.
- 8 - Nodo tiene 1 hijo: El hijo sube a ocupar el lugar del padre.
- 9 - Nodo tiene 2 hijos: Buscas al sucesor (el más pequeño del lado
- 10 derecho) para que ocupe su lugar.
- 11 4. Traversal (Recorrido):
- 12 - Inorder: Izquierda -> Raíz -> Derecha (¡Te da los números en orden!
- 13 ).

#### 14 PSEUDOCÓDIGO

---

```

15
16
17 // Búsqueda recursiva en un BST
18 función buscar(nodo, valor):
19 SI nodo es NULL: return FALSO
20 SI nodo.valor == valor: return VERDADERO
21 SI valor < nodo.valor:
22 return buscar(nodo.izq, valor)
23 SINO:
24 return buscar(nodo.der, valor)
25

```

#### 26 CÓDIGO C++

---

```

27
28
29 #include <iostream>
30 using namespace std;
31
32 struct Node {
33 int data;
34 Node *left, *right;
35 Node(int val) : data(val), left(nullptr), right(nullptr) {}
36 };
37
38 class BST {
39 public:
40 Node* root;
41 BST() : root(nullptr) {}
42
43 // Insertar -> O(log n) promedio
44 Node* insert(Node* node, int val) {
45 if (!node) return new Node(val);
46 if (val < node->data)
47 node->left = insert(node->left, val);
48 else
49 node->right = insert(node->right, val);
50 return node;
51 }
52

```

```

53 // Búsqueda -> O(log n) promedio
54 bool search(Node* node, int val) {
55 if (!node) return false;
56 if (node->data == val) return true;
57 if (val < node->data) return search(node->left, val);
58 return search(node->right, val);
59 }
60
61 // Inorder Traversal -> O(n)
62 void inorder(Node* node) {
63 if (!node) return;
64 inorder(node->left);
65 cout << node->data << " ";
66 inorder(node->right);
67 }
68 };
69
70 int main() {
71 BST tree;
72 tree.root = tree.insert(tree.root, 8);
73 tree.insert(tree.root, 3);
74 tree.insert(tree.root, 10);
75 tree.insert(tree.root, 1);
76 tree.insert(tree.root, 6);
77
78 cout << "Inorder: ";
79 tree.inorder(tree.root); // 1 3 6 8 10
80 cout << endl;
81
82 cout << "¿Está el 6? " << (tree.search(tree.root, 6) ? "Sí" : "No")
83 << endl;
84
85 return 0;
86 }

```

#### COMPLEJIDAD (Big-O)

| Operación | Promedio    | Peor caso (Degenerado) |
|-----------|-------------|------------------------|
| Search    | $O(\log n)$ | $O(n)$                 |
| Insert    | $O(\log n)$ | $O(n)$                 |
| Delete    | $O(\log n)$ | $O(n)$                 |
| Traversal | $O(n)$      | $O(n)$                 |
| Espacio   | $O(n)$      | $O(n)$                 |

\* Peor caso ocurre cuando el árbol se vuelve una línea (como linked list).

## En entrevistas

"Inorder Traversal = Sorted Array": Si recorres un BST en modo Inorder, siempre obtendrás los elementos ordenados de menor a mayor.

Lowest Common Ancestor (LCA): Un problema de Google súper común. En un BST es fácil: el LCA es el primer nodo cuyo valor está ENTRE los dos valores que buscas.

Balanceo: Un BST normal puede volverse muy ineficiente ( $O(n)$ ) si insertas los números ya ordenados. Para evitarlo existen los Self-balancing Trees (como AVL).

Pregunta clave: "¿Es un Binary Tree o un Binary Search Tree?". El segundo tiene la propiedad de orden, el primero no.

## Resumen rápido

- Izquierda menor, Derecha mayor.
- Búsqueda rápida  $O(\log n)$  como adivinar un número.
- Inorder traversal = lista ordenada.
- El peor caso es  $O(n)$  si el árbol no está balanceado.
- Términos: Root (cima), Leaf (hoja), Parent (padre), Child (hijo).
- Altura del árbol = longitud del camino más largo a una hoja.

---

---

# Capítulo 10

## AVL Trees (Árboles Binarios Balanceados)

### ¿Qué es?

Un móvil de bebé . Si pones mucho peso de un lado, el móvil se inclina y queda chueco. Para que funcione bien, debes mover las piezas hasta que quede perfectamente equilibrado.

Un árbol AVL es un Binary Search Tree (BST) que tiene una "manía" por el orden: no permite que una rama sea mucho más larga que otra. Si al insertar un número el árbol se desbalancea, el árbol se "tuerce" a sí mismo (rotación) para volver al equilibrio.

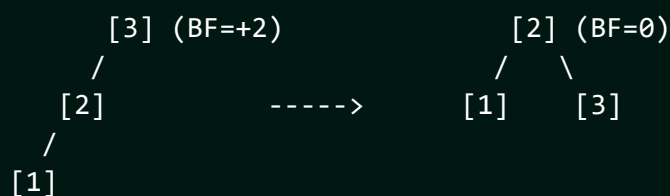
### Dicho de otra forma

El AVL es el primer árbol auto-balanceable de la historia. Su regla es estricta: para cada nodo, la diferencia de altura entre su hijo izquierdo y derecho (llamada Balance Factor) solo puede ser -1, 0 o 1.

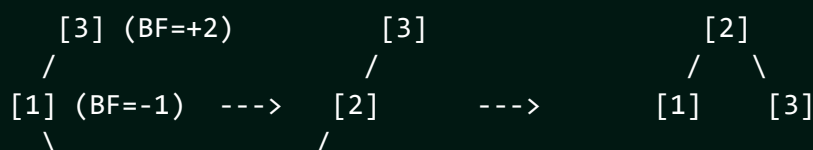
Si la diferencia es 2 o -2, el árbol realiza rotaciones (Simples o Dobles) para restaurar el balance. Esto garantiza que la altura del árbol sea siempre  $O(\log n)$ , evitando el peor caso de los BST normales.

### Diagrama

Rotación Simple a la Derecha (LL Rotation):



Rotación Doble (LR Rotation):



[2]

[1]

1. Insert: Igual que un BST, pero después de insertar, el árbol revisa la altura de los nodos hacia arriba. Si encuentra uno desbalanceado, aplica una de las 4 rotaciones (LL, RR, LR, RL).
2. Delete: También igual que BST, pero requiere revisar el balance hacia arriba, pudiendo causar múltiples rotaciones.
3. Search: Exactamente igual que en un BST, pero con la garantía de que siempre será rápido ( $O(\log n)$ ).

#### PSEUDOCÓDIGO

---

```
// Obtener factor de balance
función obtenerBalance(nodo):
 SI nodo es NULL: return 0
 return altura(nodo.izq) - altura(nodo.der)

// Rotación a la derecha
función rotarDerecha(y):
 x = y.izq
 T2 = x.der
 x.der = y
 y.izq = T2
 actualizarAlturas(y, x)
 return x
```

#### CÓDIGO C++

---

```
#include <iostream>
#include <algorithm>
using namespace std;

struct Node {
 int key, height;
 Node *left, *right;
 Node(int k) : key(k), height(1), left(nullptr), right(nullptr) {}
};

int getHeight(Node* n) { return n ? n->height : 0; }

Node* rightRotate(Node* y) {
 Node* x = y->left;
 Node* T2 = x->right;
 x->right = y;
 y->left = T2;
 y->height = max(getHeight(y->left), getHeight(y->right)) + 1;
 x->height = max(getHeight(x->left), getHeight(x->right)) + 1;
 return x;
}
```

```

49 }
50
51 Node* leftRotate(Node* x) {
52 Node* y = x->right;
53 Node* T2 = y->left;
54 y->left = x;
55 x->right = T2;
56 x->height = max(getHeight(x->left), getHeight(x->right)) + 1;
57 y->height = max(getHeight(y->left), getHeight(y->right)) + 1;
58 return y;
59 }
60
61 Node* insert(Node* node, int key) {
62 if (!node) return new Node(key);
63 if (key < node->key) node->left = insert(node->left, key);
64 else if (key > node->key) node->right = insert(node->right, key);
65 else return node;
66
67 node->height = 1 + max(getHeight(node->left), getHeight(node->right)
68);
69 int balance = getHeight(node->left) - getHeight(node->right);
70
71 if (balance > 1 && key < node->left->key) return rightRotate(node);
72 if (balance < -1 && key > node->right->key) return leftRotate(node);
73 if (balance > 1 && key > node->left->key) {
74 node->left = leftRotate(node->left);
75 return rightRotate(node);
76 }
77 if (balance < -1 && key < node->right->key) {
78 node->right = rightRotate(node->right);
79 return leftRotate(node);
80 }
81 return node;
82 }
83
84 int main() {
85 Node* root = nullptr;
86 root = insert(root, 10);
87 root = insert(root, 20);
88 root = insert(root, 30); // Esto causará una rotación RR
89
90 cout << "Raíz del árbol balanceado: " << root->key << endl; // Deber
91 ía ser 20
92 return 0;
93 }
94
95 COMPLEJIDAD (Big-O)
96

```

| Operación | Tiempo Garantizado |
|-----------|--------------------|
|           |                    |



|     |                                                                               |
|-----|-------------------------------------------------------------------------------|
| 97  |                                                                               |
| 98  | Search   $O(\log n)$                                                          |
| 99  | Insert   $O(\log n)$                                                          |
| 100 | Delete   $O(\log n)$                                                          |
| 101 | Espacio   $O(n)$                                                              |
| 102 |                                                                               |
| 103 | * A diferencia del BST normal, el peor caso en AVL sigue siendo $O(\log n)$ . |

1 

2 ¿Por qué AVL y no BST?: La respuesta siempre es **"Balance Garantizado"**.

3 Un AVL nunca se convertirá en una Linked List.

4

5 **AVL vs Red-Black Tree:**

6 - AVL es más balanceado (búsqueda más rápida).

7 - Red-Black es más rápido para insertar/eliminar (menos rotaciones).

8 - ``std::map`` en C++ suele usar Red-Black.

9

10 Implementación: Casi nunca te pedirán programar un AVL completo

11 en una entrevista de 45 min (es muy largo), pero DEBES saber explicar

12 las rotaciones y por qué son necesarias.

### Resumen rápido

- BST que se auto-balancea solo.
- Balance Factor: diferencia de altura entre hijos (máximo 1).
- Rotaciones: El truco místico para equilibrar el árbol.
- Garantiza  $O(\log n)$  para todas las operaciones.
- Más complejo de programar que un BST normal.
- Ideal para aplicaciones donde hay MUCHAS búsquedas y pocas inserciones.

# Capítulo 11

## 🌳 Recorridos de Árboles (Tree Traversals)

### 🤖 ¿Qué es?

Visitar todas las habitaciones de una casa de varios pisos . Hay dos grandes estilos:

- En profundidad (DFS): entras por una puerta y sigues bajando hasta el sótano antes de regresar. Según CUÁNDO anotes cada habitación (al entrar, en medio, o al salir) obtienes Preorder, Inorder o Postorder.
- Por niveles (BFS): recorres piso por piso: primero toda la planta baja, luego todo el primer piso, y así. Eso es Level-order.

El "cuándo apuntas el nodo" cambia por completo el orden del resultado.

### 💬 Dicho de otra forma

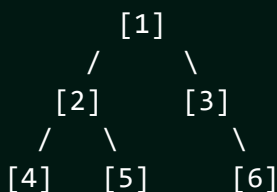
Recorrer un árbol es visitar cada nodo exactamente una vez. Los cuatro recorridos fundamentales:

- **Preorder** (Raíz → Izq → Der): útil para COPIAR/serializar un árbol.
- **Inorder** (Izq → Raíz → Der): en un BST devuelve los valores ORDENADOS.
- **Postorder** (Izq → Der → Raíz): útil para BORRAR el árbol o evaluar expresiones (procesa hijos antes que el padre).
- **Level-order** (por niveles): BFS con Queue; base de muchos problemas de árboles en entrevistas.

Los tres DFS salen naturales con recursión, pero conviene dominar la versión iterativa con Stack (evita stack overflow y la piden mucho).

### 🗺️ Diagrama

Árbol:



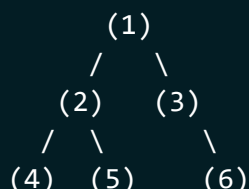
```

Preorder (R,I,D): 1 2 4 5 3 6
Inorder (I,R,D): 4 2 5 1 3 6
Postorder (I,D,R): 4 5 2 6 3 1
Level-order : 1 2 3 4 5 6 (por niveles, con Queue)

```

## ⚙️ ¿Cómo funciona?

CUÁNDO se "apunta" el nodo cambia todo (mismo árbol):



PREORDER (apunta al ENTRAR, antes de bajar):

visita 1 → baja izq → 2 → 4 → sube → 5 → sube → der → 3 → 6  
 1 [2 (4)(5)] [3 (6)] = 1 2 4 5 3 6

INORDER (apunta ENTRE izq y der):

4 (2) 5 ... 1 ... 3 6 = 4 2 5 1 3 6

POSTORDER (apunta al SALIR, después de los hijos):

(4)(5) 2 (6) 3 1 = 4 5 2 6 3 1

LEVEL-ORDER (Queue, nivel por nivel):

Queue: [1] → saca 1, mete 2,3 → [2,3] → saca 2, mete 4,5 ...  
 nivel0: 1   nivel1: 2 3   nivel2: 4 5 6   = 1 2 3 4 5 6

### 1 DFS RECURSIVO

- 2 - Preorder : procesa nodo, luego izq, luego der.
- 3 - Inorder : izq, procesa nodo, der.
- 4 - Postorder: izq, der, procesa nodo.

### 6 LEVEL-ORDER (BFS)

- 7 - Queue con la raíz. Saca un nodo, procésalo, encola sus hijos.
- 8 - Para separar por niveles: procesa exactamente **queue.size()** nodos por iteración del bucle externo.

### 11 PSEUDOCÓDIGO (level-order por niveles)

14 función levelOrder(root):

15   SI root == NULL: return

16   q = Queue con root

17   MIENTRAS q no vacia:

18     n = q.size()

// nodos de este nivel

19     REPETIR n veces:

```

20 nodo = q.dequeue()
21 procesar(nodo)
22 SI nodo.izq: q.enqueue(nodo.izq)
23 SI nodo.der: q.enqueue(nodo.der)
24 // aqui termina un nivel
25
26 CÓDIGO C++ (DFS recursivo)

27
28
29 struct Node { int val; Node *left, *right; };
30
31 void preorder(Node* n) { if(!n) return; cout<<n->val<<" ";
32 preorder(n->left); preorder(n->right); }
33 void inorder(Node* n) { if(!n) return; inorder(n->left);
34 cout<<n->val<<" "; inorder(n->right); }
35 void postorder(Node* n) { if(!n) return; postorder(n->left);
36 postorder(n->right); cout<<n->val<<" "; }
37
38 CÓDIGO C++ (inorder iterativo con Stack)

39
40
41 #include <stack>
42 #include <vector>
43 using namespace std;
44
45 vector<int> inorderIter(Node* root) {
46 vector<int> res; stack<Node*> st; Node* cur = root;
47 while (cur || !st.empty()) {
48 while (cur) { st.push(cur); cur = cur->left; } // baja a la izq
49 cur = st.top(); st.pop();
50 res.push_back(cur->val); // procesa
51 cur = cur->right; // ve a la der
52 }
53 return res;
54 }
55
56 CÓDIGO C++ (level-order con Queue)

57
58
59 #include <queue>
60 vector<vector<int>> levelOrder(Node* root) {
61 vector<vector<int>> res;
62 if (!root) return res;
63 queue<Node*> q; q.push(root);
64 while (!q.empty()) {
65 int n = q.size();
66 vector<int> nivel;
67 for (int i = 0; i < n; i++) {

```

```

68 Node* cur = q.front(); q.pop();
69 nivel.push_back(cur->val);
70 if (cur->left) q.push(cur->left);
71 if (cur->right) q.push(cur->right);
72 }
73 res.push_back(nivel);
74 }
75 return res;
76 }

```

77  
78 COMPLEJIDAD (Big-O)

| Recorrido | Tiempo | Espacio                 |
|-----------|--------|-------------------------|
| Los 4     | $O(n)$ | $O(h)$ DFS / $O(w)$ BFS |

85 \*  $n$  = nodos,  $h$  = altura,  $w$  = anchura máxima del árbol.

## En entrevistas

Inorder de un BST = ordenado: base para validar un BST o encontrar el k-ésimo menor. Si el inorder no sale creciente, NO es un BST válido.

Level-order es la puerta a decenas de problemas: "Right Side View", "Zigzag Traversal", "conectar nodos del mismo nivel", "profundidad mínima". Casi siempre es BFS por niveles con `queue.size()`.

Recursión vs iterativo: la recursiva es más limpia, pero con árboles muy profundos (miles de niveles) puede desbordar el stack. Ten la versión iterativa lista.

Reconstrucción: con Preorder + Inorder (o Postorder + Inorder) puedes reconstruir un árbol binario único. Pregunta frecuente de Google.

## Resumen rápido

- Preorder (R,I,D): copiar/serializar.
- Inorder (I,R,D): en BST da orden creciente.
- Postorder (I,D,R): borrar árbol, evaluar expresiones.
- Level-order (BFS con Queue): recorridos por niveles.
- Domina la versión iterativa con Stack (evita stack overflow).
- Todos  $O(n)$  tiempo; espacio  $O(h)$  en DFS,  $O(\text{ancho})$  en BFS.

# Capítulo 12

## Ancestro Común (LCA) y Serialización de Árboles

### ¿Qué es?

Dos primos buscando a su pariente compartido más cercano en un árbol genealógico. El Ancestro Común más Bajo (Lowest Common Ancestor) es la primera persona de la que ambos descienden: ni muy arriba (el bisabuelo de todos), ni imposible (uno mismo).

Y la Serialización es tomarle una "foto plana" al árbol para guardarlo en un archivo o mandarlo por la red, de forma que después puedas RECONSTRUIRLO idéntico. Como desarmar un mueble con instrucciones para volverlo a armar exactamente igual.

### Dicho de otra forma

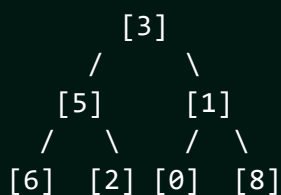
El **LCA** de dos nodos  $p$  y  $q$  es el nodo más profundo que tiene a ambos como descendientes (un nodo puede ser descendiente de sí mismo).

- En un **BST**: aprovecha el orden. Baja desde la raíz; si ambos valores son menores ve a la izquierda, si ambos son mayores ve a la derecha; en cuanto se "separan", ese nodo es el LCA.
- En un **árbol binario general**: DFS recursivo. Si encuentras  $p$  o  $q$  los devuelves; el primer nodo que recibe respuesta distinta de NULL por AMBOS lados es el LCA.

La **serialización** convierte el árbol en una cadena (típicamente un preorder con marcadores para los NULL) y la deserialización la reconstruye. Es la base de guardar árboles en disco o red.

### Diagrama

Árbol:



```
LCA(6, 2) = 5 (ambos cuelgan de 5)
LCA(6, 8) = 3 (se separan en la raíz)
LCA(5, 2) = 5 (un nodo es ancestro de sí mismo)
```

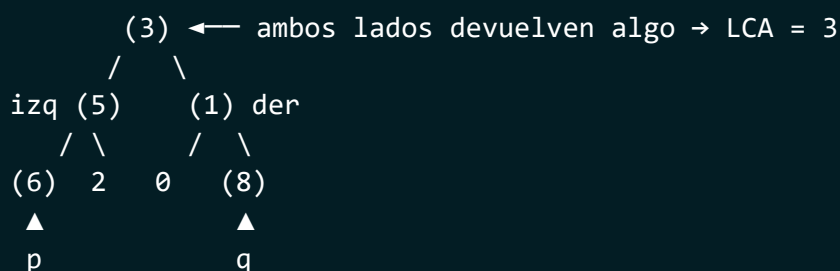
Serialización (preorder, # = NULL):

```
3,5,6,#,#,2,#,#,1,0,#,#,8,#,#
```

Deserializar lee en el mismo orden y reconstruye idéntico.

## ⚙️ ¿Cómo funciona?

LCA(6, 8) subiendo la señal desde las hojas



```
dfs(3): izq=dfs(5) der=dfs(1)
```

```
dfs(5): izq=dfs(6)=6 OK der=dfs(2)=null → devuelve 6
```

```
dfs(1): izq=dfs(0)=null der=dfs(8)=8 OK → devuelve 8
```

```
en 3: izq=6 (≠null) Y der=8 (≠null) → 3 es el LCA OK
```

SERIALIZAR (preorder, # = NULL) produce un stream lineal:

```
3 , 5 , 6 , # , # , 2 , # , # , 1 , 0 , # , # , 8 , # , #
└─raíz┘ └─subárbol izq┘ └─subárbol der┘
```

Deserializar consume el stream en el MISMO orden → árbol idéntico.

### 1 LCA EN ÁRBOL BINARIO GENERAL (DFS)

1. Caso base: si el nodo es **NULL** o es p o q, devuélvelo.
2. Busca en izquierda y en derecha.
3. Si ambos lados devuelven algo -> este nodo es el LCA.
4. Si solo un lado devuelve algo -> propaga ese resultado hacia arriba.

### 7 PSEUDOCÓDIGO (LCA general)

```
función lca(nodo, p, q):
```

```
 SI nodo == NULL O nodo == p O nodo == q: return nodo
```

```
 izq = lca(nodo.izq, p, q)
```

```
 der = lca(nodo.der, p, q)
```

```
 SI izq != NULL Y der != NULL: return nodo // p y q en lados
 distintos
```

```
 return (izq != NULL) ? izq : der
```

### 17 CÓDIGO C++ (LCA general y en BST)

---

```

18
19
20 struct Node { int val; Node *left, *right; };
21
22 // Árbol binario general -> O(n)
23 Node* lca(Node* root, Node* p, Node* q) {
24 if (!root || root == p || root == q) return root;
25 Node* l = lca(root->left, p, q);
26 Node* r = lca(root->right, p, q);
27 if (l && r) return root; // se separan aquí -> LCA
28 return l ? l : r;
29 }
30
31 // BST -> O(h), aprovecha el orden
32 Node* lcaBST(Node* root, int p, int q) {
33 while (root) {
34 if (p < root->val && q < root->val) root = root->left;
35 else if (p > root->val && q > root->val) root = root->right;
36 else return root; // se separan -> LCA
37 }
38 return nullptr;
39 }

```

CÓDIGO C++ (serializar / deserializar, preorder)

---

```

42
43
44 #include <string>
45 #include <sstream>
46 using namespace std;
47
48 void serializeH(Node* n, string& out) {
49 if (!n) { out += "#,"; return; }
50 out += to_string(n->val) + ",";
51 serializeH(n->left, out);
52 serializeH(n->right, out);
53 }
54
55 Node* deserializeH(istringstream& in) {
56 string tok; getline(in, tok, ',');
57 if (tok == "#") return nullptr;
58 Node* n = new Node{stoi(tok), nullptr, nullptr};
59 n->left = deserializeH(in);
60 n->right = deserializeH(in);
61 return n;
62 }

```

COMPLEJIDAD (Big-O)

---



|    |               |        |         |
|----|---------------|--------|---------|
| 66 |               |        |         |
| 67 | Operación     | Tiempo | Espacio |
| 68 |               |        |         |
| 69 | LCA (general) | $O(n)$ | $O(h)$  |
| 70 | LCA (BST)     | $O(h)$ | $O(1)$  |
| 71 | Serializar    | $O(n)$ | $O(n)$  |
| 72 | Deserializar  | $O(n)$ | $O(n)$  |

### En entrevistas

”Lowest Common Ancestor” (LeetCode 235 BST / 236 general): dos versiones. En BST usa el orden (más rápido,  $O(h)$ ); en árbol general usa el DFS de ”ambos lados devuelven algo”.

”Serialize and Deserialize Binary Tree” (LeetCode 297): clave usar un marcador para NULL (#) para que la reconstrucción sea única.

Sin marcador de NULL no se puede reconstruir: un preorder de solo valores es ambiguo. Los # son obligatorios.

Distancia entre dos nodos =  $\text{profundidad}(p) + \text{profundidad}(q) - 2 \cdot \text{profundidad}(\text{LCA})$ . El LCA es la pieza para medir distancias en árboles.

### Resumen rápido

- LCA: ancestro común más profundo de dos nodos.
- BST: usa el orden, baja hasta que p y q se separan;  $O(h)$ .
- General: DFS; el nodo donde ambos lados devuelven algo es el LCA.
- Serializar: preorder con # para los NULL;  $O(n)$ .
- Deserializar lee en el mismo orden y reconstruye idéntico.
- LCA es la base para medir distancias entre nodos.

# Capítulo 13

## Propiedades de Árboles (Altura, Diámetro, Balance)

### ¿Qué es?

Medir un árbol preguntando cosas desde ABAJO hacia arriba. Casi todo en un árbol se resuelve con un DFS que devuelve un dato de cada rama y lo combina en el padre:

- **Altura:** ¿qué tan lejos está la hoja más profunda?
- **Diámetro:** ¿cuál es el camino más largo entre dos hojas cualquiera?
- **¿Está balanceado?:** ¿ninguna rama es mucho más larga que su hermana?

La idea central es la **recursión post-orden**: primero resuelves los hijos, y con esos resultados decides el del padre.

### Dicho de otra forma

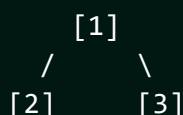
Un patrón resuelve la mayoría de preguntas sobre árboles: un DFS recursivo que retorna un valor por subárbol y lo combina en el nodo padre.

- **Altura/Profundidad:**  $1 + \max(h_{izq}, h_{der})$ , donde  $h$  es la altura de cada hijo.
- **Diámetro:** el camino más largo entre dos nodos. En cada nodo, `altura_izq + altura_der` es el diámetro que pasa por él; el resultado es el máximo sobre todos los nodos.
- **Balanceado:** en cada nodo, la diferencia de alturas de sus hijos es  $\leq 1$ . Se verifica en una sola pasada devolviendo la altura y un flag.
- **Path Sum:** existe una raíz-a-hoja que suma un objetivo, o la suma máxima de camino.

La clave es calcularlo en **una sola pasada**  $O(n)$ , no recalculando alturas dentro de otro recorrido (eso sería  $O(n^2)$ ).

### Diagrama

Árbol:



```

 / \
 [4] [5]

```

Altura = 3 (1 -> 2 -> 4)

Diámetro (camino más largo): 4 -> 2 -> 5      ó      4 -> 2 -> 1 -> 3

En el nodo 2: altura\_izq(1) + altura\_der(1) = 2 aristas

En el nodo 1: altura\_izq(2) + altura\_der(1) = 3 aristas <- máximo

Diámetro = 3 aristas.

¿Balanceado?  $|alt(2)=2 - alt(3)=1| = 1 \leq 1 \rightarrow$  Sí

## ⚙️ ¿Cómo funciona?

CÁLCULO BOTTOM-UP: cada nodo devuelve su altura (h) hacia arriba

```

 (1) h=3
 / \
 h=2 (2) (3) h=1
 / \
 h=1 (4) (5) h=1

```

Las hojas devuelven h=1. El padre = 1 + max(hijos).

altura(4)=1   altura(5)=1    $\rightarrow$  altura(2)=1+max(1,1)=2

altura(3)=1                                       $\rightarrow$  altura(1)=1+max(2,1)=3

DIÁMETRO (se actualiza en cada nodo con hIzq + hDer, en aristas):

en nodo 2: hIzq(1) + hDer(1) = 2 aristas      (camino 4-2-5)

en nodo 1: hIzq(2) + hDer(1) = 3 aristas      (camino 4-2-1-3) <- MÁXIMO

Diámetro = 3

BALANCE ( $|hIzq - hDer| \leq 1$  en todos):

nodo 1:  $|2 - 1| = 1$  OK      nodo 2:  $|1 - 1| = 0$  OK       $\rightarrow$  balanceado

1 PATRÓN GENERAL (DFS post-orden que retorna un valor)

2 - Resuelve izquierda y derecha primero.

3 - Combina sus resultados para el nodo actual.

4 - Actualiza una respuesta global si hace falta (ej. diámetro).

5  
6 PSEUDOCÓDIGO (diámetro)

7  
8  
9 diametro = 0

10 función altura(nodo):

11    SI nodo == NULL: return 0

12    izq = altura(nodo.izq)

13    der = altura(nodo.der)

14    diametro = max(diametro, izq + der)    // camino que pasa por nodo

```

15 return 1 + max(izq, der)
16
17 // llamar altura(root); la respuesta queda en 'diametro'
18
19 CÓDIGO C++
20
21
22 struct Node { int val; Node *left, *right; };
23
24 // Altura -> O(n)
25 int height(Node* n) {
26 if (!n) return 0;
27 return 1 + max(height(n->left), height(n->right));
28 }
29
30 // Diámetro en una sola pasada -> O(n)
31 int diameter = 0;
32 int dfsDiam(Node* n) {
33 if (!n) return 0;
34 int l = dfsDiam(n->left), r = dfsDiam(n->right);
35 diameter = max(diameter, l + r); // aristas que cruzan por n
36 return 1 + max(l, r);
37 }
38
39 // ¿Balanceado? -1 = no balanceado; si no, retorna la altura -> O(n)
40 int checkBalance(Node* n) {
41 if (!n) return 0;
42 int l = checkBalance(n->left); if (l == -1) return -1;
43 int r = checkBalance(n->right); if (r == -1) return -1;
44 if (abs(l - r) > 1) return -1; // desbalanceado
45 return 1 + max(l, r);
46 }
47
48 // Path Sum raíz-a-hoja que suma 'target'
49 bool hasPathSum(Node* n, int target) {
50 if (!n) return false;
51 if (!n->left && !n->right) return target == n->val; // hoja
52 return hasPathSum(n->left, target - n->val) ||
53 hasPathSum(n->right, target - n->val);
54 }
55
56 COMPLEJIDAD (Big-O)

```

| Propiedad  | Tiempo | Espacio |
|------------|--------|---------|
| Altura     | $O(n)$ | $O(h)$  |
| Diámetro   | $O(n)$ | $O(h)$  |
| Balanceado | $O(n)$ | $O(h)$  |
| Path Sum   | $O(n)$ | $O(h)$  |

65  
66

\* `h` = altura (pila de recursión). Una sola pasada, no  $O(n^2)$ .

### En entrevistas

"Maximum Depth" (104), "Diameter of Binary Tree" (543), "Balanced Binary Tree" (110), "Path Sum" (112): todos son el MISMO patrón DFS post-orden que retorna un dato por subárbol.

Truco del "flag  $-1$ ": para "¿está balanceado?", en vez de calcular altura y balance por separado ( $O(n^2)$ ), retorna la altura o  $-1$  si ya detectaste desbalance. Una sola pasada  $O(n)$ .

Diámetro se mide en ARISTAS, no en nodos, en la mayoría de enunciados (LeetCode 543). Lee bien: a veces el camino no pasa por la raíz.

"Binary Tree Maximum Path Sum" (124): variante dura. En cada nodo, la ganancia que subes es `val + max(0, izq, der)`, pero la respuesta global considera `val + izq + der`.

### Resumen rápido

- Patrón maestro: DFS post-orden que retorna un dato por subárbol.
- Altura =  $1 + \max(\text{alturas de hijos})$ .
- Diámetro = max sobre nodos de (`altura izq + altura der`); en aristas.
- Balanceado: diferencia de alturas  $\leq 1$ ; usa flag  $-1$  en una pasada.
- Path Sum: resta el valor del nodo al bajar; verifica en las hojas.
- Todo  $O(n)$ ; no recalculas alturas (evita  $O(n^2)$ ).

# Capítulo 14



## Heaps (Montículos)

### 🤖 ¿Qué es?

Un torneo de fútbol donde el mejor equipo SIEMPRE llega a la final (está en la cima).

Si el mejor equipo pierde o se retira (lo sacas), el siguiente mejor sube automáticamente a ocupar su lugar. No sabes exactamente quién es el tercer o cuarto mejor sin hacer más comparaciones, pero SIEMPRE tienes al #1 disponible de inmediato.

Es un árbol binario especial donde la raíz siempre tiene el valor máximo (Max-Heap) o mínimo (Min-Heap) de toda la estructura.

### 💬 Dicho de otra forma

Un Heap es un árbol binario completo (lleno de arriba a abajo, de izquierda a derecha) que cumple la "Propiedad de Heap": - Max-Heap: El padre siempre es mayor o igual que sus hijos. - Min-Heap: El padre siempre es menor o igual que sus hijos.

A diferencia del BST, no hay orden entre hermanos. Lo único que importa es la relación Padre-Hijo. Por su estructura compacta, se suele implementar eficientemente usando un Array.

### 🤖 Diagrama

Max-Heap Visual:

```
 [90]
 / \
 [80] [75]
 / \ / \
 [60][70][50][55]
```

Implementado como Array (¡Truco de índices!):

Idx: [0] [1] [2] [3] [4] [5] [6]

Val: [90][80][75][60][70][50][55]

Fórmulas mágicas para el índice `i`:

- Parent:  $(i - 1) / 2$
- Left Child:  $2 * i + 1$

- Right Child:  $2 * i + 2$

1. Insert: Agregas el nuevo valor al final del **array** (la hoja más a la izquierda disponible). Luego lo comparas con su padre y lo subes si es necesario (Bubble Up / Heapify Up).  $O(\log n)$ .
2. Extract Max/Min: Sacas la raíz ( $arr[0]$ ). Mueves el último elemento a la raíz y lo vas bajando comparándolo con sus hijos hasta que esté en su lugar (Bubble Down / Heapify Down).  $O(\log n)$ .
3. Peek: Miras  $arr[0]$ .  $O(1)$ .
4. Heapify (Construir Heap): Convertir un **array** desordenado en un Heap.  $O(n)$ .

#### PSEUDOCÓDIGO

---

```
// Heapify Up (al insertar)
función subir(i):
 MIENTRAS i > 0 Y arr[padre(i)] < arr[i]:
 intercambiar(arr[padre(i)], arr[i])
 i = padre(i)

// Heapify Down (al extraer)
función bajar(i):
 mayor = i
 SI hijoIzquierdo(i) existe Y arr[hijoIzquierdo(i)] > arr[mayor]:
 mayor = hijoIzquierdo(i)
 SI hijoDerecho(i) existe Y arr[hijoDerecho(i)] > arr[mayor]:
 mayor = hijoDerecho(i)
 SI mayor != i:
 intercambiar(arr[i], arr[mayor])
 bajar(mayor)
```

#### CÓDIGO C++

---

```
#include <iostream>
#include <vector>
#include <queue> // STL priority_queue es un Heap
using namespace std;

int main() {
 // 1. Max-Heap (por defecto)
 priority_queue<int> maxHeap;

 maxHeap.push(10);
 maxHeap.push(30);
 maxHeap.push(20);

 cout << "Máximo en la cima: " << maxHeap.top() << endl; // 30
```

```

49 maxHeap.pop(); // Quita el 30
50 cout << "Nuevo máximo: " << maxHeap.top() << endl; // 20
51
52 // 2. Min-Heap
53 priority_queue<int, vector<int>, greater<int>> minHeap;
54 minHeap.push(10);
55 minHeap.push(30);
56 minHeap.push(20);
57
58 cout << "Mínimo en la cima: " << minHeap.top() << endl; // 10
59
60 return 0;
61 }

```

### COMPLEJIDAD (Big-O)

| Operación       | Tiempo      | Razón                              |
|-----------------|-------------|------------------------------------|
| Insert          | $O(\log n)$ | Altura del árbol                   |
| Extract Max/Min | $O(\log n)$ | Altura del árbol                   |
| Peek Max/Min    | $O(1)$      | Es la raíz ( <code>arr[0]</code> ) |
| Heapify (Build) | $O(n)$      | Análisis matemático avanzado       |
| Search          | $O(n)$      | No es un BST, hay que buscar todo  |

1 

2 **"Top K Elements":** Si te piden los K elementos más grandes, usa un

3 Min-Heap de tamaño K. Si te piden los más chicos, usa un Max-Heap. 

4

5 Priority Queue = Heap: En el 99% de los casos, cuando necesites un

6 Heap en una entrevista, usa `std::priority_queue`. No lo programes

7 desde cero a menos que te lo pidan explícitamente. 

8

9 No es un BST: Recuerda que un Heap NO está ordenado de izquierda a

10 derecha. Solo garantiza la relación Padre > Hijo. 

11

12 Heapsort: Puedes ordenar un **array** metiendo todo a un Heap y

13 sacándolo uno por uno.  $O(n \log n)$ .

### Resumen rápido

- Árbol binario completo donde el jefe (raíz) es el mayor o menor.
- Implementado como Array para máxima velocidad y ahorro de memoria.
- Insertar y sacar el máximo/mínimo es  $O(\log n)$ .
- Ver el máximo/mínimo es instantáneo  $O(1)$ .
- Usos: Algoritmo de Dijkstra, colas de prioridad, problemas de "Top K".
- Heapify (construir desde array) es sorprendentemente  $O(n)$ .



# Capítulo 15

## Segment Tree (Árbol de Segmentos)

### ¿Qué es?

Un array donde te preguntan miles de veces "¿cuál es la suma (o el mínimo) entre la posición 3 y la 8?" y ADEMÁS cambian valores entre pregunta y pregunta. Recalculando sumando a mano cada vez cuesta  $O(n)$  por pregunta: lento. El Segment Tree precalcula las respuestas de bloques y las organiza en un árbol. Así responde cualquier rango en  $O(\log n)$  y también actualiza en  $O(\log n)$ . Cada nodo del árbol "resume" un pedazo del array; la raíz resume todo, y las hojas son los elementos individuales.

### Dicho de otra forma

Un Segment Tree es un árbol binario donde cada nodo guarda información agregada (suma, mínimo, máximo, gcd...) de un segmento contiguo del array. La raíz cubre  $[0, n - 1]$ ; cada nodo se divide en dos mitades hasta llegar a hojas de un solo elemento.

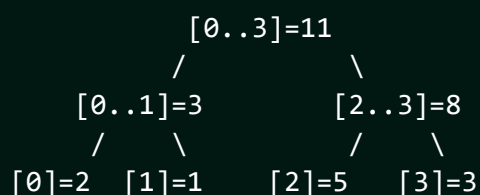
Permite dos operaciones en  $O(\log n)$ :

- **Query de rango:** combinar los nodos que cubren  $[l, r]$ .
- **Update de punto:** cambiar un elemento y propagar hacia la raíz.

Con **lazy propagation** soporta también updates de RANGO (sumar a todo un intervalo) en  $O(\log n)$ . Se almacena en un array de tamaño  $\sim 4n$ .

### Diagrama

Array: [2, 1, 5, 3] (Segment Tree de SUMA)



Query suma(1..2) = combina [1]=1 y [2]=5 = 6

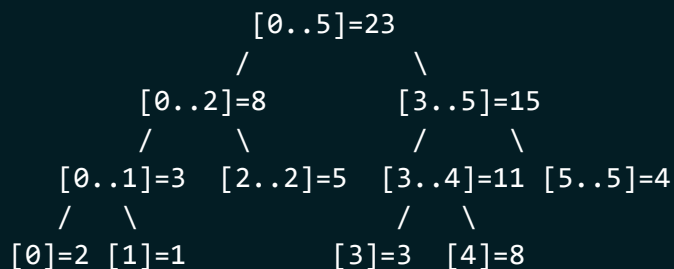
Update pos 3 a 10: actualiza hoja [3] y sube: [2..3]=15, raíz=18

Cada consulta/actualización toca  $O(\log n)$  nodos.

## ⚙️ ¿Cómo funciona?

QUERY de rango descompuesta en  $O(\log n)$  nodos

Array: [2, 1, 5, 3, 8, 4] ← pedimos suma(1..4) = 1+5+3+8 = 17



query(1..4) recorre y se queda con nodos TOTALMENTE dentro (◀):

[1..1]=1 ◀ [2..2]=5 ◀ [3..4]=11 ◀  
 suma = 1 + 5 + 11 = 17 (solo 3 nodos, no 4 hojas)

UPDATE punto (pos 4 = 8 → 10): sube recombinando la rama

[4]:8→10 → [3..4]:11→13 → [3..5]:15→17 → raíz:23→25  
 solo se tocan los nodos del camino a la raíz =  $O(\log n)$

CÓMO FUNCIONA (suma, **array** de tamaño  $4n$ )

- build(node, l, r): si  $l=r$ , hoja = arr[l]; si no, construye hijos y tree[node] = tree[izq] + tree[der].
- query(node, l, r, ql, qr): si el rango no toca → 0; si está contenido → tree[node]; si no, combina las dos mitades.
- update(node, l, r, pos, val): baja a la hoja, cámbiala y recombina.

PSEUDOCÓDIGO (query de rango)

función query(nodo, l, r, ql, qr):

```

 SI qr < l O r < ql: return 0 // fuera de rango
 SI ql <= l Y r <= qr: return tree[nodo] // totalmente dentro
 mid = (l + r) / 2
 return query(2*nodo, l, mid, ql, qr)
 + query(2*nodo+1, mid+1, r, ql, qr)

```

CÓDIGO C++ (Segment Tree de suma)

```

#include <vector>
using namespace std;

```

```

24 struct SegTree {
25 int n; vector<long long> tree;
26 SegTree(vector<int>& a) { n = a.size(); tree.assign(4*n, 0); build(a
 ,1,0,n-1); }
27
28 void build(vector<int>& a, int node, int l, int r) {
29 if (l == r) { tree[node] = a[l]; return; }
30 int mid = (l + r) / 2;
31 build(a, 2*node, l, mid);
32 build(a, 2*node+1, mid+1, r);
33 tree[node] = tree[2*node] + tree[2*node+1];
34 }
35 // suma en [ql, qr]
36 long long query(int node, int l, int r, int ql, int qr) {
37 if (qr < l || r < ql) return 0; // sin solape
38 if (ql <= l && r <= qr) return tree[node]; // contenido
39 int mid = (l + r) / 2;
40 return query(2*node, l, mid, ql, qr)
41 + query(2*node+1, mid+1, r, ql, qr);
42 }
43 // arr[pos] = val
44 void update(int node, int l, int r, int pos, int val) {
45 if (l == r) { tree[node] = val; return; }
46 int mid = (l + r) / 2;
47 if (pos <= mid) update(2*node, l, mid, pos, val);
48 else update(2*node+1, mid+1, r, pos, val);
49 tree[node] = tree[2*node] + tree[2*node+1];
50 }
51 };

```

COMPLEJIDAD (Big-O)

| Operación       | Tiempo      | Espacio          |
|-----------------|-------------|------------------|
| Build           | $O(n)$      | $O(4n)$          |
| Query de rango  | $O(\log n)$ | $O(\log n)$ pila |
| Update de punto | $O(\log n)$ | $O(\log n)$ pila |

\* Con lazy propagation, el update de RANGO también es  $O(\log n)$ .

## En entrevistas

¿Cuándo usarlo? Muchas consultas de rango CON actualizaciones intercaladas. Si el array NUNCA cambia, un prefix-sum simple ( $O(1)$  por query) es mejor y más fácil.

”Range Sum Query - Mutable” (LeetCode 307): el caso canónico. Actualizacio-

nes + sumas de rango = Segment Tree o Fenwick.

Tamaño  $4n$ : reserva  $4n$  para el árbol, no  $2n$ ; con  $n$  que no es potencia de 2 puede desbordar si escatimas memoria.

Segment Tree vs Fenwick: Fenwick (BIT) es más corto y rápido para sumas de prefijo, pero el Segment Tree es más general (mínimo, máximo, gcd, y updates de rango con lazy).

### Resumen rápido

- Árbol binario; cada nodo resume un segmento del array.
- Query de rango y update de punto en  $O(\log n)$ .
- Raíz cubre todo; hojas son elementos individuales.
- Lazy propagation habilita updates de RANGO en  $O(\log n)$ .
- Se guarda en un array de tamaño  $4n$ .
- Si el array no cambia, usa prefix-sum (más simple).

# Capítulo 16

## Fenwick Tree / BIT (Binary Indexed Tree)

### ¿Qué es?

Una versión súper compacta del Segment Tree para sumas de prefijo. Imagina que quieres el saldo acumulado hasta el día 7, pero los montos cambian a diario. El Fenwick Tree guarda sumas parciales inteligentes: cada posición del array "cubre" un bloque de tamaño potencia de 2, definido por su bit menos significativo. Para sumar hasta  $i$ , saltas hacia atrás quitando ese bit; para actualizar, saltas hacia adelante sumándolo. Ambos recorridos son  $O(\log n)$  y el código cabe en cuatro líneas.

### Dicho de otra forma

Un Fenwick Tree (o Binary Indexed Tree, BIT) es una estructura que calcula sumas de prefijo y actualizaciones de punto en  $O(\log n)$ , usando solo un array de tamaño  $n + 1$  (mucho menos que el  $4n$  del Segment Tree).

Su magia es el operador  $i \& (-i)$ , que aísla el bit menos significativo de  $i$  y define el tamaño del bloque que cada índice resume:

- **Query prefijo( $i$ ):** parte de  $i$  y resta el bit menos significativo hasta llegar a 0, acumulando.
- **Update( $i$ , delta):** parte de  $i$  y SUMA el bit menos significativo hasta pasar  $n$ , propagando el cambio.

Una suma de rango  $[l, r]$  se obtiene como  $\text{prefijo}(r) - \text{prefijo}(l - 1)$ .

### Diagrama

Índices (1-based) y qué bloque cubre cada uno (por lowbit):

|        |     |       |     |       |     |       |     |       |
|--------|-----|-------|-----|-------|-----|-------|-----|-------|
| i:     | 1   | 2     | 3   | 4     | 5   | 6     | 7   | 8     |
| cubre: | [1] | [1-2] | [3] | [1-4] | [5] | [5-6] | [7] | [1-8] |

```
lowbit(i) = i & (-i) (bit menos significativo)
lowbit(6)=2 -> el 6 cubre 2 elementos: [5,6]
lowbit(8)=8 -> el 8 cubre 8 elementos: [1..8]
```

```

query prefijo(7): 7 -> 6 -> 4 -> 0 (resta lowbit)
 suma tree[7]+tree[6]+tree[4]
update(5,+3): 5 -> 6 -> 8 -> fin (suma lowbit)

```

## ⚙️ ¿Cómo funciona?

QUÉ RANGO cubre cada índice (largo = lowbit) y los SALTOS

```

i: 1 2 3 4 5 6 7 8
 | | | | | | |
 | | | | | | |
cubre: [1] [1-2] [3] [1-4] [5] [5-6] [7] [1-8]

```

QUERY prefijo(7) = suma de 1..7 (resta lowbit: baja)  
 7 —lowbit1→ 6 —lowbit2→ 4 —lowbit4→ 0 (fin)  
 suma = tree[7] + tree[6] + tree[4]  
 = [7] + [5-6] + [1-4] = cubre 1..7 OK

UPDATE(3, +5) (suma lowbit: sube)  
 3 —lowbit1→ 4 —lowbit4→ 8 → fin (>n)  
 tree[3] += 5 ; tree[4] += 5 ; tree[8] += 5  
 (todos los bloques que "contienen" al índice 3 se enteran)

Cada salto quita/suma un bit → a lo más  $\log(n)$  pasos.

### CÓMO FUNCIONA (1-indexado)

- lowbit(i) =  $i \& (-i)$  aísla el bit menos significativo.
- update(i, d): mientras  $i \leq n$ : tree[i] += d;  $i += \text{lowbit}(i)$ .
- query(i): suma = 0; mientras  $i > 0$ : suma += tree[i];  $i -= \text{lowbit}(i)$ .
- rango(l, r) = query(r) - query(l-1).

### PSEUDOCÓDIGO

función update(i, delta):

```

 MIENTRAS i <= n:
 tree[i] += delta
 i += i & (-i) // avanza al siguiente bloque

```

función query(i): // suma de 1..i

```

 suma = 0
 MIENTRAS i > 0:
 suma += tree[i]
 i -= i & (-i) // retrocede quitando el bit
 return suma

```

### CÓDIGO C++

```

25 #include <vector>
26 using namespace std;
27
28 struct Fenwick {
29 int n; vector<long long> tree;
30 Fenwick(int n) : n(n), tree(n + 1, 0) {} // 1-indexado
31
32 void update(int i, long long delta) {
33 for (; i <= n; i += i & (-i)) tree[i] += delta;
34 }
35 long long query(int i) { // suma de 1..i
36 long long s = 0;
37 for (; i > 0; i -= i & (-i)) s += tree[i];
38 return s;
39 }
40 long long rango(int l, int r) { // suma de l..r
41 return query(r) - query(l - 1);
42 }
43 };

```

44 COMPLEJIDAD (Big-O)

| Operación       | Tiempo        | Espacio |
|-----------------|---------------|---------|
| Update de punto | $O(\log n)$   | $O(n)$  |
| Query prefijo   | $O(\log n)$   | $O(n)$  |
| Construir       | $O(n \log n)$ | $O(n)$  |

54 \* Solo **array** de tamaño  $n+1$ : mucho más ligero que el Segment Tree.

## En entrevistas

Fenwick vs Segment Tree: si solo necesitas sumas de prefijo con updates de punto, Fenwick gana (código corto, menos memoria, más rápido en la práctica). Para mínimo/máximo o updates de rango, ve al Segment Tree.

”Count of Smaller Numbers After Self” (LeetCode 315): se resuelve con un BIT sobre valores comprimidos. Clásico truco de conteo de inversiones.

Es 1-indexado: el índice 0 no se usa. Si tu array es 0-based, suma 1 al indexar o te quedas en un bucle infinito con  $i \& (-i)$ .

Conteo de inversiones: recorre de derecha a izquierda y consulta cuántos elementos menores ya insertaste. Es EL uso estrella del BIT en entrevistas.



## Resumen rápido

- BIT: sumas de prefijo + update de punto en  $O(\log n)$ .
- Usa  $i \& (-i)$  (bit menos significativo) para saltar bloques.
- query: resta el bit; update: suma el bit.
- Rango  $[l, r] = \text{prefijo}(r) - \text{prefijo}(l-1)$ .
- Es 1-indexado; array de tamaño  $n+1$  (más ligero que Segment Tree).
- Estrella para contar inversiones y "smaller after self".



# Capítulo 17

## Hash Tables (Tablas Hash / Mapas)

### ¿Qué es?

Un casillero de escuela . Cada estudiante tiene un número de casillero asignado por una fórmula mágica (la Hash Function).

Para guardar la mochila de "Carlos", aplicas la fórmula:  $\text{Hash}(\text{"Carlos"}) = 42$ . Vas directo al casillero 42 y la guardas. Para buscarla, vuelves a aplicar la fórmula:  $\text{Hash}(\text{"Carlos"}) = 42$ , y vas DIRECTO al casillero 42.

¡No tienes que revisar todos los casilleros uno por uno! Es como tener un acceso instantáneo a cualquier dato si conoces su "llave".

### Dicho de otra forma

Una Hash Table es una estructura que asocia Llaves (Keys) con Valores (Values). Usa una función matemática para convertir la Key en un índice de un array.

El objetivo principal es lograr operaciones de búsqueda, inserción y eliminación en tiempo constante promedio  $O(1)$ . Es la estructura más eficiente para búsquedas rápidas en la industria.

### Diagrama

```
Key: "Carlos" --> [Hash Function] --> Index: 2
Key: "Maria" --> [Hash Function] --> Index: 5
```

Array Interno (Buckets):

```
[0] : NULL
[1] : NULL
[2] : {Key: "Carlos", Value: "Mochila Azul"}
[3] : NULL
[4] : NULL
[5] : {Key: "Maria", Value: "Mochila Roja"}
```

Colisión (Collision):

Si  $\text{Hash}(\text{"Juan"})$  también da 2, ocurre una colisión.

Solución común: Chaining (una Linked List en el índice 2).

```
[2] : ["Carlos"] -> ["Juan"] -> NULL
```

1. Hash Function: Convierte cualquier Key (**string**, objeto, etc.) en un número entero dentro del rango del **array**.
2. Insert (**set**): Calculas el hash, vas al índice y guardas el par {K,V}.
3. Search (get): Calculas el hash, vas al índice y devuelves el valor.
4. Delete: Calculas el hash y eliminas la entrada en ese índice.
5. Rehashing: Si la tabla se llena mucho (Load Factor > 0.75), se crea un **array** más grande y se vuelven a calcular todos los hashes.

#### PSEUDOCÓDIGO

```
// Insertar en un Hash Map (con Chaining)
función insertar(llave, valor):
 índice = hash(llave) % tamaño_tabla
 lista = buckets[índice]
 SI llave ya existe en lista:
 actualizar valor
 SINO:
 agregar {llave, valor} al final de la lista
```

#### CÓDIGO C++

```
#include <iostream>
#include <unordered_map> // Hash Map estándar en C++
#include <unordered_set> // Hash Set (solo llaves únicas)
#include <string>
using namespace std;

int main() {
 // 1. Declaración
 unordered_map<string, int> edades;

 // 2. Insertar -> O(1) promedio
 edades["Carlos"] = 25;
 edades["Maria"] = 30;
 edades.insert({"Juan", 22});

 // 3. Acceder -> O(1) promedio
 cout << "Edad de Maria: " << edades["Maria"] << endl;

 // 4. Verificar existencia
 if (edades.find("Pedro") == edades.end()) {
 cout << "Pedro no está en la tabla." << endl;
 }

 // 5. Eliminar -> O(1)
 edades.erase("Carlos");
```

```

49 // 6. Recorrer
50 for (auto const& [nombre, edad] : edades) {
51 cout << nombre << " tiene " << edad << " años." << endl;
52 }
53
54
55 return 0;
56 }

```

57 COMPLEJIDAD (Big-O)

| Operación | Promedio | Peor caso |
|-----------|----------|-----------|
| Insert    | $O(1)$   | $O(n)$    |
| Search    | $O(1)$   | $O(n)$    |
| Delete    | $O(1)$   | $O(n)$    |
| Espacio   | $O(n)$   | $O(n)$    |

68 \* El peor caso  $O(n)$  ocurre si la función hash es muy mala y todos los  
69 elementos caen en el mismo casillero (colisión masiva).

## En entrevistas

Regla de Oro: Si necesitas buscar algo en  $O(1)$ , LA respuesta es un Hash Map o Hash Set.

"Two Sum": El problema más famoso que se resuelve con Hash Map. Guardas el complemento de lo que buscas para encontrarlo en  $O(1)$ .

map vs unordered\_map (C++): - 'unordered\_map': Usa Hash Table,  $O(1)$  promedio, no tiene orden. - 'map': Usa Red-Black Tree,  $O(\log n)$ , mantiene las llaves ordenadas.

Collision Handling: Debes saber explicar Chaining (listas) y Open Addressing (buscar el siguiente hueco libre).

## Resumen rápido

- Key -> Hash Function -> Index.
- Acceso, búsqueda e inserción instantáneos  $O(1)$ .
- La función hash debe ser rápida y distribuir bien los datos.
- Colisiones: Cuando dos llaves quieren el mismo lugar.
- Load Factor: Qué tan llena está la tabla.
- Estructura más usada en la vida real por su velocidad.

# Capítulo 18

## ✧ Hashing a Fondo: Colisiones e Implementación

### 🤖 ¿Qué es?

Un estacionamiento donde una fórmula te asigna el cajón según tu placa. El problema: a veces dos coches distintos reciben el MISMO cajón. Eso es una colisión, y toda tabla hash necesita un plan para resolverla:

- Chaining: en cada cajón cabe una fila de coches (una lista). Si chocan, se forman detrás.
- Open Addressing: si tu cajón está ocupado, buscas el siguiente libre siguiendo una regla.

Y si el estacionamiento se llena demasiado (Load Factor alto), se construye uno más grande y se reacomodan todos los coches (rehashing).

### 💬 Dicho de otra forma

Una hash table promedia  $O(1)$  SOLO si las colisiones se manejan bien. Dos grandes estrategias:

**Chaining (encadenamiento):** cada bucket guarda una lista enlazada de pares que colisionaron. Simple y tolerante a Load Factors altos. Es lo que usa `std::unordered_map`.

**Open Addressing (direccionamiento abierto):** todo vive en el array; ante colisión se busca otro hueco mediante probing:

- Linear probing: prueba  $i + 1, i + 2, \dots$  (sufre "clustering").
- Quadratic probing: prueba  $i + 1, i + 4, i + 9, \dots$
- Double hashing: el salto lo decide una segunda función hash.

El **Load Factor**  $\alpha = n/m$  (elementos / buckets) mide qué tan llena está la tabla. Cuando supera un umbral ( 0.75), se hace **rehashing**: se duplica el array y se reinsertan todas las claves.

### 🧠 Diagrama

CHAINING (listas por bucket)

[0] -> NULL

OPEN ADDRESSING (linear probing)

hash("A")=2, hash("B")=2, hash("C")=3

```

[1] -> NULL
[2] -> (A) -> (B) -> (C) [0] -
[3] -> NULL [1] -
[4] -> (D) [2] A <- va a su hueco
 [3] B <- ocupado el 2, prueba 3? no,
 C <- desplazados en cadena
Colisiones -> se apilan en la
lista del bucket. [4] -

Load Factor $\alpha = n/m$; si $\alpha > 0.75$ -> rehash (duplicar m y reinsertar).

```

## ⚙️ ¿Cómo funciona?

CHAINING vs OPEN ADDRESSING (insertar A,B,C con hash 2,2,3)

CHAINING (lista por bucket):

```

[0] → NULL
[1] → NULL
[2] → (A) → (B) → NULL
[3] → (C) → NULL
[4] → NULL

```

OPEN ADDR. linear probing:

```

[0] |
[1] |
[2] | A hash 2 libre
[3] | B C quería 3 → ocupado por B?
[4] | C no: B fue a 3 (2 lleno),
 C a 4 (3 lleno) → clustering

```

BORRAR en open addressing NO deja hueco vacío (rompería el probing):

```

[2] A [3] B [4] C
borrar B → [3] pasa a "tombstone" (marca de borrado, no NULL)
 las búsquedas siguen saltando el tombstone

```

REHASH (load factor  $\alpha = n/m > 0.75$ ): m: 8 → 16, reinsertar todo  
cada clave se recoloca con  $\text{hash \% nueva\_capacidad}$

### IMPLEMENTACIÓN PROPIA (chaining)

1. Array de buckets; **cada** bucket es una lista de pares {clave, valor}.
2.  $\text{índice} = \text{hash}(\text{clave}) \% \text{capacidad}$ .
3. Insert: si la clave existe en la lista, actualiza; si no, agrégala.
4. Get: recorre la lista del bucket buscando la clave.
5. Si load factor > umbral -> rehash (nueva capacidad, reinsertar todo).

### PSEUDOCÓDIGO (get con chaining)

función get(clave):

```

 idx = hash(clave) % capacidad
 PARA cada par en buckets[idx]:
 SI par.clave == clave: return par.valor
 return NO_ENCONTRADO

```

### CÓDIGO C++ (hash map propio con chaining)

```

18
19
20 #include <vector>
21 #include <list>
22 #include <utility>
23 using namespace std;
24
25 class MyHashMap {
26 vector<list<pair<int,int>>> buckets;
27 int capacity, size_;
28 int idx(int key) { return (key % capacity + capacity) % capacity; }
29
30 void rehash() {
31 auto old = buckets;
32 capacity *= 2;
33 buckets.assign(capacity, {});
34 size_ = 0;
35 for (auto& b : old) for (auto& kv : b) put(kv.first, kv.second);
36 }
37 public:
38 MyHashMap(int cap = 8) : buckets(cap), capacity(cap), size_(0) {}
39
40 void put(int key, int val) {
41 auto& b = buckets[idx(key)];
42 for (auto& kv : b) if (kv.first == key) { kv.second = val;
43 return; }
44 b.push_back({key, val});
45 if (++size_ > capacity * 0.75) rehash(); // load factor
46 }
47 int get(int key) {
48 for (auto& kv : buckets[idx(key)])
49 if (kv.first == key) return kv.second;
50 return -1; // no encontrado
51 }
52 void remove(int key) {
53 auto& b = buckets[idx(key)];
54 for (auto it = b.begin(); it != b.end(); ++it)
55 if (it->first == key) { b.erase(it); size_--; return; }
56 }
57 };

```

COMPLEJIDAD (Big-O)

| Método            | Promedio | Peor caso                         |
|-------------------|----------|-----------------------------------|
| Insert/Get/Delete | $O(1)$   | $O(n)$                            |
| Rehash            | $O(n)$   | $O(n)$ (amortizado $O(1)$ por op) |

\* Peor caso  $O(n)$ : función hash mala -> todo cae en un bucket.

### En entrevistas

Chaining vs Open Addressing: chaining tolera load factors altos y borra fácil; open addressing usa mejor la cache y menos memoria por puntero, pero se degrada al llenarse. Sabe explicar ambos.

Una buena hash function distribuye uniforme y es rápida. Para strings, el hash polinómico ( $h = h*31 + c$ ) es el clásico.

Borrar en open addressing es delicado: no puedes dejar el hueco vacío (rompería las búsquedas por probing); se marca como "tombstone" (borrado lógico).

`unordered_map` se degrada a  $O(n)$ : con claves maliciosas que colisionan a propósito (hash flooding). Por eso algunos lenguajes aleatorizan la semilla del hash.

### Resumen rápido

- Colisión: dos claves caen en el mismo bucket. Toda tabla necesita plan.
- Chaining: lista por bucket (lo usa `unordered_map`).
- Open Addressing: buscar otro hueco (linear/quadratic/double hashing).
- Load Factor  $\alpha = n/m$ ; si pasa 0.75 -> rehash (duplicar y reinsertar).
- Promedio  $O(1)$ ; peor caso  $O(n)$  con hash mala.
- Borrar en open addressing usa tombstones.

# Capítulo 19

## 🔑 Patrones con Hash Map

### 🤖 ¿Qué es?

El Hash Map es la navaja suiza de las entrevistas: cuando algo tarda  $O(n^2)$  "buscando de nuevo", casi siempre un Hash Map lo baja a  $O(n)$  recordando lo que ya viste.

Tres patrones que aparecen una y otra vez:

- Contar frecuencias: cuántas veces aparece cada cosa.
- Complemento (Two Sum): guardar lo que YA viste para encontrar su pareja al instante.
- Prefix-Sum + Hash: contar sub-arreglos cuya suma es un objetivo, recordando las sumas acumuladas.

### 💬 Dicho de otra forma

El Hash Map convierte "¿ya vi esto antes?" en una consulta  $O(1)$ . Los patrones más rentables:

- **Frequency Count**: un mapa **valor** -> **conteo**. Base de anagramas, mayoría, top-K, duplicados.
- **Two Sum / complemento**: por cada  $x$  busca si  $target - x$  ya está en el mapa. Una sola pasada,  $O(n)$ .
- **Group By (agrupación)**: mapa **clave\_canónica** -> **lista**. Para anagramas la clave es la palabra ordenada o su vector de conteo de letras.
- **Prefix-Sum + Hash**: para "subarreglos que suman K", guarda cuántas veces apareció cada suma acumulada; si **suma** - K ya se vio, hay subarreglos que suman K.

### 🧠 Diagrama

TWO SUM (target = 9)

nums = [2, 7, 11, 15]

visto = {}

x=2 -> falta 7? no. guarda 2

x=7 -> falta 2? SÍ (índice 0)  
=> par (0,1)

SUBARRAY SUM = K (K = 3)

nums = [1, 2, 1, 3], K = 3

prefix acumulada: 1, 3, 4, 7

mapa {suma\_prefija -> veces}, inicia {0:1}

suma=1 -> busca 1-3=-2? no. mapa{0:1,1:1}

suma=3 -> busca 3-3=0? SÍ (1 vez) -> +1



```

suma=4 -> busca 4-3=1? SÍ (1 vez) -> +1
suma=7 -> busca 7-3=4? SÍ (1 vez) -> +1
total subarreglos con suma 3: 3

```

## ⚙️ ¿Cómo funciona?

TWO SUM    nums=[2,7,11,15]    target=9    (mapa valor→índice)

| i | nums[i] | falta=9-x | ¿falta en mapa? | mapa después         |
|---|---------|-----------|-----------------|----------------------|
| 0 | 2       | 7         | no              | {2:0}                |
| 1 | 7       | 2         | SÍ (índice 0)   | → respuesta (0,1) OK |

SUBARRAY SUM = K    nums=[1,2,1,3]    K=3    mapa{sumaPrefija:veces}

| x | suma | busca suma-K | ¿está? +cuenta   | mapa (inicia {0:1})  |
|---|------|--------------|------------------|----------------------|
| 1 | 1    | -2           | no               | {0:1, 1:1}           |
| 2 | 3    | 0            | SÍ x1    total=1 | {0:1, 1:1, 3:1}      |
| 1 | 4    | 1            | SÍ x1    total=2 | {0:1, 1:1, 3:1, 4:1} |
| 3 | 7    | 4            | SÍ x1    total=3 | {..., 7:1}           |

Total subarreglos con suma 3 = 3    ([1,2] [3] [2,1] desde prefijos)  
Clave: el mapa "recuerda" sumas ya vistas → cada consulta O(1).

```

1 TWO SUM
2 - Un mapa valor->índice. Por cada x, si target-x ya está, encontraste
3 el par. Si no, guarda x. Una sola pasada.
4
5 SUBARRAY SUM = K
6 - Suma acumulada 'suma'. Un mapa 'cuántas veces salió cada suma'.
7 - Si (suma - K) está en el mapa, esos son subarreglos que suman K.
8 - Inicializa el mapa con {0: 1} (el prefijo vacío).
9
10 PSEUDOCÓDIGO (subarray sum = K)
11
12
13 función subarraysConSuma(nums, K):
14 conteo = {0: 1}; suma = 0; total = 0
15 PARA cada x en nums:
16 suma += x
17 SI (suma - K) en conteo: total += conteo[suma - K]
18 conteo[suma] += 1
19 return total
20
21 CÓDIGO C++ (Two Sum)

```

```

23
24 #include <vector>
25 #include <unordered_map>
26 using namespace std;
27
28 vector<int> twoSum(vector<int>& nums, int target) {
29 unordered_map<int,int> visto; // valor -> índice
30 for (int i = 0; i < (int)nums.size(); i++) {
31 int falta = target - nums[i];
32 if (visto.count(falta)) return {visto[falta], i};
33 visto[nums[i]] = i;
34 }
35 return {};
36 }

```

37 CÓDIGO C++ (subarray sum = K)

---

```

39
40
41 int subarraySum(vector<int>& nums, int k) {
42 unordered_map<int,int> conteo{{0, 1}}; // prefijo vacío
43 int suma = 0, total = 0;
44 for (int x : nums) {
45 suma += x;
46 if (conteo.count(suma - k)) total += conteo[suma - k];
47 conteo[suma]++;
48 }
49 return total;
50 }

```

51 CÓDIGO C++ (agrupar anagramas)

---

```

53
54
55 #include <string>
56 #include <algorithm>
57 vector<vector<string>> groupAnagrams(vector<string>& strs) {
58 unordered_map<string, vector<string>> grupos;
59 for (auto& s : strs) {
60 string clave = s;
61 sort(clave.begin(), clave.end()); // clave canónica
62 grupos[clave].push_back(s);
63 }
64 vector<vector<string>> res;
65 for (auto& [k, v] : grupos) res.push_back(v);
66 return res;
67 }

```

68 COMPLEJIDAD (Big-O)

---

70

| Patrón             | Tiempo                | Espacio        |
|--------------------|-----------------------|----------------|
| Frequency / TwoSum | $O(n)$                | $O(n)$         |
| Subarray sum = K   | $O(n)$                | $O(n)$         |
| Group anagrams     | $O(n \cdot k \log k)$ | $O(n \cdot k)$ |

\* k = longitud de la palabra en group anagrams.

## En entrevistas

Regla mental: si te descubres haciendo un bucle DENTRO de otro para "volver a buscar", pregúntate "¿un Hash Map recuerda esto en  $O(1)$ ?". Suele bajar  $O(n^2)$  a  $O(n)$ .

Prefix-Sum + Hash es el patrón estrella oculto: "Subarray Sum Equals K" (LeetCode 560), "Continuous Subarray Sum", "Contiguous Array". Inicializar el mapa con  $\{0: 1\}$  es el detalle que a todos se les olvida.

Group anagrams por conteo de letras: en vez de ordenar ( $k \log k$ ), usa un vector de 26 conteos como clave  $\rightarrow O(k)$ . Mejora que impresiona.

Hash Set vs Hash Map: si solo importa "¿existe?" usa un Set; si necesitas asociar un dato (índice, conteo) usa un Map.

## Resumen rápido

- Hash Map convierte "¿ya lo vi?" en  $O(1)$ : rompe bucles  $O(n^2)$ .
- Frequency count: base de anagramas, mayoría, duplicados, top-K.
- Two Sum: guarda visto, busca el complemento  $\text{target} - x$ .
- Prefix-Sum + Hash: subarreglos con suma K; inicia el mapa con 0:1.
- Group by: mapa de clave canónica  $\rightarrow$  lista.
- Set si solo importa existencia; Map si asocias datos.

# Capítulo 20

## 🔗 LRU Cache (Hash Map + Lista Doblemente Enlazada)

### 🤖 ¿Qué es?

Tu escritorio solo tiene espacio para 5 carpetas. Cada vez que usas una, la pones ENCIMA (la más reciente). Cuando llega una carpeta nueva y ya no cabe, tiras la de HASTA ABAJO: la que llevas más tiempo sin tocar ("Least Recently Used", la menos usada recientemente).

Una LRU Cache hace justo eso con datos, y el reto es lograr que TODO (guardar, buscar, mover a la cima y tirar la vieja) sea instantáneo  $O(1)$ . La combinación mágica: un Hash Map para encontrar rápido + una lista doblemente enlazada para reordenar rápido.

### 💬 Dicho de otra forma

Una LRU (Least Recently Used) Cache guarda hasta **capacity** pares clave-valor y, al llenarse, desaloja el elemento usado hace más tiempo. Debe hacer **get** y **put** en  $O(1)$ .

Ninguna estructura sola lo logra, así que se combinan dos:

- **Hash Map clave -> nodo:** encuentra cualquier elemento en  $O(1)$ .
- **Lista doblemente enlazada:** mantiene el orden de uso. El frente es el más reciente; el fondo, el candidato a desalojar. Mover un nodo o quitarlo es  $O(1)$  porque tienes punteros a ambos vecinos.

Cada acceso (**get** o **put**) mueve el nodo al frente. Al exceder la capacidad, se elimina el nodo del fondo (y su clave del mapa).

### 🧠 Diagrama

Capacidad = 3. Frente = más reciente, Fondo = menos reciente.

```
put(1) put(2) put(3):
FRENTE [3] <-> [2] <-> [1] FONDO
mapa: {1:n1, 2:n2, 3:n3}
```

```
get(1) -> mueve 1 al frente:
FRENTE [1] <-> [3] <-> [2] FONDO
```

```
put(4) -> lleno! desaloja el fondo (2):
```

```
FRENTE [4] <-> [1] <-> [3] FONDO
```

```
mapa: {1,3,4} (el 2 se eliminó del mapa y de la lista)
```

Nodos "centinela" (head/tail falsos) evitan casos borde con NULL.

## ⚙️ ¿Cómo funciona?

ESTADO tras cada operación (capacidad = 2) FRENTE=reciente

```
put(1,A) head <-> [1:A] <-> tail mapa{1}
```

```
put(2,B) head <-> [2:B] <-> [1:A] <-> tail mapa{1,2}
```

```
get(1) → mueve 1 al frente
```

```
head <-> [1:A] <-> [2:B] <-> tail devuelve A
```

```
put(3,C) → LLENO: desaloja el del FONDO (2:B, menos reciente)
```

```
head <-> [3:C] <-> [1:A] <-> tail mapa{1,3} (2 eliminado)
```

```
get(2) → no está → devuelve -1
```

POR QUÉ lista DOBLE: para borrar el fondo en  $O(1)$  necesitas su 'prev'. El mapa da el nodo en  $O(1)$ ; la lista lo reordena en  $O(1)$ .

mapa: clave → puntero al nodo



(los nodos viven en la lista; el mapa solo apunta a ellos)

### 1 CÓMO FUNCIONA (todo $O(1)$ )

- 2 - Hash Map clave -> puntero al nodo de la lista.
- 3 - Lista doble con centinelas head/tail falsos.
- 4 - get(k): si existe, mueve su nodo al frente y devuelve el valor.
- 5 - put(k,v): si existe, actualiza y mueve al frente; si no, crea al frente. Si superas la capacidad, borra el nodo antes del tail.

### 8 PSEUDOCÓDIGO

```
11 get(clave):
```

```
12 SI clave no en mapa: return -1
```

```
13 nodo = mapa[clave]; moverAlFrente(nodo)
```

```
14 return nodo.valor
```

```
16 put(clave, valor):
```

```
17 SI clave en mapa:
```

```

18 nodo = mapa[clave]; nodo.valor = valor; moverAlFrente(nodo)
19 SINO:
20 SI tamaño == capacidad:
21 viejo = tail.prev; quitar(viejo); borrar mapa[viejo.clave]
22 nodo = nuevo Nodo(clave, valor)
23 insertarAlFrente(nodo); mapa[clave] = nodo
24

```

#### CÓDIGO C++

```

26
27
28 #include <unordered_map>
29 using namespace std;
30
31 class LRUCache {
32 struct Node { int key, val; Node *prev, *next; };
33 int cap;
34 unordered_map<int, Node*> mp;
35 Node *head, *tail; // centinelas
36
37 void remove(Node* n) { n->prev->next = n->next; n->next->prev = n->
 prev; }
38 void addFront(Node* n) {
39 n->next = head->next; n->prev = head;
40 head->next->prev = n; head->next = n;
41 }
42 public:
43 LRUCache(int capacity) : cap(capacity) {
44 head = new Node(); tail = new Node();
45 head->next = tail; tail->prev = head;
46 }
47 int get(int key) {
48 if (!mp.count(key)) return -1;
49 Node* n = mp[key];
50 remove(n); addFront(n); // marcar como reciente
51 return n->val;
52 }
53 void put(int key, int value) {
54 if (mp.count(key)) {
55 Node* n = mp[key]; n->val = value;
56 remove(n); addFront(n);
57 return;
58 }
59 if ((int)mp.size() == cap) { // desalojar el menos reciente
60 Node* lru = tail->prev;
61 remove(lru); mp.erase(lru->key); delete lru;
62 }
63 Node* n = new Node{key, value, nullptr, nullptr};
64 addFront(n); mp[key] = n;
65 }
66 };
67

```

## 68 COMPLEJIDAD (Big-O)

69

70

71

72

73

74

75

76

| Operación | Tiempo | Espacio              |
|-----------|--------|----------------------|
| get       | $O(1)$ | $O(\text{capacity})$ |
| put       | $O(1)$ | $O(\text{capacity})$ |

\* El  $O(1)$  es EL punto del problema; una lista simple daría  $O(n)$ .



## En entrevistas

"LRU Cache" (LeetCode 146): pregunta top de Google/Meta/Amazon. La respuesta esperada es EXACTAMENTE Hash Map + lista doble. Menos que eso no da  $O(1)$ .

Nodos centinela (dummy head/tail): eliminan todos los casos borde de NULL al insertar/borrar. No los omitas: simplifican mucho el código.

¿Por qué lista DOBLE y no simple? Para borrar un nodo en  $O(1)$  necesitas su nodo anterior. La lista simple obligaría a recorrer  $O(n)$ .

Variante LFU (Least Frequently Used): desaloja por frecuencia, no por recencia. Más difícil; suele necesitar un mapa extra de frecuencias. Menciónala si te la piden.



## Resumen rápido

- LRU: al llenarse, desaloja el elemento usado hace más tiempo.
- Hash Map (buscar  $O(1)$ ) + lista doble (reordenar  $O(1)$ ).
- Frente = más reciente; fondo = candidato a desalojar.
- get/put mueven el nodo al frente; al exceder capacidad, borra el fondo.
- Centinelas head/tail eliminan casos borde con NULL.
- Lista DOBLE es obligatoria para borrar en  $O(1)$ .

# Capítulo 21

## Tries (Árboles de Prefijos)

### ¿Qué es?

Un diccionario físico donde navegas letra por letra. Para buscar la palabra "GATO": 1. Vas a la sección G. 2. Dentro de G, buscas la A. 3. Dentro de GA, buscas la T. 4. Dentro de GAT, buscas la O. ¡Encontrado!

Cada nivel del árbol representa una letra. Si compartes el mismo inicio (prefijo), compartes los mismos nodos. Por ejemplo, "GATO" y "GOTA" comparten la 'G', pero luego se bifurcan.

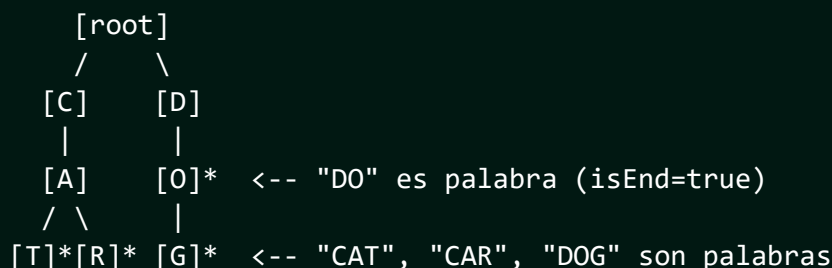
### Dicho de otra forma

Un Trie (se pronuncia como "try") es un árbol especializado para guardar strings. Cada nodo representa un caracter.

Su gran ventaja es la eficiencia para búsquedas de prefijos y autocompletado. A diferencia de un Hash Map, el tiempo de búsqueda en un Trie depende únicamente de la longitud de la palabra (L), no del número de palabras guardadas (N).

### Diagrama

Insertando: "CAT", "CAR", "DO", "DOG"



\* : Indica el final de una palabra (isEnd = true).

1. `insert(word)`: Empiezas en el root. Para cada letra, si no existe el hijo, lo creas. Al final de la palabra, marcas `isEnd = true`.
2. `search(word)`: Sigues el camino de las letras. Si llegas al final y el nodo tiene `isEnd = true`, la palabra existe.
3. `startsWith(prefix)`: Igual que `search`, pero no importa si `isEnd` es `true`; con que el camino exista, el prefijo es válido.  $O(L)$ .



## PSEUDOCÓDIGO

---

*// Insertar palabra en el Trie*

función insertar(palabra):

    actual = root

**PARA** cada letra EN palabra:

**SI** actual.hijos[letra] NO existe:

            actual.hijos[letra] = nuevo Nodo()

        actual = actual.hijos[letra]

    actual.esFinDePalabra = **VERDADERO**

## CÓDIGO C++

---

**#include** <iostream>

**#include** <unordered\_map>

**#include** <string>

**using namespace** std;

**struct** TrieNode {

**unordered\_map**<char, TrieNode\*> children;

**bool** isEndOfWord = **false**;

};

**class** Trie {

private:

    TrieNode\* root;

public:

    Trie() { root = **new** TrieNode(); }

*// Insertar -> O(L) donde L es la longitud de la palabra*

**void** insert(**string** word) {

        TrieNode\* curr = root;

**for** (char c : word) {

**if** (curr->children.find(c) == curr->children.end()) {

                curr->children[c] = **new** TrieNode();

            }

            curr = curr->children[c];

        }

        curr->isEndOfWord = **true**;

    }

*// Buscar palabra completa -> O(L)*

**bool** search(**string** word) {

        TrieNode\* curr = root;

**for** (char c : word) {

**if** (curr->children.find(c) == curr->children.end()) **return**

```

 false;
57 curr = curr->children[c];
58 }
59 return curr->isEndOfWord;
60 }
61
62 // Buscar prefijo -> O(L)
63 bool startsWith(string prefix) {
64 TrieNode* curr = root;
65 for (char c : prefix) {
66 if (curr->children.find(c) == curr->children.end()) return
 false;
67 curr = curr->children[c];
68 }
69 return true;
70 }
71 };
72
73 int main() {
74 Trie t;
75 t.insert("gato");
76 t.insert("goma");
77
78 cout << "¿Existe gato? " << t.search("gato") << endl; // 1 (true)
79 cout << "¿Existe prefijo go? " << t.startsWith("go") << endl; // 1 (
 true)
80 cout << "¿Existe gatito? " << t.search("gatito") << endl; // 0 (
 false)
81
82 return 0;
83 }
84
85 COMPLEJIDAD (Big-O)
86
87
88
89
90
91
92
93
94
95
96

```

| Operación  | Tiempo | Espacio                            |
|------------|--------|------------------------------------|
| Insert     | $O(L)$ | $O(\text{ALPHABET\_SIZE} * L * N)$ |
| Search     | $O(L)$ | $O(1)$                             |
| StartsWith | $O(L)$ | $O(1)$                             |

\* L = Longitud de la palabra.  
 \* N = Número de palabras.  
 \* Nota: El espacio puede ser grande si las palabras no comparten prefijos.

## En entrevistas

Cuándo usar: Si el problema menciona "autocompletar", "diccionario", o "buscar todas las palabras que empiecen con...", usa un Trie.

Trie + DFS: Una combinación clásica para problemas como "Word Search II" (encontrar palabras en una cuadrícula de letras).

Hash Map vs Trie: - Hash Map: Busca la palabra completa en  $O(1)$  promedio.  
- Trie: Puede buscar prefijos y es más rápido si hay muchas colisiones en el hash map.

Memoria: Si te preocupa el espacio, menciona que se puede usar un "Compressed Trie" (Radix Tree) donde los nodos con un solo hijo se fusionan.

## Resumen rápido

- Árbol de letras, nivel por nivel.
  - Ideal para búsquedas de prefijos y autocompletado.
  - Tiempo de búsqueda dependiente solo del largo de la palabra.
  - 'startsWith' es su operación estrella.
  - Puede usar mucha memoria, pero ahorra espacio si hay muchos prefijos compartidos.
- 
-

# Capítulo 22

## Union-Find (Disjoint Set Union - DSU)

### ¿Qué es?

Los clubes de la escuela . Al principio, cada estudiante es su propio club (un conjunto de 1 persona).

Cuando dos personas se hacen amigas, sus clubes se UNEN. Para saber si dos personas están en el mismo club, sigues la cadena de amistades hasta llegar al "líder" del grupo. Si ambos tienen el mismo líder, entonces ¡están en el mismo club!

Es una estructura que gestiona grupos de elementos que no se traslapan y te permite unirlos o preguntar si están conectados muy rápido.

### Dicho de otra forma

Union-Find (o DSU) es una estructura de datos que mantiene un conjunto de elementos divididos en varios conjuntos disjuntos. Soporta dos operaciones principales:

1. Find: Determina a qué conjunto pertenece un elemento (devuelve el "representante" o "líder" del conjunto). 2. Union: Une dos conjuntos en uno solo.

Con optimizaciones como "Path Compression" y "Union by Rank", estas operaciones se vuelven prácticamente instantáneas  $O(\alpha(n))$ .

### Diagrama

Estado Inicial: {0} {1} {2} {3} {4}  
(Cada uno apunta a sí mismo como líder)

Union(0, 1): {0, 1} {2} {3} {4}  
[0] ← [1] (0 es el líder de 1)

Union(1, 2): {0, 1, 2} {3} {4}  
[0] ← [1] ← [2]

Path Compression (al buscar el líder de 2):  
[0] ← [1], [0] ← [2] (Ahora 2 apunta directo al líder 0)

<sup>1</sup> 1. Find(i): Sigue los punteros `parent` desde `i` hasta llegar al

```

2 nodo que es su propio padre. Ese es el representante.
3 - Optimización (Path Compression): Mientras subes, haces que
4 cada nodo apunte directo al representante final.
5 2. Union(i, j): Encuentras los representantes de `i` y `j`. Si son
6 diferentes, haces que uno apunte al otro.
7 - Optimización (Union by Rank): Siempre haces que el árbol más
8 bajito cuelgue del árbol más alto para mantenerlo balanceado.
9

```

#### 10 PSEUDOCÓDIGO

---

```

11
12
13 // Find con Path Compression
14 función buscar(i):
15 SI parent[i] == i: return i
16 parent[i] = buscar(parent[i]) // ¡Compresión aquí!
17 return parent[i]
18
19 // Union por Rango
20 función unir(i, j):
21 raiz_i = buscar(i)
22 raiz_j = buscar(j)
23 SI raiz_i != raiz_j:
24 SI rango[raiz_i] < rango[raiz_j]:
25 parent[raiz_i] = raiz_j
26 SINO:
27 parent[raiz_j] = raiz_i
28 SI rango[raiz_i] == rango[raiz_j]:
29 rango[raiz_i]++
30

```

#### 31 CÓDIGO C++

---

```

32
33
34 #include <iostream>
35 #include <vector>
36 using namespace std;
37
38 class UnionFind {
39 vector<int> parent;
40 vector<int> rank;
41 public:
42 UnionFind(int n) {
43 parent.resize(n);
44 rank.resize(n, 0);
45 for(int i = 0; i < n; i++) parent[i] = i;
46 }
47
48 int find(int i) {
49 if (parent[i] == i) return i;
50 return parent[i] = find(parent[i]); // Path compression
51 }

```

```

52
53 void unite(int i, int j) {
54 int root_i = find(i);
55 int root_j = find(j);
56 if (root_i != root_j) {
57 if (rank[root_i] < rank[root_j]) parent[root_i] = root_j;
58 else if (rank[root_i] > rank[root_j]) parent[root_j] =
59 root_i;
60 else {
61 parent[root_i] = root_j;
62 rank[root_j]++;
63 }
64 }
65
66 bool connected(int i, int j) {
67 return find(i) == find(j);
68 }
69 };
70
71 int main() {
72 UnionFind uf(5);
73 uf.unite(0, 1);
74 uf.unite(2, 3);
75
76 cout << "¿0 y 1 conectados? " << uf.connected(0, 1) << endl; // 1 (
77 true)
78 cout << "¿0 y 2 conectados? " << uf.connected(0, 2) << endl; // 0 (
79 false)
80
81 uf.unite(1, 2);
82 cout << "¿0 y 2 conectados ahora? " << uf.connected(0, 2) << endl;
83 // 1 (true)
84
85 return 0;
86 }

```

#### COMPLEJIDAD (Big-O)

| Operación | Tiempo         | En la práctica             |
|-----------|----------------|----------------------------|
| Find      | $O(\alpha(n))$ | Casi $O(1)$                |
| Union     | $O(\alpha(n))$ | Casi $O(1)$                |
| Connected | $O(\alpha(n))$ | Casi $O(1)$                |
| Espacio   | $O(n)$         | Para guardar parents/ranks |

\* $\alpha(n)$  es la función inversa de Ackermann, que crece tan lento que para cualquier valor real de  $n$ , es menor a 5.

## En entrevistas

Cuándo usar: Problemas de conectividad, componentes conectados en grafos, o detectar ciclos en grafos no dirigidos.

Alternativa a DFS/BFS: Para saber si dos nodos están conectados, Union-Find es a veces más rápido y fácil de implementar que un recorrido completo.

Optimización: NUNCA olvides la "Path Compression". Sin ella, Union-Find puede degradarse a  $O(n)$ .

Problema Clásico: "Number of Islands" o "Redundant Connection".

## Resumen rápido

- Gestiona grupos de elementos y sus uniones.
- Operaciones: 'find' (quién es el jefe) y 'union' (hacerse amigos).
- Casi  $O(1)$  gracias a Path Compression y Union by Rank.
- Ideal para detectar ciclos y conectividad en grafos.
- Solo funciona para conjuntos disjuntos (un elemento no puede estar en dos grupos a la vez).

---

---





# Capítulo 23



## Representación de Grafos



### ¿Qué es?

Un mapa de ciudades con carreteras entre ellas.

En computación, las ciudades se llaman Nodos (o Vértices) y las carreteras se llaman Aristas (o Edges). Un grafo es simplemente un conjunto de nodos conectados entre sí.

Puedes tener: - Grafos Dirigidos: Carreteras de un solo sentido ( $A \rightarrow B$ ). - Grafos No Dirigidos: Carreteras de doble sentido ( $A \leftrightarrow B$ ). - Grafos Ponderados: Carreteras con costo o distancia ( $A \xrightarrow{-5} B$ ).



### Dicho de otra forma

Un Grafo  $G = (V, E)$  es una estructura no lineal. Para trabajar con ellos en código, necesitamos guardarlos en memoria de alguna forma.

Las dos formas más comunes son: 1. Adjacency Matrix (Matriz de Adyacencia): Una tabla de  $V \times V$ . 2. Adjacency List (Lista de Adyacencia): Un array de listas.

La elección depende de si el grafo es "Denso" (muchas conexiones) o "Disperso" (pocas conexiones).



### Diagrama

Grafo de ejemplo: A-B, A-C, B-C, C-D

Adjacency Matrix (4x4):

|   | A | B | C | D |
|---|---|---|---|---|
| A | 0 | 1 | 1 | 0 |
| B | 1 | 0 | 1 | 0 |
| C | 1 | 1 | 0 | 1 |
| D | 0 | 0 | 1 | 0 |

Adjacency List:

A: [B, C]  
B: [A, C]  
C: [A, B, D]  
D: [C]

- En la Matrix: 1 significa que hay conexión, 0 que no.
- En la List: Cada nodo tiene su propia lista de vecinos.

#### 1. Adjacency Matrix:

- Pros: Muy rápido  $O(1)$  para saber si A y B están conectados.
- Contras: Usa mucha memoria  $O(V^2)$ , incluso si hay pocos edges.
- Cuándo usar: Grafos pequeños o muy densos.

## 2. Adjacency List:

- Pros: Ahorra mucha memoria  $O(V + E)$ . Es la más usada.
- Contras: Saber si A y B están conectados puede tardar  $O(V)$ .
- Cuándo usar: Casi siempre (la mayoría de los grafos reales son dispersos).

### PSEUDOCÓDIGO

---

*// Agregar arista en Lista de Adyacencia (no dirigido)*

función agregarArista(u, v):

    adj[u].agregar(v)

    adj[v].agregar(u)

*// Agregar arista en Matriz de Adyacencia*

función agregarAristaMatrix(u, v):

    matrix[u][v] = 1

    matrix[v][u] = 1

### CÓDIGO C++

---

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
int main() {
```

```
 int V = 4; // Número de vértices
```

```
 // 1. Representación con Lista de Adyacencia (Recomendado)
```

```
 vector<vector<int>> adj(V);
```

```
 // Agregar A-B (0-1), A-C (0-2), B-C (1-2), C-D (2-3)
```

```
 adj[0].push_back(1); adj[1].push_back(0);
```

```
 adj[0].push_back(2); adj[2].push_back(0);
```

```
 adj[1].push_back(2); adj[2].push_back(1);
```

```
 adj[2].push_back(3); adj[3].push_back(2);
```

```
 cout << "Lista de Adyacencia de C (nodo 2): ";
```

```
 for(int vecino : adj[2]) {
```

```
 cout << vecino << " ";
```

```
 }
```

```
 cout << endl;
```

```
 // 2. Representación con Matriz de Adyacencia
```

```
 vector<vector<int>> matrix(V, vector<int>(V, 0));
```

```
 matrix[0][1] = 1; matrix[1][0] = 1;
```

```
 // ... completar resto
```

```
 return 0;
```

```

56 }
57
58 COMPLEJIDAD (Comparativa)
59
60
61 Operación | Matrix | List
62
63 Espacio | $O(V^2)$ | $O(V + E)$
64 Agregar Edge | $O(1)$ | $O(1)$
65 Verificar Edge | $O(1)$ | $O(\text{Grado del nodo})$
66 Iterar Vecinos | $O(V)$ | $O(\text{Grado del nodo})$
67
68 * V = Vértices, E = Edges (Aristas).

```

### En entrevistas

Default: En el 90 % de las entrevistas, usa Adjacency List. Es más eficiente en memoria y espacio.

Representación con Map: A veces los nodos no son números (0, 1, 2) sino nombres ("CDMX", "NY"). En ese caso, usa: `'unordered_map<string, vector<string> adj;'`

Edge List: Existe una tercera forma llamada "Edge List" que es simplemente un array de pares `'[(0,1), (0,2), ...]'`. Útil para algoritmos como Kruskal.

Grafo Implícito: A veces el grafo no te lo dan, sino que es una matriz (grid) donde puedes moverte arriba, abajo, izq, der. Ahí el "grafo" son las celdas y sus vecinos.

### Resumen rápido

• Nodos = Ciudades, Aristas = Caminos. • Matrix: Rápida pero traga mucha memoria  $O(V^2)$ . • List: Eficiente en memoria, la favorita de los devs  $O(V + E)$ . • Dirigido (flecha) vs No Dirigido (línea). • Ponderado (tiene peso/costo) vs No Ponderado. • Grado de un nodo = cuántos vecinos tiene.

# Capítulo 24

## 🕒 BFS y DFS (Recorridos de Grafos)

### 🤖 ¿Qué es?

Dos formas de explorar un mundo desconocido :

1. BFS (Breadth-First Search): Como lanzar una piedra al agua . Ves las ondas expandirse nivel por nivel. Primero visitas a todos tus vecinos directos, luego a los vecinos de tus vecinos, y así sucesivamente. ¡No dejas a nadie atrás en tu nivel!
2. DFS (Depth-First Search): Como explorar un laberinto . Sigues un solo camino hasta el fondo, hasta que ya no puedas avanzar. Cuando llegas a un callejón sin salida, regresas (backtrack) y pruebas el siguiente camino disponible.

### 💬 Dicho de otra forma

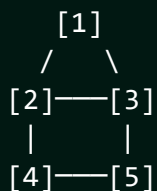
Son los dos algoritmos fundamentales para recorrer cualquier grafo o árbol.

- BFS usa una Queue (FIFO) y garantiza encontrar el camino más corto en grafos no ponderados. - DFS usa un Stack (LIFO) o Recursión. Es ideal para explorar todas las posibilidades o detectar ciclos.

En ambos casos, es CRUCIAL usar una estructura 'visited' (como un array de bools o un set) para no entrar en bucles infinitos.

### 🧠 Diagrama

Grafo:



BFS desde 1:  $1 \rightarrow 2, 3 \rightarrow 4, 5$

Orden: [1, 2, 3, 4, 5] (Nivel por nivel)

DFS desde 1:  $1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 3$

Orden: [1, 2, 4, 5, 3] (Hasta el fondo)

```

1 BFS (con Queue):
2 1. Metes el nodo inicial a la Queue y lo marcas como visitado.
3 2. Mientras la Queue no esté vacía:
4 - Sacas el nodo del frente.
5 - Metes a todos sus vecinos NO visitados a la Queue y los marcas.
6
7 DFS (con Recursión):
8 1. Marcas el nodo actual como visitado.
9 2. Para cada vecino NO visitado:
10 - Llamas a la función DFS recursivamente con ese vecino.
11

```

## 12 PSEUDOCÓDIGO

---

```

13
14
15 // BFS iterativo
16 función BFS(inicio):
17 q = crear Queue
18 q.enqueue(inicio)
19 marcar inicio como visitado
20 MIENTRAS q NO vacía:
21 u = q.dequeue()
22 PARA cada vecino v de u:
23 SI v NO visitado:
24 q.enqueue(v)
25 marcar v visitado
26
27 // DFS recursivo
28 función DFS(u):
29 marcar u como visitado
30 PARA cada vecino v de u:
31 SI v NO visitado:
32 DFS(v)
33

```

## 34 CÓDIGO C++

---

```

35
36
37 #include <iostream>
38 #include <vector>
39 #include <queue>
40 #include <stack>
41 using namespace std;
42
43 void bfs(int start, vector<vector<int>>& adj, vector<bool>& visited) {
44 queue<int> q;
45 q.push(start);
46 visited[start] = true;
47
48 while (!q.empty()) {
49 int u = q.front(); q.pop();
50 cout << u << " ";

```

```

51 for (int v : adj[u]) {
52 if (!visited[v]) {
53 visited[v] = true;
54 q.push(v);
55 }
56 }
57 }
58 }
59
60 void dfs(int u, vector<vector<int>>& adj, vector<bool>& visited) {
61 visited[u] = true;
62 cout << u << " ";
63 for (int v : adj[u]) {
64 if (!visited[v]) dfs(v, adj, visited);
65 }
66 }
67
68 int main() {
69 int V = 6;
70 vector<vector<int>> adj(V);
71 // Definir grafo...
72
73 vector<bool> visited(V, false);
74 cout << "Recorrido BFS: ";
75 bfs(0, adj, visited); // Suponiendo que empezamos en 0
76
77 fill(visited.begin(), visited.end(), false);
78 cout << "\nRecorrido DFS: ";
79 dfs(0, adj, visited);
80
81 return 0;
82 }
83
84 COMPLEJIDAD (Big-O)

```

| Algoritmo | Tiempo     | Espacio |
|-----------|------------|---------|
| BFS       | $O(V + E)$ | $O(V)$  |
| DFS       | $O(V + E)$ | $O(V)$  |

\* V = Vértices, E = Edges.

\* El espacio  $O(V)$  es para la Queue/Stack y el array `visited`.

## En entrevistas

BFS para Shortest Path: Si te piden el camino más corto en un grafo SIN pesos (o una cuadrícula), BFS es la única opción correcta.

DFS para Backtracking: Si el problema pide "encuentra todas las rutas posibles" o "prueba todas las combinaciones", DFS es tu mejor amigo.

Cycle Detection: - Undirected: Si encuentras un vecino visitado que no es tu padre. - Directed: Necesitas 3 estados (Blanco, Gris, Negro). Si ves un Gris, ¡hay ciclo!

Nivel de cada nodo: BFS es excelente para guardar la distancia (nivel) de cada nodo respecto al origen.

## Resumen rápido

- BFS = Queue = Nivel por nivel = Camino más corto.
- DFS = Stack/Recursión = Hasta el fondo = Exploración total.
- ¡Siempre usa un array 'visited'!
- Ambos tardan  $O(V + E)$ .
- Úsalos en matrices (grid), redes sociales, mapas y juegos.

## Capítulo 25

# BFS Avanzado (Grid, Multi-source y 0-1 BFS)

### ¿Qué es?

El BFS normal explora un mapa nivel por nivel. Estas variantes son el mismo BFS con pequeños giros para problemas muy comunes en entrevistas:

- Grid: el "grafo" es una cuadrícula (un laberinto o mapa). Cada celda se conecta con sus 4 vecinos (arriba, abajo, izquierda, derecha).
- Multi-source: en vez de empezar en un punto, empiezas en VARIOS a la vez. Como varios incendios propagándose simultáneamente.
- 0-1 BFS: aristas que cuestan solo 0 o 1. Con una doble cola (deque) obtienes el camino más corto sin necesitar Dijkstra.

### Dicho de otra forma

Muchos problemas de "camino más corto" no vienen como grafos explícitos, sino como matrices. Tratar cada celda como un nodo y sus 4 (u 8) vecinos como aristas convierte el problema en un BFS estándar.

**Multi-source BFS:** metes todos los orígenes en la Queue al inicio (distancia 0). El BFS expande todos los frentes en paralelo, dando la distancia mínima a CUALQUIER origen. Evita correr BFS por cada fuente.

**0-1 BFS:** cuando los pesos son solo 0 o 1, una **deque** sustituye a la cola de prioridad de Dijkstra: las aristas de peso 0 van al FRENTE y las de peso 1 al FINAL. Resultado:  $O(V + E)$  en vez de  $O(E \log V)$ .

### Diagrama

GRID (4 direcciones)

```
. . . #
. # . .
. . . .
. . .
```

vecinos de  
(r,c):  
(r+1,c)(r-1,c)  
(r,c+1)(r,c-1)

MULTI-SOURCE (2 incendios)

Inicio: F en (0,0) y (2,3)  
F . . . dist=0 en ambos  
. . . F y crece hacia  
. . . . afuera a la vez.

dr[] = {1,-1,0,0}    dc[] = {0,0,1,-1}





## CÓDIGO C++ (BFS en grid)

---

```

24
25
26
27
28 #include <vector>
29 #include <queue>
30 using namespace std;
31
32 int dr[] = {1, -1, 0, 0};
33 int dc[] = {0, 0, 1, -1};
34
35 vector<vector<int>> bfsGrid(vector<vector<int>>& grid,
36 vector<pair<int,int>>& sources) {
37 int R = grid.size(), C = grid[0].size();
38 vector<vector<int>> dist(R, vector<int>(C, -1));
39 queue<pair<int,int>> q;
40 for (auto [r, c] : sources) { dist[r][c] = 0; q.push({r, c}); }
41
42 while (!q.empty()) {
43 auto [r, c] = q.front(); q.pop();
44 for (int d = 0; d < 4; d++) {
45 int nr = r + dr[d], nc = c + dc[d];
46 if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
47 if (grid[nr][nc] == 1 || dist[nr][nc] != -1) continue; //
 muro/visto
48 dist[nr][nc] = dist[r][c] + 1;
49 q.push({nr, nc});
50 }
51 }
52 return dist;
53 }

```

## CÓDIGO C++ (0-1 BFS con deque)

---

```

56
57
58 #include <deque>
59 // adj[u] = lista de {vecino, peso} con peso 0 o 1
60 vector<int> zeroOneBFS(int V, vector<vector<pair<int,int>>>& adj, int
 src) {
61 vector<int> dist(V, INT_MAX);
62 deque<int> dq;
63 dist[src] = 0; dq.push_front(src);
64 while (!dq.empty()) {
65 int u = dq.front(); dq.pop_front();
66 for (auto [v, w] : adj[u]) {
67 if (dist[u] + w < dist[v]) {
68 dist[v] = dist[u] + w;
69 if (w == 0) dq.push_front(v); // peso 0 -> al frente
70 else dq.push_back(v); // peso 1 -> al final

```

```

71 }
72 }
73 }
74 return dist;
75 }

```

76

77 COMPLEJIDAD (Big-O)

---

| 78 Variante     | 79 Tiempo  | 80 Nota                |
|-----------------|------------|------------------------|
| 81              |            |                        |
| 82 BFS en grid  | $O(R * C)$ | R filas, C columnas    |
| 83 Multi-source | $O(R * C)$ | igual, varios orígenes |
| 84 0-1 BFS      | $O(V + E)$ | reemplaza a Dijkstra   |

## En entrevistas

”Rotting Oranges” (LeetCode 994): multi-source BFS puro. Todas las naranjas podridas son fuentes; el resultado es el tiempo hasta podrir todo.

”01 Matrix” (LeetCode 542): distancia de cada celda al 0 más cercano = multi-source BFS desde todos los ceros a la vez.

8 direcciones vs 4: lee bien el enunciado. Si permiten diagonales, agrega las 4 direcciones diagonales a `dr[]/dc[]`.

0-1 BFS gana a Dijkstra cuando los pesos son binarios: mismo resultado, más rápido y sin heap. Buen detalle para destacar.

## Resumen rápido

- Grid = grafo implícito: cada celda es un nodo, 4 u 8 vecinos.
  - Usa `dr[]/dc[]` para recorrer direcciones limpiamente.
  - Multi-source: encola TODAS las fuentes con dist 0 al inicio.
  - 0-1 BFS: deque, peso 0 al frente, peso 1 al final;  $O(V+E)$ .
  - Clásicos: Rotting Oranges, 01 Matrix, laberintos.
- 
-

# Capítulo 26

## Algoritmo de Dijkstra (Camino más Corto)

### ¿Qué es?

El GPS de tu teléfono . Cuando pides la ruta más rápida de tu casa al trabajo, el GPS no intenta probar absolutamente todos los caminos posibles al azar.

En su lugar, siempre expande primero el camino que parece más prometedor (el más corto que ha encontrado hasta ahora). Va construyendo la ruta óptima paso a paso, actualizando sus cálculos si encuentra un "atajo" mejor.

Es un algoritmo "Greedy" (Codicioso) que encuentra el camino más corto desde un nodo origen a todos los demás nodos en un grafo con pesos.

### Dicho de otra forma

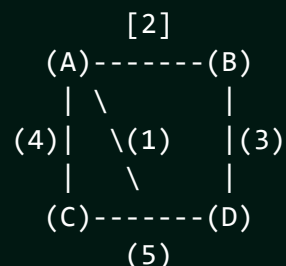
Dijkstra resuelve el problema del "Single-Source Shortest Path" en un grafo ponderado (weighted graph).

Funciona manteniendo una lista de distancias tentativas y usando un Min-Heap (Priority Queue) para elegir siempre el nodo con la distancia mínima actual.

Restricción Vital: NO funciona si el grafo tiene aristas con PESOS NEGATIVOS. (Para eso se usa Bellman-Ford).

### Diagrama

Grafo Ponderado:



Distancias desde A:

1. Inicio: {A:0, B:∞, C:∞, D:∞}
2. Visitar A: Vecinos B(2), C(4), D(1).  
Heap elige D(1).

3. Visitar D: Vecinos B(1+3=4), C(1+5=6).  
     ¿4 < dist[B]? No. ¿6 < dist[C]? No.
4. Heap elige B(2).
5. Heap elige C(4).

Final: {A:0, B:2, C:4, D:1}

1. Inicializas todas las distancias como Infinito  $\infty()$ , excepto el nodo origen que es 0.
2. Metes (distancia 0, origen) a una Priority Queue (Min-Heap).
3. Mientras la Queue no esté vacía:
  - Sacas el nodo `u` con la menor distancia.
  - Si la distancia sacada es mayor a la que ya conocemos, ignora.
  - Para cada vecino `v` de `u`:
    - Calculas `nueva\_dist = dist[u] + peso(u, v)`.
    - SI `nueva\_dist < dist[v]`:
      - Actualizas `dist[v] = nueva\_dist`.
      - Metes `(nueva\_dist, v)` a la Priority Queue.

#### PSEUDOCÓDIGO

---

```

función Dijkstra(grafo, origen):
 distancias = array de tamaño V con INFINITO
 distancias[origen] = 0
 pq = crear Min-Heap de pares (distancia, nodo)
 pq.push(0, origen)

 MIENTRAS pq NO vacía:
 (d, u) = pq.pop_min()
 SI d > distancias[u]: continuar

 PARA cada vecino v de u con peso w:
 SI distancias[u] + w < distancias[v]:
 distancias[v] = distancias[u] + w
 pq.push(distancias[v], v)

```

#### CÓDIGO C++

---

```

#include <iostream>
#include <vector>
#include <queue>
using namespace std;

const int INF = 1e9;

void dijkstra(int start, vector<vector<pair<int, int>>>& adj) {
 int V = adj.size();
 vector<int> dist(V, INF);

```

```

44 // Min-heap: priority_queue<T, container, comparison>
45 priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<
 int, int>>> pq;
46
47 dist[start] = 0;
48 pq.push({0, start});
49
50 while (!pq.empty()) {
51 int d = pq.top().first;
52 int u = pq.top().second;
53 pq.pop();
54
55 if (d > dist[u]) continue;
56
57 for (auto& edge : adj[u]) {
58 int v = edge.first;
59 int weight = edge.second;
60
61 if (dist[u] + weight < dist[v]) {
62 dist[v] = dist[u] + weight;
63 pq.push({dist[v], v});
64 }
65 }
66 }
67
68 for(int i = 0; i < V; i++)
69 cout << "Distancia a " << i << ": " << dist[i] << endl;
70 }
71
72 int main() {
73 int V = 4;
74 vector<vector<pair<int, int>>> adj(V);
75 // Definir grafo con pesos...
76 dijkstra(0, adj);
77 return 0;
78 }

```

COMPLEJIDAD (Big-O)

| Operación | Tiempo              | Razón              |
|-----------|---------------------|--------------------|
| Dijkstra  | $O((V + E) \log V)$ | Priority Queue     |
| Espacio   | $O(V + E)$          | Grafo + distancias |

\* V = Vértices, E = Aristas.

## En entrevistas

Cuándo usar: "Shortest Path" en grafos con PESOS positivos. Si no hay pesos, usa BFS (es más rápido  $O(V+E)$ ).

Pesos Negativos: Si el entrevistador pregunta "¿Y si hay pesos negativos?", responde: "Dijkstra fallaría, usaría Bellman-Ford".

C++ Priority Queue: Por defecto es un MAX-heap. Para Dijkstra necesitas un MIN-heap, así que usa 'greater<>' o guarda las distancias negativas.

Variante: "Cheapest Flights Within K Stops" es un problema común de Google que modifica Dijkstra limitando la profundidad.

## Resumen rápido

- Algoritmo Codicioso (Greedy).
  - Usa un Min-Heap para elegir siempre el camino más corto actual.
  - Encuentra distancias mínimas desde UN origen a TODOS los demás.
  - No admite pesos negativos.
  - Tiempo:  $O(E \log V)$ .
  - Es la base de los sistemas de navegación modernos.
- 
-

## Capítulo 27

### Bellman-Ford (Camino más Corto con Pesos Negativos)

#### ¿Qué es?

Buscar la ruta más barata entre ciudades, pero donde algunas carreteras te PAGAN por pasar (peso negativo), por ejemplo un descuento o un reembolso.

Dijkstra se rompe con esos reembolsos porque asume que avanzar siempre "cuesta". Bellman-Ford no asume nada: repite el cálculo muchas veces, relajando todas las carreteras una y otra vez, hasta que ninguna ruta puede mejorar más.

Además detecta las trampas: si existe un bucle que te paga infinito (ciclo negativo), Bellman-Ford lo denuncia.

#### Dicho de otra forma

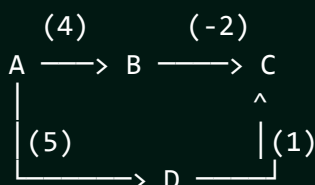
Bellman-Ford calcula la distancia más corta desde un nodo origen a todos los demás en un grafo dirigido y ponderado, admitiendo pesos negativos (algo que Dijkstra no permite).

Su idea central es la "relajación" (relaxation): recorrer todas las aristas  $V - 1$  veces, actualizando  $\text{dist}[v]$  cada vez que  $\text{dist}[u] + \text{peso}(u,v) < \text{dist}[v]$ . Tras  $V - 1$  pasadas, las distancias son óptimas si no hay ciclos negativos.

Una pasada extra ( $V$ -ésima): si alguna arista todavía se puede relajar, existe un ciclo de peso negativo alcanzable desde el origen.

#### Diagrama

Grafo dirigido con peso negativo:



dist inicial:  $A=0$ ,  $B=\text{inf}$ ,  $C=\text{inf}$ ,  $D=\text{inf}$

Relajar aristas  $V-1 = 3$  veces:



$A \rightarrow B(4): B = 4$   
 $B \rightarrow C(-2): C = 2$   
 $A \rightarrow D(5): D = 5$   
 $D \rightarrow C(1): C = \min(2, 6) = 2$

Resultado:  $A=0, B=4, C=2, D=5$

Pasada extra: ninguna arista mejora  $\rightarrow$  NO hay ciclo negativo.

## ⚙️ ¿Cómo funciona?

TABLA DE RELAJACIÓN (dist tras cada pasada), origen = A

Aristas:  $A \rightarrow B(4)$   $B \rightarrow C(-2)$   $A \rightarrow D(5)$   $D \rightarrow C(1)$

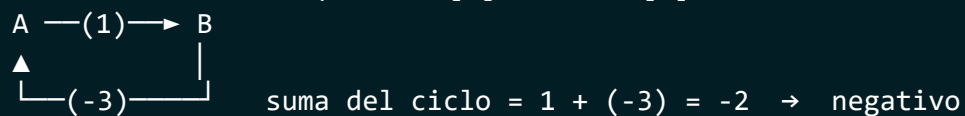
|         | A | B   | C   | D   |                                                                                          |
|---------|---|-----|-----|-----|------------------------------------------------------------------------------------------|
| inicio  | 0 | inf | inf | inf |                                                                                          |
| pasada1 | 0 | 4   | 2   | 5   | ( $A \rightarrow B=4, B \rightarrow C=2, A \rightarrow D=5, D \rightarrow C=\min(2,6)$ ) |
| pasada2 | 0 | 4   | 2   | 5   | (sin cambios $\rightarrow$ ya estable)                                                   |
| pasada3 | 0 | 4   | 2   | 5   |                                                                                          |

Cada celda se actualiza solo si  $\text{dist}[u] + \text{peso} < \text{dist}[v]$ .

Estable antes de  $V-1$  pasadas  $\rightarrow$  early exit posible.

DETECCIÓN de ciclo negativo (pasada extra):

si ALGUNA arista aún cumple  $\text{dist}[u] + w < \text{dist}[v] \rightarrow$  CICLO NEGATIVO



## CÓMO FUNCIONA

1.  $\text{dist}[\text{origen}] = 0$ , el resto = infinito.
2. Repite  $V-1$  veces: por CADA arista ( $u, v, \text{peso}$ ), relaja:  
si  $\text{dist}[u] + \text{peso} < \text{dist}[v]$ , entonces  $\text{dist}[v] = \text{dist}[u] + \text{peso}$ .
3. Pasada  $V$ -ésima: si alguna arista aún relaja  $\rightarrow$  ciclo negativo.

## PSEUDOCÓDIGO

función bellmanFord( $V, \text{aristas}, \text{origen}$ ):

dist = arreglo lleno de **INFINITO**

dist[origen] = 0

**REPETIR**  $V-1$  veces:

**PARA** cada ( $u, v, w$ ) en aristas:

**SI**  $\text{dist}[u] \neq \text{INF}$  Y  $\text{dist}[u] + w < \text{dist}[v]$ :

$\text{dist}[v] = \text{dist}[u] + w$

  // Deteccion de ciclo negativo

**PARA** cada ( $u, v, w$ ) en aristas:

```

19 SI dist[u] != INF Y dist[u] + w < dist[v]:
20 return "CICLO NEGATIVO"
21 return dist
22
23 CÓDIGO C++
24
25
26 #include <iostream>
27 #include <vector>
28 #include <climits>
29 using namespace std;
30
31 struct Edge { int u, v, w; };
32
33 bool bellmanFord(int V, vector<Edge>& edges, int src, vector<long long>&
34 dist) {
35 dist.assign(V, LLONG_MAX);
36 dist[src] = 0;
37
38 // V-1 pasadas de relajación
39 for (int i = 0; i < V - 1; i++) {
40 for (auto& e : edges) {
41 if (dist[e.u] != LLONG_MAX && dist[e.u] + e.w < dist[e.v])
42 dist[e.v] = dist[e.u] + e.w;
43 }
44 }
45 // Pasada extra: si algo relaja, hay ciclo negativo
46 for (auto& e : edges) {
47 if (dist[e.u] != LLONG_MAX && dist[e.u] + e.w < dist[e.v])
48 return false; // ciclo negativo detectado
49 }
50 return true;
51 }

```

COMPLEJIDAD (Big-O)

| Caso   | Tiempo     | Espacio |
|--------|------------|---------|
|        |            |         |
| Tiempo | $O(V * E)$ | $O(V)$  |

\* Más lento que Dijkstra ( $O(E \log V)$ ) pero admite pesos negativos.

📌

Optimización: si en una pasada completa NO se relajó ninguna arista, ya terminaste antes de tiempo (early exit).📌

SPFA: una variante con Queue (Shortest Path Faster Algorithm) que suele ser más rápida en la práctica, aunque su peor caso sigue

```

7 siendo $O(V \cdot E)$.
8
9 Aristas negativas SÍ, ciclos negativos NO: un peso negativo aislado
10 está bien; lo que rompe todo es un CICLO cuya suma sea negativa.

```

### En entrevistas

¿Cuándo elegirlo sobre Dijkstra? Cuando haya pesos negativos o te pidan DETECTAR ciclos negativos. Es la respuesta esperada.

”Cheapest Flights Within K Stops” (LeetCode 787): variante de Bellman-Ford limitando el número de pasadas a  $K+1$ .

Inicializa con cuidado: nunca relajes desde un nodo con distancia infinita (evita overflow). Por eso el chequeo `dist[u] != INF`.

Frase clave: ” $V-1$  relajaciones porque el camino más corto tiene a lo sumo  $V-1$  aristas”. Dila y sumas puntos.

### Resumen rápido

- Camino más corto de una fuente a todos, CON pesos negativos.
- Relaja TODAS las aristas  $V-1$  veces.
- Pasada extra detecta ciclos de peso negativo.
- $O(V \cdot E)$ : más lento que Dijkstra pero más general.
- Usa Dijkstra si todos los pesos son positivos (más rápido).
- Clásico: rutas con reembolsos, arbitraje de divisas.

# Capítulo 28

## 🌐 Floyd-Warshall (Caminos más Cortos entre Todos los Pares)

### 🧐 ¿Qué es?

Una tabla de distancias de un mapa de carreteras, donde cada casilla te dice cuánto cuesta ir de la ciudad X a la ciudad Y.

Floyd-Warshall llena esa tabla probando algo simple para cada par: "¿me conviene ir directo de X a Y, o es más barato pasar por una ciudad intermedia K?". Prueba TODAS las ciudades intermedias posibles, una por una, y se queda con la ruta más barata.

Al terminar, tienes la distancia mínima entre cualquier par de ciudades.

### 💬 Dicho de otra forma

Floyd-Warshall calcula la distancia más corta entre TODOS los pares de nodos de un grafo dirigido y ponderado. Admite pesos negativos (pero no ciclos negativos).

Es programación dinámica pura: para cada nodo intermedio  $k$ , mejora la distancia de todo par  $(i, j)$  preguntando si pasar por  $k$  es más corto:

$$d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$$

donde  $d$  es la matriz de distancias.

Su elegancia es un triple bucle de 3 líneas. Su precio es  $O(V^3)$  en tiempo y  $O(V^2)$  en memoria, así que solo sirve para grafos pequeños o densos (cientos de nodos, no millones).

### 🧠 Diagrama

Matriz de distancias (inf = sin conexión directa):

|   | A     | B   | C     |
|---|-------|-----|-------|
| A | [ 0   | 3   | inf ] |
| B | [ inf | 0   | 1 ]   |
| C | [ 2   | inf | 0 ]   |

Probando intermedio  $k = B$ :

A->C ahora puede pasar por B: A->B(3) + B->C(1) = 4

```
dist[A][C] = min(inf, 4) = 4
```

Probando intermedio k = C:

B→A puede pasar por C: B→C(1) + C→A(2) = 3

```
dist[B][A] = min(inf, 3) = 3
```

Matriz final (todas las distancias mínimas):

|   | A             | B | C |
|---|---------------|---|---|
| A | [ 0   3   4 ] |   |   |
| B | [ 3   0   1 ] |   |   |
| C | [ 2   5   0 ] |   |   |

## ⚙️ ¿Cómo funciona?

EVOLUCIÓN DE LA MATRIZ por cada intermedio k

Inicial (inf = sin arista directa):

|   | A               | B | C |
|---|-----------------|---|---|
| A | [ 0   3   inf ] |   |   |
| B | [ inf   0   1 ] |   |   |
| C | [ 2   inf   0 ] |   |   |

k = A (pasar por A):

|   | A               | B | C |
|---|-----------------|---|---|
| A | [ 0   3   inf ] |   |   |
| B | [ inf   0   1 ] |   |   |
| C | [ 2   5   0 ]   |   |   |

C→B vía A: 2+3=5

k = B (pasar por B):

|   | A               | B | C |
|---|-----------------|---|---|
| A | [ 0   3   4 ]   |   |   |
| B | [ inf   0   1 ] |   |   |
| C | [ 2   5   0 ]   |   |   |

A→C vía B: 3+1=4

k = C (pasar por C):

|   | A             | B | C |
|---|---------------|---|---|
| A | [ 0   3   4 ] |   |   |
| B | [ 3   0   1 ] |   |   |
| C | [ 2   5   0 ] |   |   |

B→A vía C: 1+2=3

Regla en cada celda:  $dist[i][j] = \min(dist[i][j], dist[i][k] + dist[k][j])$   
 El bucle de k SIEMPRE va por fuera (si no, resultado incorrecto).

### 1 CÓMO FUNCIONA

1.  $dist[i][j]$  = peso de la arista i→j (0 si i=j, inf si no hay arista).
2. Por cada nodo intermedio k (bucle externo):  
 por cada par (i, j):  
 $dist[i][j] = \min(dist[i][j], dist[i][k] + dist[k][j])$
3. ¡El orden importa! k DEBE ser el bucle más externo.
4. Si algún  $dist[i][i]$  queda negativo → hay ciclo negativo.

### 9 PSEUDOCÓDIGO

```
función floydWarshall(dist): // dist es matriz V x V
 PARA k desde 0 hasta V-1:
 PARA i desde 0 hasta V-1:
 PARA j desde 0 hasta V-1:
 SI dist[i][k] + dist[k][j] < dist[i][j]:
```

```

17 dist[i][j] = dist[i][k] + dist[k][j]
18 return dist
19
20 CÓDIGO C++
21
22
23 #include <iostream>
24 #include <vector>
25 using namespace std;
26
27 const long long INF = 1e18;
28
29 void floydWarshall(vector<vector<long long>>& dist) {
30 int V = dist.size();
31 for (int k = 0; k < V; k++)
32 for (int i = 0; i < V; i++)
33 for (int j = 0; j < V; j++)
34 if (dist[i][k] < INF && dist[k][j] < INF &&
35 dist[i][k] + dist[k][j] < dist[i][j])
36 dist[i][j] = dist[i][k] + dist[k][j];
37 }
38
39 int main() {
40 int V = 3;
41 vector<vector<long long>> dist(V, vector<long long>(V, INF));
42 for (int i = 0; i < V; i++) dist[i][i] = 0;
43 dist[0][1] = 3; dist[1][2] = 1; dist[2][0] = 2; // aristas
44 floydWarshall(dist);
45 // dist[i][j] ahora tiene la distancia mínima entre cada par
46 return 0;
47 }
48
49 COMPLEJIDAD (Big-O)
50
51
52
53
54
55
56
57

```

| Recurso | Costo    |
|---------|----------|
| Tiempo  | $O(V^3)$ |
| Espacio | $O(V^2)$ |

\* Ideal para  $V$  pequeño  $\leq (400-500)$ . Para  $V$  grande usa Dijkstra por nodo.



### En entrevistas

¿Cuándo usarlo? Cuando pidan distancias entre TODOS los pares y el grafo sea pequeño/denso. Si solo necesitas de UNA fuente, usa Dijkstra o Bellman-Ford.

Cierre transitivo: cambia (min, +) por (OR, AND) y Floyd-Warshall te dice qué nodos son alcanzables desde cuáles. Mismo esqueleto.

El orden de los bucles NO se negocia:  $k$  SIEMPRE va afuera. Si lo pones adentro, el resultado es incorrecto.

Ciclo negativo: revisa la diagonal. Si algún  $\text{dist}[i][i] < 0$ , existe un ciclo de peso negativo pasando por  $i$ .

### Resumen rápido

- Distancias más cortas entre TODOS los pares de nodos.
- Programación dinámica: intermedio  $k$  mejora cada par  $(i, j)$ .
- Triple bucle con  $k$  por fuera;  $O(V^3)$  tiempo,  $O(V^2)$  espacio.
- Admite pesos negativos, detecta ciclos negativos en la diagonal.
- Solo para grafos pequeños o densos.
- Alternativa: Dijkstra por cada nodo si el grafo es disperso.

# Capítulo 29

## Topological Sort (Ordenamiento Topológico)

### ¿Qué es?

Ponerte la ropa en el orden correcto . No puedes ponerte los zapatos antes que los calcetines, ni el saco antes que la camisa.

Cada prenda "depende" de otra. El Ordenamiento Topológico te da una lista válida de en qué orden vestirtte para que nunca rompas una regla: "primero A, luego B".

Solo funciona si NO hay contradicciones (ciclos). Si la regla dijera "ponte los zapatos antes que los calcetines Y los calcetines antes que los zapatos", sería imposible: eso es un ciclo.

### Dicho de otra forma

El Topological Sort es un ordenamiento lineal de los vértices de un grafo dirigido acíclico (DAG) tal que, para toda arista  $u \rightarrow v$ , el nodo  $u$  aparece antes que  $v$  en el orden.

Solo existe en grafos dirigidos y SIN ciclos (DAG). Si hay un ciclo, no hay orden válido. Se usa para modelar dependencias: compilación de módulos, prerequisites de materias, planificación de tareas y resolución de fórmulas en hojas de cálculo.

Hay dos formas de calcularlo: el algoritmo de Kahn (BFS con in-degree) y el enfoque DFS (post-orden invertido).

### Diagrama

Grafo de dependencias (DAG):

```
[Calcetines] --> [Zapatos]
[Camisa] --> [Saco] --> [Corbata]
[Ropa interior] --> [Pantalon] --> [Zapatos]
```

In-degree (aristas entrantes):

```
Calcetines:0 Camisa:0 RopaInterior:0
Pantalon:1 Zapatos:2 Saco:1 Corbata:1
```



Un orden topologico valido:

RopaInterior -> Pantalon -> Calcetines -> Zapatos  
-> Camisa -> Saco -> Corbata

(No es unico: cualquier orden que respete las flechas sirve).

## ⚙️ ¿Cómo funciona?

TRAZA DE KAHN paso a paso (grafo simple):

5 → 0 ← 4            Aristas: 5→0, 5→2, 4→0, 4→1, 2→3, 3→1  
↓                    ↓  
2 → 3 → 1

in-degree inicial: 0:2 1:2 2:1 3:1 4:0 5:0

| Paso | Queue (indeg=0) | Saca | Baja indeg de       | Orden       |
|------|-----------------|------|---------------------|-------------|
| 1    | [5, 4]          | 5    | 0→1, 2→0 (entra)    | 5           |
| 2    | [4, 2]          | 4    | 0→0 (entra), 1→1    | 5 4         |
| 3    | [2, 0]          | 2    | 3→0 (entra)         | 5 4 2       |
| 4    | [0, 3]          | 0    | (0 no tiene salida) | 5 4 2 0     |
| 5    | [3]             | 3    | 1→0 (entra)         | 5 4 2 0 3   |
| 6    | [1]             | 1    | —                   | 5 4 2 0 3 1 |

Procesados 6 = V → es DAG, orden válido: 5 4 2 0 3 1

```

1 ALGORITMO DE KAHN (BFS con grados de entrada)
2 1. Calcula el in-degree (aristas entrantes) de cada nodo.
3 2. Mete a una Queue todos los nodos con in-degree = 0.
4 3. Mientras la Queue no esté vacía:
5 - Saca un nodo u, agrégalo al resultado.
6 - Por cada vecino v de u: resta 1 a su in-degree.
7 Si su in-degree llega a 0, métele a la Queue.
8 4. Si el resultado tiene menos nodos que V -> HAY CICLO (no es DAG).
9
10 PSEUDOCÓDIGO (Kahn)
11
12
13 función topoSort(V, adj):
14 indeg = arreglo de ceros de tamaño V
15 PARA cada u, PARA cada v en adj[u]: indeg[v]++
16 q = Queue con todos los nodos donde indeg == 0
17 orden = []
18 MIENTRAS q NO vacía:
19 u = q.dequeue(); orden.push(u)
20 PARA cada vecino v de u:
21 indeg[v]--

```

```

22 SI indeg[v] == 0: q.enqueue(v)
23 SI orden.size < V: return "CICLO DETECTADO"
24 return orden

```

#### CÓDIGO C++ (Kahn)

---

```

27
28
29 #include <iostream>
30 #include <vector>
31 #include <queue>
32 using namespace std;
33
34 vector<int> topoSort(int V, vector<vector<int>>& adj) {
35 vector<int> indeg(V, 0);
36 for (int u = 0; u < V; u++)
37 for (int v : adj[u]) indeg[v]++;
38
39 queue<int> q;
40 for (int i = 0; i < V; i++)
41 if (indeg[i] == 0) q.push(i);
42
43 vector<int> orden;
44 while (!q.empty()) {
45 int u = q.front(); q.pop();
46 orden.push_back(u);
47 for (int v : adj[u])
48 if (--indeg[v] == 0) q.push(v);
49 }
50 // Si no salieron todos, había un ciclo
51 if ((int)orden.size() != V) return {}; // vacío = no es DAG
52 return orden;
53 }

```

#### CÓDIGO C++ (DFS, post-orden invertido)

---

```

56
57
58 void dfs(int u, vector<vector<int>>& adj,
59 vector<bool>& vis, vector<int>& pila) {
60 vis[u] = true;
61 for (int v : adj[u])
62 if (!vis[v]) dfs(v, adj, vis, pila);
63 pila.push_back(u); // se agrega al TERMINAR
64 }
65 // El orden topológico es 'pila' invertida.
66

```

#### COMPLEJIDAD (Big-O)

---

68  
69

| Método     | Tiempo     | Espacio |
|------------|------------|---------|
| Kahn (BFS) | $O(V + E)$ | $O(V)$  |
| DFS        | $O(V + E)$ | $O(V)$  |

\* V = Vértices, E = Aristas.

### En entrevistas

Detección de ciclos gratis: Kahn te dice si hay ciclo sin esfuerzo extra: si procesas menos de V nodos, el grafo NO es un DAG.

”Course Schedule” (LeetCode 207/210): el problema estrella. ”¿Puedes tomar todas las materias dado un mapa de prerequisites?” = ¿existe orden topológico? Respuesta directa con Kahn.

Solo en DAGs dirigidos: si el grafo es no dirigido o tiene ciclos, el ordenamiento topológico NO existe. Menciónalo siempre.

Kahn vs DFS: Kahn es iterativo (sin riesgo de stack overflow) y detecta ciclos naturalmente. DFS es más corto pero recuerda invertir el post-orden al final.

### Resumen rápido

- Ordena un DAG respetando dependencias: u antes que v si  $u \rightarrow v$ .
- Solo existe en grafos dirigidos SIN ciclos.
- Kahn: BFS con in-degree = 0; detecta ciclos si sobran nodos.
- DFS: apila en post-orden e invierte el resultado.
- Ambos  $O(V + E)$ .
- Usos: prerequisites, build systems, planificación de tareas.

## Capítulo 30

### MST: Kruskal y Prim (Árbol de Expansión Mínima)

#### ¿Qué es?

Conectar todas las casas de un pueblo con cable de internet gastando lo MENOS posible. No importa la ruta exacta; solo que TODAS queden conectadas y el cable total sea el más barato.

Dos estrategias para lograrlo:

- Kruskal: ordena todos los cables del más barato al más caro y ve agregándolos, saltándote cualquiera que forme un círculo inútil.
- Prim: empieza en una casa y va "creciendo" la red, agregando siempre el cable más barato que toque una casa nueva.

Ambas llegan al mismo costo mínimo por caminos distintos.

#### Dicho de otra forma

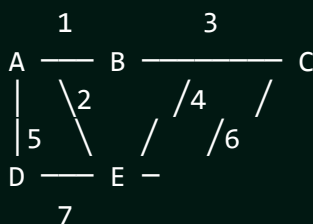
Un Árbol de Expansión Mínima (Minimum Spanning Tree) de un grafo no dirigido y ponderado es un subconjunto de aristas que conecta TODOS los vértices, sin ciclos, con el peso total mínimo. Tiene exactamente  $V - 1$  aristas.

**Kruskal** es "goloso por aristas": ordena las aristas por peso y las agrega si no crean ciclo. Para saber si crean ciclo usa Union-Find (DSU). Ideal para grafos dispersos.

**Prim** es "goloso por nodos": crece un solo árbol desde un nodo, eligiendo con una min-heap la arista más barata hacia un nodo fuera del árbol. Ideal para grafos densos.

#### Diagrama

Grafo no dirigido con pesos:



Aristas ordenadas: A-B(1), A-E(2), B-C(3), B-E(4), A-D(5)...

Kruskal agrega (si no forma ciclo):

A-B(1) OK   A-E(2) OK   B-C(3) OK  
 B-E(4) X (A,B,E ya conectados -> ciclo)  
 A-D(5) OK

MST = {A-B, A-E, B-C, A-D}, peso total = 1+2+3+5 = 11  
 (V=5 nodos -> V-1 = 4 aristas)

## ⚙️ ¿Cómo funciona?

KRUSKAL: cómo el Union-Find evita ciclos (bosque que se fusiona)

Aristas ordenadas: A-B(1) A-E(2) B-C(3) B-E(4) A-D(5)

| Arista | find(u) | find(v) | ¿mismo set? | Acción   | Componentes         |
|--------|---------|---------|-------------|----------|---------------------|
| A-B(1) | A       | B       | NO          | unir OK  | {A,B} {C} {D} {E}   |
| A-E(2) | A       | E       | NO          | unir OK  | {A,B,E} {C} {D}     |
| B-C(3) | A       | C       | NO          | unir OK  | {A,B,E,C} {D}       |
| B-E(4) | A       | A       | SÍ          | saltar X | (formaría ciclo)    |
| A-D(5) | A       | D       | NO          | unir OK  | {A,B,E,C,D} ← 1 set |

Peso total MST = 1+2+3+5 = 11

PRIM: el árbol CRECE desde un nodo (frontera con min-heap)

{A} -1→ añade B      {A,B} -2→ añade E      {A,B,E} -3→ añade C ...  
 siempre toma la arista más barata que sale del árbol hacia afuera.

1 KRUSKAL (goloso por aristas + Union-Find)

2 1. Ordena todas las aristas por peso ascendente.

3 2. Recorre las aristas: si u y v están en componentes distintas  
 4 (find(u) != find(v)), agrégala al MST y hazles union().

5 3. Para cuando el MST tenga V-1 aristas.

6  
 7 PSEUDOCÓDIGO (Kruskal)

8  
 9  
 10 función kruskal(V, aristas):

11    ordenar aristas por peso

12    dsu = UnionFind(V)

13    mst = 0; usadas = 0

14    PARA cada (w, u, v) en aristas:

15       SI dsu.find(u) != dsu.find(v):

16           dsu.union(u, v)

17           mst += w; usadas++

18       SI usadas == V-1: romper

19    return mst

## CÓDIGO C++ (Kruskal)

---

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

struct DSU {
 vector<int> p, r;
 DSU(int n): p(n), r(n, 0) { for (int i=0;i<n;i++) p[i]=i; }
 int find(int x){ return p[x]==x ? x : p[x]=find(p[x]); }
 bool unite(int a,int b){
 a=find(a); b=find(b);
 if(a==b) return false;
 if(r[a]<r[b]) swap(a,b);
 p[b]=a; if(r[a]==r[b]) r[a]++;
 return true;
 }
};

// aristas: {peso, u, v}
long long kruskal(int V, vector<array<int,3>>& edges) {
 sort(edges.begin(), edges.end()); // ordena por peso (primer campo)
 DSU dsu(V);
 long long total = 0; int usadas = 0;
 for (auto& e : edges) {
 if (dsu.unite(e[1], e[2])) { // no forma ciclo
 total += e[0];
 if (++usadas == V - 1) break;
 }
 }
 return total;
}

```

## CÓDIGO C++ (Prim con min-heap)

---

```

#include <queue>
long long prim(int V, vector<vector<pair<int,int>>>& adj) {
 // adj[u] = lista de {vecino, peso}
 vector<bool> inMST(V, false);
 priority_queue<pair<int,int>, vector<pair<int,int>>,
 greater<>> pq; // min-heap {peso, nodo}
 pq.push({0, 0});
 long long total = 0;
 while (!pq.empty()) {
 auto [w, u] = pq.top(); pq.pop();
 }
}

```

```

69 if (inMST[u]) continue;
70 inMST[u] = true; total += w;
71 for (auto [v, pw] : adj[u])
72 if (!inMST[v]) pq.push({pw, v});
73 }
74 return total;
75 }

```

76  
77 COMPLEJIDAD (Big-O)

| Algoritmo   | Tiempo        | Mejor para       |
|-------------|---------------|------------------|
| Kruskal     | $O(E \log E)$ | Grafos dispersos |
| Prim (heap) | $O(E \log V)$ | Grafos densos    |

## En entrevistas

Kruskal reusa Union-Find: si ya dominas DSU, Kruskal es casi gratis. Es la razón por la que se enseñan juntos.

"Min Cost to Connect All Points" (LeetCode 1584): MST clásico sobre distancias de Manhattan. Prim suele ganar por ser el grafo denso.

MST solo en grafos NO dirigidos y conexos. Si el grafo está desconectado no existe MST (existe un "bosque" de expansión mínima).

Corte (cut property): la arista más barata que cruza cualquier partición del grafo SIEMPRE está en algún MST. Es la justificación teórica de por qué el enfoque goloso funciona aquí.

## Resumen rápido

- MST: conecta todos los nodos, sin ciclos, con peso mínimo ( $V-1$  aristas).
- Kruskal: ordena aristas + Union-Find;  $O(E \log E)$ ; grafos dispersos.
- Prim: crece un árbol con min-heap;  $O(E \log V)$ ; grafos densos.
- Ambos son golosos y dan el mismo costo mínimo.
- Solo en grafos no dirigidos y conexos.
- Kruskal es la mejor excusa para practicar Union-Find.

# Capítulo 31

## Detección de Ciclos en Grafos

### ¿Qué es?

Caminar por una ciudad y de repente darte cuenta de que ya pasaste por esa esquina: diste una vuelta completa. Eso es un ciclo.

Detectarlo depende del tipo de calle:

- Calles de doble sentido (grafo no dirigido): hay ciclo si llegas a un lugar ya visitado que NO es de donde acabas de venir.
- Calles de un solo sentido (grafo dirigido): hay ciclo si vuelves a pisar un lugar que todavía está "en tu camino actual", no uno que ya terminaste de explorar.

### Dicho de otra forma

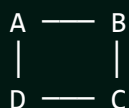
Detectar ciclos es fundamental: un ciclo cambia por completo qué algoritmos aplican (por ejemplo, el ordenamiento topológico solo existe si NO hay ciclos).

En grafos **no dirigidos** usamos DFS guardando el "padre": si encontramos un vecino ya visitado distinto del padre, hay ciclo. Alternativa: Union-Find (si dos extremos de una arista ya están unidos, esa arista cierra un ciclo).

En grafos **dirigidos** usamos DFS con 3 estados (colores): Blanco (sin visitar), Gris (en la pila de recursión actual) y Negro (terminado). Si durante el DFS tocamos un nodo Gris, hay ciclo. El algoritmo de Kahn también sirve: si el topological sort no procesa todos los nodos, hay ciclo.

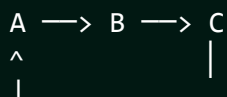
### Diagrama

NO DIRIGIDO (padre):



DFS: A->B->C->D->A  
Al ver A de nuevo (no es padre de D) => CICLO

DIRIGIDO (3 colores):



DFS desde A:  
A(gris) B(gris) C(gris)  
C apunta a A que esta GRIS  
=> CICLO



Estados: Blanco=0 Gris=1 (en recursion) Negro=2 (listo)

## ⚙️ ¿Cómo funciona?

TRAZA 3 COLORES (dirigido) grafo: 0→1→2→0

| Paso | Nodo | Acción                                  | Estados (0,1,2)   |
|------|------|-----------------------------------------|-------------------|
| 1    | 0    | entra → GRIS                            | (G, B, B)         |
| 2    | 1    | entra → GRIS                            | (G, G, B)         |
| 3    | 2    | entra → GRIS                            | (G, G, G)         |
| 4    | 2    | vecino 0 es GRIS →<br>arista de retorno | ¡CICLO detectado! |

Blanco(B)=sin visitar Gris(G)=en la pila actual Negro(N)=terminado  
Ver un GRIS = arista hacia un ancestro = ciclo.  
Ver un NEGRO ≠ ciclo (ya cerró, es otra rama).

NO DIRIGIDO (con padre): 0-1-2 y 2-0  
DFS 0(pad=-1) → 1(pad=0) → 2(pad=1)  
desde 2 veo vecino 0: visitado y 0≠padre(1) → CICLO

```

1 NO DIRIGIDO (DFS con padre)
2 - Recorre con DFS. Al visitar un vecino v de u:
3 * Si v no visitado -> DFS(v, padre=u).
4 * Si v visitado y v != padre -> hay ciclo.
5
6 DIRIGIDO (DFS con 3 colores)
7 - color[u] = GRIS al entrar, NEGRO al salir.
8 - Si un vecino v es GRIS -> hay ciclo (arista hacia atrás).
9
10 PSEUDOCÓDIGO (dirigido, 3 colores)
11
12
13 función tieneCicloDirigido(u):
14 color[u] = GRIS
15 PARA cada vecino v de u:
16 SI color[v] == GRIS: return true // ciclo
17 SI color[v] == BLANCO Y tieneCicloDirigido(v): return true
18 color[u] = NEGRO
19 return false
20
21 CÓDIGO C++ (no dirigido, con padre)
22
23
24 #include <vector>
25 using namespace std;

```

```

26
27 bool dfsUndirected(int u, int parent, vector<vector<int>>& adj,
28 vector<bool>& vis) {
29 vis[u] = true;
30 for (int v : adj[u]) {
31 if (!vis[v]) {
32 if (dfsUndirected(v, u, adj, vis)) return true;
33 } else if (v != parent) {
34 return true; // visitado y no es el padre
35 }
36 }
37 return false;
38 }

```

CÓDIGO C++ (dirigido, 3 colores)

---

```

41
42
43 bool dfsDirected(int u, vector<vector<int>>& adj, vector<int>& color) {
44 color[u] = 1; // GRIS (en recursión)
45 for (int v : adj[u]) {
46 if (color[v] == 1) return true; // ciclo
47 if (color[v] == 0 && dfsDirected(v, adj, color)) return true;
48 }
49 color[u] = 2; // NEGRO (terminado)
50 return false;
51 }

```

COMPLEJIDAD (Big-O)

---

| Método             | Tiempo           | Espacio |
|--------------------|------------------|---------|
| DFS (ambos)        | $O(V + E)$       | $O(V)$  |
| Union-Find (undir) | $O(E \alpha(V))$ | $O(V)$  |

\* $\alpha$  = función inversa de Ackermann, prácticamente constante.



### En entrevistas

Elige el método según el grafo: no dirigido -> padre o Union-Find; dirigido -> 3 colores o Kahn. Confundirlos es un error típico.

"Course Schedule" (LeetCode 207): "¿es posible terminar el plan?" = "¿el grafo de prerequisites NO tiene ciclos?". Ciclo dirigido.

El truco del "padre" solo aplica a NO dirigidos. En dirigidos un nodo NEGRO (ya terminado) no implica ciclo; solo un GRIS lo implica.

Detección de deadlocks: en sistemas operativos, un ciclo en el grafo de "espera de recursos" = interbloqueo. Buen dato para mencionar.

### Resumen rápido

- No dirigido: DFS con padre, o Union-Find (arista que une lo ya unido).
- Dirigido: DFS con 3 colores; un vecino GRIS = ciclo. • Alternativa dirigida: Kahn no procesa todos los nodos = ciclo. • Todo en  $O(V + E)$ . • Vital para topo-sort, detección de deadlocks y validar DAGs.

# Capítulo 32

## Componentes Fuertemente Conexas (SCC · Kosaraju)

### ¿Qué es?

Grupos de amigos en una red social donde "seguir" es de un solo sentido. Una Componente Fuertemente Conexa es un grupo donde TODOS pueden llegar a TODOS siguiendo flechas: desde cualquier persona del grupo puedes alcanzar a cualquier otra y regresar.

Kosaraju encuentra estos grupos con un truco astuto: recorre el grafo, luego INVIERTE todas las flechas y lo recorre de nuevo. Los nodos que siguen "atrapados juntos" incluso con las flechas al revés forman una componente fuertemente conexa.

### Dicho de otra forma

En un grafo dirigido, una Componente Fuertemente Conexa (Strongly Connected Component) es un subconjunto máximo de nodos donde existe un camino dirigido entre cualquier par de ellos en ambos sentidos.

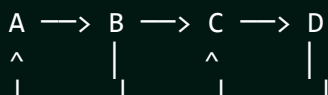
El algoritmo de Kosaraju las halla en dos pasadas de DFS:

1. DFS sobre el grafo original, apilando cada nodo cuando TERMINA (orden de finalización, como en topo-sort).
2. Transponer el grafo (invertir todas las aristas) y hacer DFS sacando nodos de la pila; cada árbol de este segundo DFS es una SCC.

Alternativa de una sola pasada: el algoritmo de Tarjan (usa índices de descubrimiento y **low-link**), más eficiente pero más difícil de recordar bajo presión.

### Diagrama

Grafo dirigido:



A <-> B forman ciclo -> SCC {A, B}  
C <-> D forman ciclo -> SCC {C, D}

Paso 1 (DFS original): apila por orden de finalizacion

Paso 2 (grafo transpuesto): cada DFS-tree = una SCC

Resultado: {A, B} y {C, D}

Grafo condensado (cada SCC = 1 super-nodo) SIEMPRE es un DAG.

## ⚙️ ¿Cómo funciona?

LAS DOS PASADAS de Kosaraju

Grafo:  $A \rightarrow B \rightarrow C \rightarrow D$

PASADA 1 (DFS en original, apilar al TERMINAR):

$\text{dfs}(A) \rightarrow \text{dfs}(B) \rightarrow \text{dfs}(C) \rightarrow \text{dfs}(D)$

termina: D, luego C, luego B, luego A

PILA (tope arriba):

|       |
|-------|
| [ A ] |
| [ B ] |
| [ C ] |
| [ D ] |

TRANSPONER (invertir todas las flechas):  $A \rightarrow B \leftarrow C \rightarrow D$

PASADA 2 (sacar de la pila, DFS en transpuesto):

saca A  $\rightarrow$  dfs alcanza {A,B} ..... SCC #1 = {A, B}

saca B  $\rightarrow$  ya visitado, salta

saca C  $\rightarrow$  dfs alcanza {C,D} ..... SCC #2 = {C, D}

saca D  $\rightarrow$  ya visitado, salta

Resultado: 2 componentes fuertemente conexas.

La transposición "corta" los puentes entre SCC y aísla cada grupo.

```

1 KOSARAJU (dos pasadas de DFS)
2 1. DFS en el grafo original; al terminar cada nodo, apílalo.
3 2. Construye el grafo transpuesto (invierte todas las aristas).
4 3. Saca nodos de la pila; por cada no visitado, un DFS en el
5 transpuesto marca una SCC completa.
6
7 PSEUDOCÓDIGO
8
9
10 función kosaraju(V, adj):
11 // Pasada 1: orden por finalizacion
12 PARA cada u no visitado: dfs1(u) -> apila u al terminar
13 adjT = transponer(adj)
14 // Pasada 2: sacar de la pila sobre el transpuesto
15 MIENTRAS pila no vacia:
16 u = pila.pop()

```

```

17 SI u no visitado_2:
18 dfs2(u, adjT) // marca una SCC entera
19 sccCount++
20 return sccCount
21
22 CÓDIGO C++
23
24
25 #include <vector>
26 #include <stack>
27 using namespace std;
28
29 void dfs1(int u, vector<vector<int>>& adj, vector<bool>& vis,
30 stack<int>& st) {
31 vis[u] = true;
32 for (int v : adj[u]) if (!vis[v]) dfs1(v, adj, vis, st);
33 st.push(u); // apila al TERMINAR
34 }
35
36 void dfs2(int u, vector<vector<int>>& adjT, vector<bool>& vis,
37 vector<int>& comp) {
38 vis[u] = true; comp.push_back(u);
39 for (int v : adjT[u]) if (!vis[v]) dfs2(v, adjT, vis, comp);
40 }
41
42 int kosaraju(int V, vector<vector<int>>& adj) {
43 vector<bool> vis(V, false);
44 stack<int> st;
45 for (int i = 0; i < V; i++) if (!vis[i]) dfs1(i, adj, vis, st);
46
47 // Transponer
48 vector<vector<int>> adjT(V);
49 for (int u = 0; u < V; u++)
50 for (int v : adj[u]) adjT[v].push_back(u);
51
52 fill(vis.begin(), vis.end(), false);
53 int scc = 0;
54 while (!st.empty()) {
55 int u = st.top(); st.pop();
56 if (!vis[u]) {
57 vector<int> comp;
58 dfs2(u, adjT, vis, comp); // una SCC
59 scc++;
60 }
61 }
62 return scc;
63 }
64
65 COMPLEJIDAD (Big-O)

```

|    |                                                                         |            |            |
|----|-------------------------------------------------------------------------|------------|------------|
| 67 |                                                                         |            |            |
| 68 | Algoritmo                                                               | Tiempo     | Espacio    |
| 69 |                                                                         |            |            |
| 70 | Kosaraju                                                                | $O(V + E)$ | $O(V + E)$ |
| 71 | Tarjan                                                                  | $O(V + E)$ | $O(V)$     |
| 72 |                                                                         |            |            |
| 73 | * Tarjan hace una sola pasada; Kosaraju hace dos pero es más intuitivo. |            |            |

### En entrevistas

Grafo condensado = DAG: si colapsas cada SCC en un solo nodo, el grafo resultante nunca tiene ciclos. Sirve para aplicar topo-sort encima. Frase que impresiona.

”Critical Connections” / puentes (LeetCode 1192): variantes de conectividad que se resuelven con low-link estilo Tarjan.

SCC es solo para grafos DIRIGIDOS. En no dirigidos el concepto equivalente es ”componentes conexas” simples (un DFS/BFS o Union-Find).

Recuerda el orden de Kosaraju: primero apilas por finalización en el grafo original, luego DFS en el TRANSPUESTO. Invertir ese orden lo rompe.

### Resumen rápido

- SCC: grupo máximo de nodos mutuamente alcanzables (grafo dirigido).
- Kosaraju: DFS original (apilar al terminar) + DFS en el transpuesto.
- Cada árbol del segundo DFS es una SCC.
- Tarjan: una sola pasada con low-link (más rápido, más difícil).
- Ambos  $O(V + E)$ .
- Condensar las SCC produce siempre un DAG.

# Capítulo 33

## 🧠 Grafo Bipartito (Coloreo con 2 Colores)

### 🧠 ¿Qué es?

Repartir a un grupo de personas en DOS equipos de modo que cada persona quede en equipo CONTRARIO al de todos sus rivales. Si logras hacerlo sin conflictos, el grafo es bipartito.

El truco: pinta a alguien de rojo, obliga a todos sus vecinos a ser azules, a los vecinos de esos a ser rojos, y así. Si en algún momento alguien DEBE ser rojo y azul a la vez (un vecino ya tiene tu mismo color), es imposible: el grafo NO es bipartito.

### 💬 Dicho de otra forma

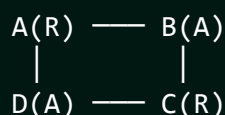
Un grafo es **bipartito** si sus vértices se pueden partir en dos conjuntos disjuntos tal que toda arista conecta un vértice de un conjunto con uno del otro (ninguna arista queda dentro del mismo conjunto).

Se verifica con un coloreo de 2 colores por BFS o DFS: asigna un color al nodo inicial y el color OPUESTO a cada vecino. Si encuentras una arista entre dos nodos del MISMO color, no es bipartito.

Teorema clave: un grafo es bipartito **si y solo si** no contiene ningún ciclo de longitud impar.

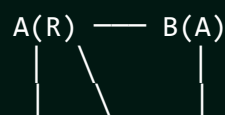
### 🧠 Diagrama

BIPARTITO (ciclo par)



2 equipos: {A,C} y {B,D}  
Toda arista cruza -> SÍ

NO BIPARTITO (ciclo impar)



D(A) - C(R)? A-C exige A(R)-C debe ser azul, pero C ya conecta con B(A)...  
triangulo A-B-C (ciclo 3) -> NO



## ⚙️ ¿Cómo funciona?

TRAZA del coloreo BFS grafo: 0-1, 1-2, 2-3, 3-0 (ciclo par)

color[] = -1 (sin pintar)

| Paso | Saca | Vecinos → color        | Queue | color[0..3] |
|------|------|------------------------|-------|-------------|
| 1    | 0=R  | 1 sin color → AZUL     | [1]   | R A - -     |
| 2    | 1=A  | 0 ya R (ok), 2 → ROJO  | [2]   | R A R -     |
| 3    | 2=R  | 1 ok, 3 → AZUL         | [3]   | R A R A     |
| 4    | 3=A  | 2 ok, 0 ROJO ≠ 3(A) ok | []    | R A R A     |

Ninguna arista une dos del mismo color → BIPARTITO OK

Equipos: {0,2}=Rojo {1,3}=Azul

CASO NO BIPARTITO (triángulo 0-1-2-0):

0=R → 1=A → 2=R ... pero arista 2-0: ambos ROJO → CONFLICTO X

CÓMO FUNCIONA (BFS coloring)

1. color[] = -1 (sin colorear) para todos.
2. Por **cada** componente no coloreada: colorea el inicio con 0, métele a la Queue.
3. Al sacar u, **cada** vecino v: si no tiene color, dale el opuesto (1 - color[u]); si ya tiene el MISMO color que u → NO bipartito.

PSEUDOCÓDIGO

```
función esBipartito(V, adj):
 color = arreglo de -1
 PARA cada nodo s sin color:
 color[s] = 0; q = Queue con s
 MIENTRAS q no vacia:
 u = q.dequeue()
 PARA cada vecino v de u:
 SI color[v] == -1:
 color[v] = 1 - color[u]; q.enqueue(v)
 SINO SI color[v] == color[u]:
 return FALSO
 return VERDADERO
```

CÓDIGO C++

```
#include <vector>
#include <queue>
using namespace std;
```

```

31 bool isBipartite(int V, vector<vector<int>>& adj) {
32 vector<int> color(V, -1);
33 for (int s = 0; s < V; s++) {
34 if (color[s] != -1) continue; // ya visitado
35 color[s] = 0;
36 queue<int> q; q.push(s);
37 while (!q.empty()) {
38 int u = q.front(); q.pop();
39 for (int v : adj[u]) {
40 if (color[v] == -1) {
41 color[v] = 1 - color[u]; // color opuesto
42 q.push(v);
43 } else if (color[v] == color[u]) {
44 return false; // conflicto
45 }
46 }
47 }
48 }
49 return true;
50 }

```

51 COMPLEJIDAD (Big-O)

52

53

54

55

56

57

58

59

60

|         |  |            |       |
|---------|--|------------|-------|
| Recurso |  | Costo      | _____ |
| Tiempo  |  | $O(V + E)$ |       |
| Espacio |  | $O(V)$     |       |

\* Recorre cada nodo y arista una vez; el array color es  $O(V)$ .



### En entrevistas

"Is Graph Bipartite?" (LeetCode 785) y "Possible Bipartition" (886): ambos son 2-coloring puro. Recuerda recorrer TODAS las componentes (el grafo puede estar desconectado).

Aplicación real: asignar tareas a 2 máquinas, detectar conflictos de horarios, o modelar relaciones "esto vs aquello" (matching bipartito).

Grafo desconectado: si olvidas el bucle externo sobre nodos sin color, solo verificas una componente y das un falso positivo.

Ciclo impar = no bipartito: si te preguntan la intuición, esa es la frase. Un triángulo (ciclo de 3) nunca es bipartito.

 **Resumen rápido**

- Bipartito: partir los nodos en 2 grupos; toda arista cruza entre ellos.
- Se verifica coloreando con 2 colores por BFS o DFS.
- Conflicto: una arista une dos nodos del mismo color  $\rightarrow$  no bipartito.
- Equivalente: no tiene ciclos de longitud impar.
- Recorre TODAS las componentes (grafo puede estar desconectado).
- $O(V + E)$ .

# Capítulo 34

## Puntos de Articulación y Puentes (Tarjan)

### ¿Qué es?

Una red de ciudades conectadas por carreteras . Algunas conexiones son tan críticas que, si se rompen, dejan zonas incomunicadas:

- Puente (bridge): una CARRETERA que, si la quitas, parte el mapa en pedazos aislados.
- Punto de articulación (cut vertex): una CIUDAD que, si desaparece (con todas sus carreteras), desconecta el mapa.

Son los "puntos débiles" de una red. Tarjan los encuentra en un solo recorrido DFS midiendo, para cada nodo, qué tan "atrás" puede regresar sin usar la arista por la que llegó.

### Dicho de otra forma

En un grafo no dirigido conexo:

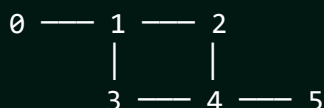
- Un **puente** es una arista cuya eliminación aumenta el número de componentes conexas.
- Un **punto de articulación** es un vértice cuya eliminación (y la de sus aristas) hace lo mismo.

El algoritmo de Tarjan usa DFS con dos marcas por nodo: **disc[u]** (tiempo de descubrimiento) y **low[u]** (el menor **disc** alcanzable desde  $u$  usando aristas del árbol DFS y a lo sumo una arista "de retorno"). Para una arista de árbol  $u \rightarrow v$ :

- Es **puente** si  $\text{low}[v] > \text{disc}[u]$  ( $v$  no puede volver a  $u$  ni a un ancestro sin esa arista).
- $u$  es **articulación** si  $\text{low}[v] \geq \text{disc}[u]$  (con manejo especial de la raíz: lo es si tiene 2+ hijos en el DFS).

### Diagrama

Grafo:



disc = tiempo de descubrimiento, low = mínimo alcanzable

Arista 4-5: al quitarla, el 5 queda aislado -> PUENTE

Arista 0-1: al quitarla, el 0 queda aislado -> PUENTE

Vertice 1: al quitarlo, el 0 se desconecta -> ARTICULACIÓN

Vertice 4: al quitarlo, el 5 se desconecta -> ARTICULACIÓN

El ciclo 1-2-4-3-1 NO tiene puentes (hay ruta alterna).

## ⚙️ ¿Cómo funciona?

disc = orden de descubrimiento, low = mínimo alcanzable

Grafo: 0 - 1 - 2 - 0 (triángulo) y 2 - 3 (cola)

DFS desde 0:

| nodo | disc | low | cómo se calcula low                 |
|------|------|-----|-------------------------------------|
| 0    | 0    | 0   |                                     |
| 1    | 1    | 0   | back-edge 1→0 (disc 0) baja low a 0 |
| 2    | 2    | 0   | back-edge 2→0 (disc 0) baja low a 0 |
| 3    | 3    | 3   | hoja, sin retorno                   |

PUENTE: arista (u,v) es puente si  $low[v] > disc[u]$

arista 2-3:  $low[3]=3 > disc[2]=2 \rightarrow$  ¡PUENTE! (al quitarla, 3 aislado)

arista 0-1:  $low[1]=0 \leq disc[0]=0 \rightarrow$  no (el triángulo tiene ruta alterna)

ARTICULACIÓN: u es corte si algún hijo v cumple  $low[v] \geq disc[u]$

nodo 2:  $low[3]=3 \geq disc[2]=2 \rightarrow$  2 es punto de articulación

```

1 CÓMO FUNCIONA (Tarjan, un solo DFS)
2 1. disc[u] = low[u] = timer++ al visitar u.
3 2. Por cada vecino v:
4 - Si v es el padre: ignora.
5 - Si v ya visitado: low[u] = min(low[u], disc[v]) (arista de retorno)
6 - Si no: DFS(v); low[u] = min(low[u], low[v]);
7 * low[v] > disc[u] -> arista (u,v) es PUENTE
8 * low[v] >= disc[u] -> u es ARTICULACIÓN (raíz: 2+ hijos)
9
10 PSEUDOCÓDIGO (puentes)
11
12
13 función dfs(u, padre):
14 disc[u] = low[u] = timer++
15 PARA cada vecino v de u:
16 SI v == padre: continuar

```

```

17 SI disc[v] != -1: // ya visitado
18 low[u] = min(low[u], disc[v])
19 SINO:
20 dfs(v, u)
21 low[u] = min(low[u], low[v])
22 SI low[v] > disc[u]:
23 registrar (u, v) como PUENTE
24
25 CÓDIGO C++ (puentes y articulaciones)

```

---

```

26
27
28 #include <vector>
29 using namespace std;
30
31 vector<int> disc, low;
32 vector<bool> esArtic;
33 vector<pair<int,int>> puentes;
34 int timer_ = 0;
35
36 void dfs(int u, int parent, vector<vector<int>>& adj) {
37 disc[u] = low[u] = timer_++;
38 int hijos = 0;
39 for (int v : adj[u]) {
40 if (v == parent) continue;
41 if (disc[v] != -1) {
42 low[u] = min(low[u], disc[v]); // arista de retorno
43 } else {
44 hijos++;
45 dfs(v, u, adj);
46 low[u] = min(low[u], low[v]);
47 if (low[v] > disc[u])
48 puentes.push_back({u, v}); // PUENTE
49 if (parent != -1 && low[v] >= disc[u])
50 esArtic[u] = true; // ARTICULACIÓN
51 }
52 }
53 if (parent == -1 && hijos > 1)
54 esArtic[u] = true; // raíz con 2+ hijos
55 }

```

COMPLEJIDAD (Big-O)

|         |            |
|---------|------------|
| Recurso | Costo      |
| Tiempo  | $O(V + E)$ |
| Espacio | $O(V)$     |

\* Un solo DFS calcula puentes Y articulaciones a la vez.

## En entrevistas

"Critical Connections in a Network" (LeetCode 1192): es exactamente encontrar todos los puentes. Respuesta directa con Tarjan.

disc vs low: **disc** es "cuándo te descubrí"; **low** es "lo más arriba que puedes trepar". Si un hijo no puede trepar por encima de ti, la arista o el nodo son críticos. Explicarlo así suma puntos.

Cuidado con la raíz: la regla  $\text{low}[v] \geq \text{disc}[u]$  falla para la raíz del DFS. La raíz es articulación solo si tiene 2 o más hijos en el árbol DFS.

Aristas múltiples: si hay dos aristas paralelas entre  $u$  y  $v$ , ninguna es puente (hay ruta alterna). Trátalo con cuidado si el enunciado las permite.

## Resumen rápido

- Puente: arista que, al quitarla, desconecta el grafo.
- Punto de articulación: vértice que, al quitarlo, lo desconecta.
- Tarjan: un DFS con  $\text{disc}[]$  (descubrimiento) y  $\text{low}[]$  (mínimo alcanzable).
- Puente si  $\text{low}[v] > \text{disc}[u]$ ; articulación si  $\text{low}[v] \geq \text{disc}[u]$ .
- Raíz es articulación solo con 2+ hijos en el DFS.
- Todo en  $O(V + E)$ .

## Capítulo 35

# Flujo Máximo (Ford-Fulkerson / Edmonds-Karp)

### ¿Qué es?

Una red de tuberías desde una fuente (source) hasta un desagüe (sink). Cada tubo aguanta un caudal máximo (capacidad). Pregunta: ¿cuánta agua total puedes empujar de la fuente al desagüe al mismo tiempo?

La idea: mientras exista un camino con espacio libre de la fuente al desagüe, manda tanta agua como el tubo más estrecho de ese camino permita. Repite hasta que ya no quede ningún camino con espacio. La suma de todo lo que enviaste es el flujo máximo.

El truco genial: dejas "aristas de retorno" que permiten DESHACER decisiones previas si conviene reencaminar el agua.

### Dicho de otra forma

El problema de **flujo máximo** busca enviar la mayor cantidad de flujo posible desde un nodo fuente  $s$  a un sumidero  $t$  en un grafo dirigido con capacidades en las aristas, sin exceder ninguna capacidad.

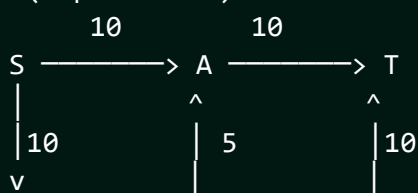
**Ford-Fulkerson** repite: encuentra un "camino de aumento" (augmenting path) de  $s$  a  $t$  con capacidad residual  $> 0$ , empuja por él el mínimo de sus capacidades (el cuello de botella) y actualiza el grafo residual (resta en la arista, suma en su arista inversa). Termina cuando no hay más caminos.

**Edmonds-Karp** es Ford-Fulkerson eligiendo SIEMPRE el camino de aumento más corto vía BFS, lo que garantiza  $O(VE^2)$ .

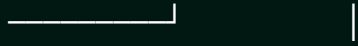
Teorema **max-flow min-cut**: el flujo máximo iguala la capacidad del corte mínimo que separa  $s$  de  $t$ .

### Diagrama

Red (capacidades):





B   
 (B→A cap 5, B→T cap 10 ...)

Camino de aumento S→A→T: cuello de botella  $\min(10,10)=10$  → envía 10

Camino de aumento S→B→T: cuello  $\min(10,10)=10$  → envía 10

No hay más caminos con espacio → Flujo máximo = 20

Grafo residual: cada envío resta capacidad y crea arista inversa  
 (para poder "devolver" flujo si un camino mejor lo requiere).

## ⚙️ ¿Cómo funciona?

GRAFO RESIDUAL evolucionando (flujo/capacidad en cada arista)

Red: S →10→ A →10→ T      S →10→ B →10→ T

Inicio (residual = capacidad):

S=10⇒A    A=10⇒T    S=10⇒B    B=10⇒T      flujo total = 0

Camino de aumento 1: S→A→T, cuello =  $\min(10,10) = 10$

S: 0/10 →A (residual 0, inversa A→S = 10)

A: 0/10 →T

flujo total = 10

Camino de aumento 2: S→B→T, cuello =  $\min(10,10) = 10$

S: 0/10 →B

B: 0/10 →T

flujo total = 20

BFS ya no encuentra camino con residual > 0 → FIN

Flujo máximo = 20

MIN-CUT equivalente: aristas saturadas que separan S de T

{S→A, S→B} capacidad 10+10 = 20 = flujo máximo OK (max-flow=min-cut)

1 EDMONDS-KARP (BFS para el camino de aumento)

2 1. Matriz de capacidad residual  $\text{cap}[u][v]$ .

3 2. BFS de s a t por aristas con  $\text{cap} > 0$ , guardando el padre.

4 3. Si llegas a t: el cuello de botella es el mínimo cap del camino.

5 Resta ese flujo en las aristas y súmalo en las inversas.

6 4. Repite hasta que el BFS ya no alcance t. Suma total = flujo máximo.

7 PSEUDOCÓDIGO

10 función  $\text{maxFlow}(\text{cap}, s, t)$ :

11     flujo = 0

REPETIR:

padre = BFS(cap, s, t) *// camino con capacidad > 0*

SI t no alcanzable: romper

cuello = min de cap a lo largo del camino s..t

PARA cada arista (u,v) del camino:

cap[u][v] -= cuello

cap[v][u] += cuello *// arista de retorno*

flujo += cuello

return flujo

CÓDIGO C++ (Edmonds-Karp)

---

```
#include <vector>
```

```
#include <queue>
```

```
#include <climits>
```

```
#include <cstring>
```

```
using namespace std;
```

```
int V;
```

```
long long bfs(vector<vector<long long>>& cap, int s, int t,
 vector<int>& parent) {
```

```
 fill(parent.begin(), parent.end(), -1);
```

```
 parent[s] = s;
```

```
 queue<pair<int, long long>> q; // {nodo, flujo hasta aquí}
```

```
 q.push({s, LLONG_MAX});
```

```
 while (!q.empty()) {
```

```
 auto [u, f] = q.front(); q.pop();
```

```
 for (int v = 0; v < V; v++) {
```

```
 if (parent[v] == -1 && cap[u][v] > 0) {
```

```
 parent[v] = u;
```

```
 long long nf = min(f, cap[u][v]);
```

```
 if (v == t) return nf; // llegó al sumidero
```

```
 q.push({v, nf});
```

```
 }
```

```
 }
```

```
 }
```

```
 return 0; // no hay camino
```

```
}
```

```
long long maxFlow(vector<vector<long long>>& cap, int s, int t) {
```

```
 long long flow = 0, nf;
```

```
 vector<int> parent(V);
```

```
 while ((nf = bfs(cap, s, t, parent)) > 0) {
```

```
 flow += nf;
```

```
 int v = t;
```

```
 while (v != s) // recorre el camino al revés
```

```
 int u = parent[v];
```

```
 cap[u][v] -= nf; // consume capacidad
```

```
 cap[v][u] += nf; // arista de retorno
```

```
 }
```

```

63 v = u;
64 }
65 }
66 return flow;
67 }
68
69 COMPLEJIDAD (Big-O)
70
71
72 | Algoritmo | Tiempo | Nota |
73 |-----|-----|-----|
74 | Edmonds-Karp | $O(V \cdot E^2)$ | BFS, capacidades enteras |
75 | Dinic | $O(V^2 \cdot E)$ | mucho más rápido en la práctica |
76
77 * Ford-Fulkerson genérico puede no terminar con capacidades irracionales
 .

```

### En entrevistas

Max-flow min-cut: el flujo máximo = capacidad del corte mínimo. Muchos problemas "¿mínimo para desconectar/separar?" se reducen a max-flow. Sabértelo abre la puerta a modelar problemas difíciles.

Matching bipartito máximo se resuelve con max-flow: fuente  $\rightarrow$  lado A  $\rightarrow$  lado B  $\rightarrow$  sumidero, capacidades 1. El flujo máximo es el número de emparejamientos.

Aristas de retorno son OBLIGATORIAS: sin la capacidad inversa, el algoritmo no puede "arrepentirse" y da respuestas menores a la óptima.

Elige BFS (Edmonds-Karp): usar DFS sin control puede degradar el tiempo enormemente. BFS asegura caminos más cortos y el límite  $O(VE^2)$ .

### Resumen rápido

- Flujo máximo: máximo caudal de la fuente  $s$  al sumidero  $t$  sin pasarse.
- Ford-Fulkerson: repite empujar flujo por caminos de aumento.
- Edmonds-Karp: elige el camino más corto con BFS;  $O(V \cdot E^2)$ .
- Grafo residual con aristas de retorno para reencaminar flujo.
- Teorema max-flow min-cut: flujo máx = corte mínimo.
- Modela matching bipartito y problemas de separación mínima.

## Algoritmos de Ordenamiento

# Capítulo 36

## □ Ordenamientos Básicos (Bubble, Insertion, Selection)

### 🧐 ¿Qué es?

Tres formas manuales de ordenar cosas en la vida real:

1. Bubble Sort (Burbujas): Imagina burbujas subiendo en un refresco. Comparas dos elementos juntos; si el de la izquierda es mayor, lo cambias. Los números más grandes "burbujean" hacia el final en cada pasada.
2. Insertion Sort (Cartas): Como cuando ordenas cartas en tu mano. Tomas una carta nueva y la vas comparando con las que ya tienes ordenadas hasta que encuentras su lugar exacto y la "insertas".
3. Selection Sort (Selección): Buscas el objeto más pequeño de todo el grupo y lo pones al principio. Luego buscas el más pequeño de los que quedan y lo pones en segundo lugar, y así...

### 💬 Dicho de otra forma

Son algoritmos de ordenamiento  $O(n^2)$ . Aunque son lentos para grandes cantidades de datos, son fundamentales para entender la lógica de sorting.

- Bubble Sort: Se basa en intercambios adyacentes. - Insertion Sort: Divide el array en una parte ordenada y otra no. Es muy eficiente para arrays que ya están casi ordenados. - Selection Sort: Se basa en encontrar el mínimo en cada iteración.

### 🧠 Diagrama

Bubble Sort (Paso 1):

[64, 34, 25, 12] -> 64 > 34? SÍ -> [34, 64, 25, 12]

[34, 64, 25, 12] -> 64 > 25? SÍ -> [34, 25, 64, 12]

[34, 25, 64, 12] -> 64 > 12? SÍ -> [34, 25, 12, 64]\* (64 ya está al final)

Insertion Sort (Paso 2 - ordenando el 25):

Mano: [34, 64] | Pendiente: [25, 12]

Tomo el 25. ¿25 < 64? SÍ. ¿25 < 34? SÍ.

Mano: [25, 34, 64] | Pendiente: [12]

Selection Sort (Paso 1):

[64, 34, 25, 12] -> Mínimo es 12.  
 Intercambio 64 y 12 -> [12, 34, 25, 64]

1 Bubble:

- 2 - Compara pares (i, i+1).
- 3 - Swapea si están en orden incorrecto.
- 4 - Repite n-1 veces.

5  
6 Insertion:

- 7 - Empieza en el segundo elemento.
- 8 - Compáralo hacia atrás y muévelo hasta que sea mayor que el anterior.

9  
10 Selection:

- 11 - Encuentra el índice del valor mínimo en el rango [i...n].
- 12 - Intercambia `arr[i]` con `arr[min\_idx]`.

13  
14 PSEUDOCÓDIGO

---

15  
16  
17 *// Bubble Sort*

```
18 PARA i desde 0 hasta n-1:
19 PARA j desde 0 hasta n-i-2:
20 SI arr[j] > arr[j+1]:
21 intercambiar(arr[j], arr[j+1])
```

22  
23 *// Insertion Sort*

```
24 PARA i desde 1 hasta n-1:
25 clave = arr[i]
26 j = i - 1
27 MIENTRAS j >= 0 Y arr[j] > clave:
28 arr[j+1] = arr[j]
29 j--
30 arr[j+1] = clave
```

31  
32 CÓDIGO C++

---

```
33
34
35 #include <iostream>
36 #include <vector>
37 using namespace std;
38
39 void bubbleSort(vector<int>& arr) {
40 int n = arr.size();
41 for (int i = 0; i < n-1; i++) {
42 for (int j = 0; j < n-i-1; j++) {
43 if (arr[j] > arr[j+1]) swap(arr[j], arr[j+1]);
44 }
45 }
46 }
47
```

```

48 void insertionSort(vector<int>& arr) {
49 int n = arr.size();
50 for (int i = 1; i < n; i++) {
51 int key = arr[i];
52 int j = i - 1;
53 while (j >= 0 && arr[j] > key) {
54 arr[j + 1] = arr[j];
55 j--;
56 }
57 arr[j + 1] = key;
58 }
59 }
60
61 int main() {
62 vector<int> data = {64, 34, 25, 12, 22};
63 insertionSort(data);
64 for(int x : data) cout << x << " "; // 12 22 25 34 64
65 return 0;
66 }

```

67 COMPLEJIDAD (Big-O)

| 68 Algoritmo | 69 Best     | 70 Average  | 71 Worst    | 72 Stable? |
|--------------|-------------|-------------|-------------|------------|
| 73 Bubble    | 74 $O(n)$   | 75 $O(n^2)$ | 76 $O(n^2)$ | 77 Sí      |
| 78 Insertion | 79 $O(n)$   | 80 $O(n^2)$ | 81 $O(n^2)$ | 82 Sí      |
| 83 Selection | 84 $O(n^2)$ | 85 $O(n^2)$ | 86 $O(n^2)$ | 87 NO      |

88 \* Todos usan  $O(1)$  de espacio extra (In-place).

1  Insertion Sort es el mejor para arrays "casi ordenados" o para pocos elementos ( $n < 20$ ).

2

3 Estabilidad (Stability): Significa que elementos con el mismo valor mantienen su orden relativo original. Bubble e Insertion son estables; Selection NO.

4

5 Casi nunca usarás estos en producción (C++ usa `std::sort` que es  $O(n \log n)$ ), pero te los piden para evaluar tus bases.

6

7 Optimización Bubble: Puedes agregar un flag `swapped`. Si en una pasada completa no hubo cambios, ¡el **array** ya está ordenado!

8

9

10

11

12

13



### Resumen rápido

- Los 3 son  $O(n^2)$  en el peor caso.
  - Bubble: El más sencillo e ineficiente.
  - Insertion: El mejor para casos pequeños o casi listos.
  - Selection: Hace el mínimo de swaps posibles (útil si escribir en memoria es muy caro).
  - In-place: No necesitan memoria extra.
- 
-



# Capítulo 37

## ☒ Merge Sort (Ordenamiento por Mezcla)

### 🧐 ¿Qué es?

Dividir un problema grande con un amigo . Tienes una pila gigante de cartas desordenadas. En lugar de ordenarlas tú solo, le das la mitad a tu amigo.

Cada uno ordena su mitad (y si son muchas, vuelven a dividir con alguien más). Al final, cuando cada quien tiene su pequeña parte ordenada, las fusionan (merge) comparando carta por carta para armar la pila final perfecta.

Es un algoritmo basado en la estrategia "Divide y Vencerás" (Divide and Conquer).

### 💬 Dicho de otra forma

Merge Sort es un algoritmo de ordenamiento recursivo, estable y de comparación. Su funcionamiento se basa en dos fases: 1. Divide: Parte el array recursivamente a la mitad hasta llegar a unidades de tamaño 1. 2. Conquer (Merge): Combina los subarrays ordenados para formar uno más grande, manteniendo el orden.

A diferencia de Quick Sort, Merge Sort GARANTIZA un tiempo de ejecución de  $O(n \log n)$  incluso en el peor caso posible.

### 🧠 Diagrama

Array inicial: [38, 27, 43, 3]

Divide:

```
[38, 27] [43, 3]
[38] [27] [43] [3]
```

Merge (ordenando al combinar):

```
[27, 38] [3, 43]
 \ /
 [3, 27, 38, 43] <-- ¡Ordenado!
```

1. Si el **array** tiene tamaño 1 o 0, ya está ordenado (caso base).
2. Encuentras el punto medio: `mid = size / 2``.
3. Llamas a ``mergeSort`` para la mitad izquierda.
4. Llamas a ``mergeSort`` para la mitad derecha.

## 5. Funcionas (Merge) las dos mitades:

- Comparas los elementos de **cada** mitad uno por uno.
- Copias el menor al **array** original.
- Repites hasta terminar ambas mitades.

## PSEUDOCÓDIGO

---

función mergeSort(arr):

SI arr.tamaño &lt;= 1: return arr

mitad = arr.tamaño / 2

izq = mergeSort(arr[0...mitad])

der = mergeSort(arr[mitad...fin])

return merge(izq, der)

función merge(izq, der):

resultado = []

MIENTRAS izq Y der NO vacíos:

SI izq[0] &lt;= der[0]:

agregar izq[0] a resultado, quitar de izq

SINO:

agregar der[0] a resultado, quitar de der

agregar lo que sobre de izq o der a resultado

return resultado

## CÓDIGO C++

---

#include <iostream>

#include &lt;vector&gt;

using namespace std;

void merge(vector&lt;int&gt;&amp; arr, int l, int m, int r) {

int n1 = m - l + 1;

int n2 = r - m;

vector&lt;int&gt; L(n1), R(n2);

for (int i = 0; i &lt; n1; i++) L[i] = arr[l + i];

for (int j = 0; j &lt; n2; j++) R[j] = arr[m + 1 + j];

int i = 0, j = 0, k = l;

while (i &lt; n1 &amp;&amp; j &lt; n2) {

if (L[i] &lt;= R[j]) arr[k++] = L[i++];

else arr[k++] = R[j++];

}

while (i &lt; n1) arr[k++] = L[i++];

while (j &lt; n2) arr[k++] = R[j++];

}

```

55
56 void mergeSort(vector<int>& arr, int l, int r) {
57 if (l >= r) return;
58 int m = l + (r - l) / 2;
59 mergeSort(arr, l, m);
60 mergeSort(arr, m + 1, r);
61 merge(arr, l, m, r);
62 }
63
64 int main() {
65 vector<int> data = {38, 27, 43, 3, 9, 82, 10};
66 mergeSort(data, 0, data.size() - 1);
67 for(int x : data) cout << x << " "; // 3 9 10 27 38 43 82
68 return 0;
69 }

```

70  
71 COMPLEJIDAD (Big-O)

| Caso    | Tiempo        | Espacio |
|---------|---------------|---------|
| Best    | $O(n \log n)$ | $O(n)$  |
| Average | $O(n \log n)$ | $O(n)$  |
| Worst   | $O(n \log n)$ | $O(n)$  |

79  
80 \* El espacio  $O(n)$  es porque necesita arrays auxiliares para mezclar.

## En entrevistas

"Sort a Linked List": Merge Sort es el algoritmo preferido para ordenar Linked Lists, porque no necesita acceso aleatorio.

Estabilidad: Merge Sort es un Stable Sort. Esto es vital si ordenas objetos por varios criterios (ej: ordenar personas por nombre y luego por edad).

Memoria: Su punto débil es el espacio  $O(n)$ . Si la memoria es muy limitada, podrías preferir Quick Sort o Heap Sort.

Merge Step: La lógica de la función 'merge' se usa en otros problemas como "Merge Two Sorted Lists" o "Merge Intervals".

## Resumen rápido

- Divide y vencerás.
- Siempre tarda  $O(n \log n)$ , sin importar qué tan desordenado esté.
- Es un ordenamiento estable.
- Usa memoria extra  $O(n)$ .
- Ideal para datasets grandes o Linked Lists.
- Base para problemas de fusionar listas o intervalos.

# Capítulo 38

## ⚡ Quick Sort (Ordenamiento Rápido)

### 🤖 ¿Qué es?

Elegir a un jugador de referencia en el equipo (el Pivot).

Una vez que eliges al Pivot (por ejemplo, el último de la fila), separas a todos los demás en dos grupos: 1. Los que son "peores" (número menor) van a la izquierda del Pivot. 2. Los que son "mejores" (número mayor) van a la derecha del Pivot.

Ahora el Pivot quedó en su lugar correcto para siempre. Luego, haces lo mismo con los dos grupos que quedaron a los lados, hasta que todos estén ordenados.

### 💬 Dicho de otra forma

Quick Sort es un algoritmo de ordenamiento por comparación, recursivo e In-place.

Se basa en la operación "Partition": elegir un elemento como Pivot y reorganizar el array de modo que los menores queden a la izquierda y los mayores a la derecha.

Aunque su peor caso es  $O(n^2)$ , en la práctica es mucho más rápido que Merge Sort debido a que tiene una excelente "Cache Locality" y no requiere memoria extra significativa.

### 🧠 Diagrama

Array: [3, 6, 8, 10, 1, 2, 5] (Pivot = 5)

Partition:

- Elementos < 5: [3, 1, 2]
- Pivot: 5
- Elementos > 5: [6, 8, 10]

Estado: [3, 1, 2] [5] [6, 8, 10]  
(El 5 ya no se mueve más).

Se repite el proceso para [3, 1, 2] y para [6, 8, 10].

1. Caso base: Si el rango tiene 0 o 1 elementos, ya está ordenado.
2. Partition:
  - Eliges un Pivot (el primero, el último, el medio o uno aleatorio).

- Reacomodas el **array**: elementos < Pivot a la izquierda, > a la der.
  - Devuelves la posición final del Pivot.
3. Llamas recursivamente a Quick Sort para la mitad izquierda.
  4. Llamas recursivamente a Quick Sort para la mitad derecha.

#### PSEUDOCÓDIGO

---

```
función quickSort(arr, inicio, fin):
 SI inicio < fin:
 p_idx = partition(arr, inicio, fin)
 quickSort(arr, inicio, p_idx - 1)
 quickSort(arr, p_idx + 1, fin)
```

```
función partition(arr, inicio, fin):
 pivot = arr[fin]
 i = inicio - 1
 PARA j desde inicio hasta fin - 1:
 SI arr[j] < pivot:
 i++
 intercambiar(arr[i], arr[j])
 intercambiar(arr[i+1], arr[fin])
 return i + 1
```

#### CÓDIGO C++

---

```
#include <iostream>
#include <vector>
using namespace std;

int partition(vector<int>& arr, int low, int high) {
 int pivot = arr[high]; // Elegimos el último como pivot
 int i = low - 1; // Índice del elemento más pequeño

 for (int j = low; j < high; j++) {
 if (arr[j] < pivot) {
 i++;
 swap(arr[i], arr[j]);
 }
 }
 swap(arr[i + 1], arr[high]);
 return i + 1;
}

void quickSort(vector<int>& arr, int low, int high) {
 if (low < high) {
 int pi = partition(arr, low, high);
 quickSort(arr, low, pi - 1);
 quickSort(arr, pi + 1, high);
 }
}
```

```

54 }
55 }
56
57 int main() {
58 vector<int> data = {10, 7, 8, 9, 1, 5};
59 quickSort(data, 0, data.size() - 1);
60 for(int x : data) cout << x << " "; // 1 5 7 8 9 10
61 return 0;
62 }
63
64 COMPLEJIDAD (Big-O)

```

| Caso    | Tiempo        | Cuándo ocurre                     |
|---------|---------------|-----------------------------------|
| Best    | $O(n \log n)$ | El pivot siempre parte a la mitad |
| Average | $O(n \log n)$ | Pivot aleatorio / balanceado      |
| Worst   | $O(n^2)$      | Array ya ordenado + pivot extremo |
| Espacio | $O(\log n)$   | Stack de recursión                |

 **std::sort:** En C++, `std::sort` usa "Introsort", que empieza con Quick Sort y cambia a Heap Sort si la recursión es muy profunda.

Pivot Selection: Para evitar el peor caso  $O(n^2)$ , menciona que se puede usar un Pivot aleatorio o la técnica "Median of Three".

No es estable: A diferencia de Merge Sort, Quick Sort NO garantiza mantener el orden relativo de elementos iguales.

In-place: Su gran ventaja es que no necesita arrays extras, solo memoria para las llamadas recursivas (el **stack**).

### Resumen rápido

- El más rápido en la práctica.
- Usa la estrategia "Divide y Vencerás".
- In-place (ahorra memoria).
- Peor caso  $O(n^2)$  pero muy raro con buen pivot.
- No es estable.
- Operación clave: 'partition'.

## Capítulo 39

# Heap Sort (Ordenamiento por Montículo)

### ¿Qué es?

Ordenar sacando SIEMPRE el más grande primero . Metes todos los números en un montículo (heap), una estructura donde el mayor siempre está en la cima. Sacas la cima (el máximo), la pones al final, reacomodas el heap, y repites. Poco a poco el arreglo queda ordenado de menor a mayor.

Su gracia: garantiza  $O(n \log n)$  SIEMPRE (nunca se degrada como Quick Sort) y ordena in-place, sin memoria extra.

### Explicado para un niño de 12

Imagina una torre de cajas donde la regla es que cada caja de arriba siempre pesa más que las de abajo que cuelgan de ella. La caja más pesada siempre queda hasta arriba.

Para ordenar: agarras la caja de arriba (la más pesada) y la pones al final de una fila. Como quitaste la cima, subes otra caja a su lugar y reacomodas la torre para que la regla se cumpla otra vez. Vuelves a agarrar la nueva cima (la segunda más pesada) y la pones antes de la anterior. Repites hasta vaciar la torre. Al final, tu fila quedó ordenada de la más ligera a la más pesada, ¡sin necesitar otra mesa aparte!

### Dicho de otra forma

Heap Sort ordena usando un **binary heap** (montículo binario) representado en el propio arreglo. Dos fases:

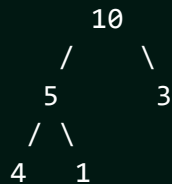
1. **Build Max-Heap:** reorganiza el arreglo para que cumpla la propiedad de max-heap (cada padre  $\geq$  sus hijos). Se hace en  $O(n)$  aplicando **heapify** de abajo hacia arriba.
2. **Sort:** intercambia la raíz (máximo) con el último elemento, reduce el tamaño del heap en 1 y aplica **heapify** a la raíz. Repite hasta vaciar.

En un arreglo, el nodo  $i$  tiene hijos en  $2i + 1$  y  $2i + 2$ , y padre en  $(i - 1)/2$ . No necesita punteros ni memoria extra: por eso es in-place.

 Diagrama

Array: [4, 10, 3, 5, 1]

1) Build Max-Heap:



array: [10, 5, 3, 4, 1]

2) Extraer max (10) -> al final; heapify el resto:

swap(10,1): [1, 5, 3, 4 | 10] -> heapify -> [5, 4, 3, 1 | 10]

swap(5,1): [1, 4, 3 | 5, 10] -> heapify -> [4, 1, 3 | 5, 10]

swap(4,3): [3, 1 | 4, 5, 10] -> heapify -> [3, 1 | ...]

swap(3,1): [1 | 3, 4, 5, 10]

Ordenado: [1, 3, 4, 5, 10]

## CÓMO FUNCIONA

- heapify(i): hunde el elemento i comparándolo con sus hijos ( $2i+1$ ,  $2i+2$ ); si un hijo es mayor, intercambia y sigue bajando.
- Build: aplica heapify desde el último padre ( $n/2 - 1$ ) hacia el 0.
- Sort: swap(0, fin); reduce tamaño; heapify(0). Repite.

## PSEUDOCÓDIGO

función heapify(a, n, i):

    mayor = i; izq =  $2*i+1$ ; der =  $2*i+2$

    SI izq < n Y a[izq] > a[mayor]: mayor = izq

    SI der < n Y a[der] > a[mayor]: mayor = der

    SI mayor != i:

        intercambiar(a[i], a[mayor])

        heapify(a, n, mayor)

función heapSort(a, n):

    PARA i desde  $n/2-1$  hasta 0: heapify(a, n, i)      // build

    PARA i desde n-1 hasta 1:

        intercambiar(a[0], a[i])

        heapify(a, i, 0)      // saca el max

                                 // reacomoda

## CÓDIGO C++

```
#include <vector>
```

```
using namespace std;
```

```
void heapify(vector<int>& a, int n, int i) {
```

```
 int mayor = i, izq = $2*i + 1$, der = $2*i + 2$;
```



```

32 if (izq < n && a[izq] > a[mayor]) mayor = izq;
33 if (der < n && a[der] > a[mayor]) mayor = der;
34 if (mayor != i) { swap(a[i], a[mayor]); heapify(a, n, mayor); }
35 }
36
37 void heapSort(vector<int>& a) {
38 int n = a.size();
39 for (int i = n/2 - 1; i >= 0; i--) heapify(a, n, i); // build max-
40 heap
41 for (int i = n - 1; i >= 1; i--) {
42 swap(a[0], a[i]); // el máximo va al final
43 heapify(a, i, 0); // reacomoda el heap reducido
44 }
45 }
46
47 COMPLEJIDAD (Big-O)
48
49

Caso	Tiempo	Nota
Best	$O(n \log n)$	siempre igual
Average	$O(n \log n)$	sin sorpresas
Worst	$O(n \log n)$	NUNCA se degrada
Espacio	$O(1)$	in-place
Build heap	$O(n)$	(no $O(n \log n)$)

85
86 * No es estable (no conserva el orden de elementos iguales).

```

## En entrevistas

Heap Sort vs Quick Sort: Heap Sort garantiza  $O(n \log n)$  SIEMPRE, sin el peor caso  $O(n^2)$  de Quick Sort. Pero en la práctica Quick Sort suele ser más rápido por mejor cache locality. Introsort usa ambos.

Build-heap es  $O(n)$ , no  $O(n \log n)$ : construir el heap desde abajo cuesta lineal. Es un resultado que sorprende y que preguntan. La suma  $\sum n/2^h \cdot h$  converge a  $O(n)$ .

No es estable: si necesitas conservar el orden relativo de elementos iguales, Heap Sort no sirve; usa Merge Sort.

Top-K con heap: rara vez ordenas todo con Heap Sort en una entrevista; más común es usar un heap (priority\_queue) para los K mayores/menores en  $O(n \log k)$ . Ese es el uso real del heap.



### Resumen rápido

- Heap Sort: build max-heap, luego extrae el máximo repetidamente.
- En arreglo: hijos de  $i$  en  $2i+1$  y  $2i+2$ ; padre en  $(i-1)/2$ .
- $O(n \log n)$  garantizado en TODOS los casos; in-place  $O(1)$ .
- Build-heap cuesta  $O(n)$  (no  $O(n \log n)$ ).
- No es estable.
- En entrevistas, el heap brilla más para Top-K que para ordenar todo.

# Capítulo 40

## □ Counting, Radix y Bucket Sort (Ordenar sin Comparar)

### ¿Qué es?

Todos los ordenamientos anteriores (Quick, Merge, Heap) COMPARAN pares de elementos, y por eso ninguno baja de  $O(n \log n)$ . Estos tres hacen trampa : no comparan, CUENTAN o REPARTEN, y logran  $O(n)$  cuando los datos cumplen ciertas condiciones (rango pequeño, enteros, distribución uniforme).

- Counting Sort: cuenta cuántas veces aparece cada valor.
- Radix Sort: ordena dígito por dígito.
- Bucket Sort: reparte en cubetas y ordena cada una.

### Explicado para un niño de 12

Imagina que quieres ordenar las cartas de un juego numeradas del 1 al 10. En vez de comparar carta con carta, haces 10 montoncitos en la mesa, uno por número. Avientas cada carta a su montoncito y luego recoges los montoncitos en orden: 1, 2, 3... ¡Listo, ya están ordenadas y nunca comparaste dos cartas! Eso es Counting Sort.

Si los números fueran gigantes (como 375), lo haces por partes: primero ordenas por el último dígito, luego por el de en medio, luego por el primero. Como una fila que se reorganiza tres veces. Eso es Radix Sort. Trucos que solo funcionan cuando los números no son "cualquier cosa".

### Dicho de otra forma

Los ordenamientos por comparación tienen una cota inferior de  $\Omega(n \log n)$ . Estos algoritmos la rompen porque **no comparan** elementos entre sí; usan la estructura de los datos.

**Counting Sort:** cuenta la frecuencia de cada valor en un rango  $[0, k]$  y reconstruye el arreglo ordenado.  $O(n + k)$ . Estable. Solo sirve si  $k$  (rango) no es mucho mayor que  $n$ .

**Radix Sort:** aplica Counting Sort dígito por dígito, del menos al más significativo (LSD).  $O(d \cdot (n + b))$  con  $d$  dígitos y base  $b$ . Ideal para enteros o strings de longitud fija.

**Bucket Sort:** reparte los elementos en cubetas según su valor, ordena cada

cubeta (con otro sort) y concatena.  $O(n)$  promedio si la distribución es uniforme;  $O(n^2)$  en el peor caso.

## ☺ Diagrama

COUNTING SORT de [2, 5, 3, 0, 2, 3, 0, 3] (rango 0..5)

Conteo de cada valor:

|        |   |   |   |   |   |   |
|--------|---|---|---|---|---|---|
| valor: | 0 | 1 | 2 | 3 | 4 | 5 |
| veces: | 2 | 0 | 2 | 3 | 0 | 1 |

Reconstruir en orden leyendo el conteo:

0,0, 2,2, 3,3,3, 5 -> [0, 0, 2, 2, 3, 3, 3, 5]

RADIX SORT de [170, 45, 75, 90, 802, 24, 2, 66]

por unidades: [170,90,802,2,24,45,75,66]

por decenas : [802,2,24,45,66,170,75,90]

por centenas: [2,24,45,66,75,90,170,802] <- ordenado

COUNTING SORT (estable)

1. Cuenta cuántas veces aparece **cada** valor (arreglo count de tamaño k+1).
2. Acumula (prefix sum) para saber la posición final de **cada** valor.
3. Recorre el original de DERECHA a IZQUIERDA colocando **cada** elemento (así se mantiene estable).

PSEUDOCÓDIGO (counting sort)

```
función countingSort(a, k): // valores en [0, k]
 count = arreglo de ceros de tamaño k+1
 PARA cada x en a: count[x]++
 PARA i desde 1 hasta k: count[i] += count[i-1] // prefijos
 salida = arreglo del tamaño de a
 PARA i desde n-1 hasta 0: // de derecha a izquierda = estable
 salida[--count[a[i]]] = a[i]
 return salida
```

CÓDIGO C++ (Counting Sort)

```
#include <vector>
using namespace std;

vector<int> countingSort(vector<int>& a, int k) {
 vector<int> count(k + 1, 0), out(a.size());
 for (int x : a) count[x]++; // frecuencias
```

```

28 for (int i = 1; i <= k; i++) count[i] += count[i-1]; // prefijos
29 for (int i = a.size() - 1; i >= 0; i--) // estable
30 out[--count[a[i]]] = a[i];
31 return out;
32 }
33
34 CÓDIGO C++ (Radix Sort, LSD para enteros >= 0)
35
36
37 #include <algorithm>
38 void radixSort(vector<int>& a) {
39 if (a.empty()) return;
40 int maxv = *max_element(a.begin(), a.end());
41 for (int exp = 1; maxv / exp > 0; exp *= 10) { // dígito por dígito
42 vector<int> out(a.size()), count(10, 0);
43 for (int x : a) count[(x / exp) % 10]++;
44 for (int i = 1; i < 10; i++) count[i] += count[i-1];
45 for (int i = a.size() - 1; i >= 0; i--) {
46 int d = (a[i] / exp) % 10;
47 out[--count[d]] = a[i];
48 }
49 a = out;
50 }
51 }
52
53 COMPLEJIDAD (Big-O)
54
55
56

Algoritmo	Tiempo	Estable	Cuándo usar
Counting	$O(n + k)$	Sí	rango k pequeño
Radix (LSD)	$O(d \cdot (n + b))$	Sí	enteros/strings
Bucket	$O(n)$ promedio	Sí*	datos uniformes

86
87 * k = rango, d = número de dígitos, b = base.

```

## En entrevistas

Rompen la cota  $O(n \log n)$  porque NO comparan. Si un entrevistador pregunta "¿se puede ordenar en menos de  $O(n \log n)$ ?", la respuesta es "sí, si los datos lo permiten (rango acotado, enteros)".

"Sort Colors" (LC 75): ordenar un arreglo de 0,1,2 es Counting Sort trivial (o Dutch flag en una pasada). "Maximum Gap" (LC 164) usa Bucket Sort para lograr  $O(n)$ .

Counting Sort explota si k es enorme: ordenar valores hasta  $10^9$  con counting necesitaría un arreglo de mil millones. Solo sirve con rango chico.

Estabilidad importa en Radix: cada pasada por dígito DEBE ser estable (por eso usa Counting Sort). Si no, el orden de los dígitos previos se pierde y el resultado es incorrecto.



### Resumen rápido

- No comparan: cuentan o reparten, rompen la cota  $O(n \log n)$ .
- Counting Sort: cuenta frecuencias;  $O(n + k)$ ; estable; rango  $k$  pequeño.
- Radix Sort: Counting dígito por dígito (LSD); enteros/strings.
- Bucket Sort: reparte en cubetas + ordena cada una;  $O(n)$  si uniforme.
- Radix depende de que cada pasada sea estable.
- Úsalos solo cuando los datos cumplan las condiciones (rango, tipo).

## Algoritmos de Búsqueda

# Capítulo 41

## 👁 Linear Search (Búsqueda Lineal)

### 🤖 ¿Qué es?

Buscar tus llaves en casa revisando cuarto por cuarto, cajón por cajón, sin ningún orden especial.

Empiezas en un lugar y revisas todo hasta que las encuentras o hasta que verificas que no están en ningún lado. Es la forma más básica y natural de buscar algo.

### 💬 Dicho de otra forma

Linear Search es un algoritmo de búsqueda que recorre un conjunto de datos secuencialmente (uno por uno) de principio a fin.

No requiere que los datos estén ordenados. Su simplicidad lo hace ideal para datasets pequeños o cuando solo vas a buscar algo una vez y no vale la pena ordenar todo el conjunto primero.

### 🧠 Diagrama

Buscar el número 7 en el array:  
[ 3, 1, 9, 7, 5 ]

Paso 1: ¿3 == 7? No.

Paso 2: ¿1 == 7? No.

Paso 3: ¿9 == 7? No.

Paso 4: ¿7 == 7? ¡SÍ! 🎉

Encontrado en 4 pasos.

1. Empiezas en el primer elemento (índice 0).
2. Comparas el elemento actual con el valor que buscas.
3. Si son iguales, devuelves el índice o `true``.
4. Si no son iguales, pasas al siguiente elemento.
5. Repites hasta encontrarlo o llegar al final del `array`.
6. Si llegas al final y no lo encontraste, devuelves `-1` o `false``.

### PSEUDOCÓDIGO

```
función busquedaLineal(array, objetivo):
```



```

12 PARA cada i desde 0 hasta tamaño(array) - 1:
13 SI array[i] == objetivo:
14 return i
15 return -1

```

17 CÓDIGO C++

```

18
19
20 #include <iostream>
21 #include <vector>
22 using namespace std;
23
24 int linearSearch(const vector<int>& arr, int target) {
25 for (int i = 0; i < arr.size(); i++) {
26 if (arr[i] == target) {
27 return i; // Encontrado, devolvemos el índice
28 }
29 }
30 return -1; // No encontrado
31 }
32
33 int main() {
34 vector<int> data = {10, 50, 30, 70, 80, 60, 20, 90, 40};
35 int target = 20;
36
37 int result = linearSearch(data, target);
38
39 if (result != -1)
40 cout << "Elemento encontrado en el índice: " << result << endl;
41 else
42 cout << "Elemento no encontrado." << endl;
43
44 return 0;
45 }

```

46 COMPLEJIDAD (Big-O)

| Caso    | Tiempo | Razón                                 |
|---------|--------|---------------------------------------|
| Best    | $O(1)$ | El elemento es el primero             |
| Average | $O(n)$ | Tienes que revisar la mitad ( $n/2$ ) |
| Worst   | $O(n)$ | Está al final o no existe             |
| Espacio | $O(1)$ | No usa memoria extra                  |

### En entrevistas

Cuándo usar: - Datasets pequeños ( $n < 50$ ). - Datos desordenados. - Búsquedas únicas (one-off). - Estructuras sin acceso aleatorio (como Linked Lists).

Strings: Linear Search es la base para encontrar un caracter en un string o una subcadena (algoritmo naive).

Eficiencia: Si vas a hacer MUCHAS búsquedas sobre el mismo conjunto de datos, ¡no uses Linear Search! Mejor ordena los datos y usa Binary Search, o guárdalos en un Hash Map.

Sentinel Search: Una pequeña optimización que elimina una de las dos comparaciones del loop (la que revisa si llegaste al final).

### Resumen rápido

|                                      |                              |                              |
|--------------------------------------|------------------------------|------------------------------|
| • Busca uno por uno.                 | • Muy simple de implementar. | •                            |
| No requiere orden previo.            | • Tiempo lineal $O(n)$ .     | • Memoria constante $O(1)$ . |
| • Ineficiente para datasets grandes. |                              |                              |

---

---

# Capítulo 42

## Binary Search (Búsqueda Binaria)

### ¿Qué es?

El juego de adivinar el número entre 1 y 100.

Dices "50" y te dicen: "¡Más alto!". Ahora sabes que el número NO está entre 1 y 50. ¡Acabas de descartar la MITAD de las posibilidades en un solo paso!

Luego dices "75", te dicen: "¡Más bajo!". Ahora sabes que está entre 51 y 74. Vuelves a descartar la mitad.

En solo 7 intentos puedes adivinar cualquier número entre 128 opciones. Es increíblemente rápido porque el problema se encoge exponencialmente en cada paso.

### Dicho de otra forma

Binary Search es un algoritmo de búsqueda eficiente que funciona sobre un conjunto de datos ORDENADO.

En lugar de revisar uno por uno, siempre revisa el elemento del medio. Si el objetivo es menor al medio, busca en la mitad izquierda. Si es mayor, busca en la derecha.

Este enfoque de "Divide y Vencerás" reduce la complejidad de tiempo a  $O(\log n)$ , lo cual es ideal para datasets masivos.

### Diagrama

Buscar 23 en array ordenado:

[1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25]

idx: 0 1 2 3 4 5 6 7 8 9 10 11 12

Paso 1: izq=0, der=12, mid=6.

arr[6] es 13. ¿13 < 23? Sí.

Nuevo rango: [7...12] (la derecha)

Paso 2: izq=7, der=12, mid=9.

arr[9] es 19. ¿19 < 23? Sí.

Nuevo rango: [10...12]

Paso 3: izq=10, der=12, mid=11.  
arr[11] es 23. ¿23 == 23? ¡SÍ! 🎉

Solo 3 pasos para 13 elementos.

Requisito: El **array** DEBE estar ordenado de menor a mayor.

Pasos:

1. Define dos punteros: `izq` al inicio y `der` al final.
2. Mientras `izq <= der`:
  - Calcula el punto medio: `mid = izq + (der - izq) / 2`.`
  - Si `arr[mid] == objetivo``, ¡terminaste! Devuelve el índice.
  - Si `arr[mid] < objetivo``, mueve `izq = mid + 1`.`
  - Si `arr[mid] > objetivo``, mueve `der = mid - 1`.`
3. Si sales del loop sin encontrarlo, devuelve -1.

PSEUDOCÓDIGO

---

```
función busquedaBinaria(arr, x):
 izq = 0
 der = arr.tamaño - 1
 MIENTRAS izq <= der:
 mid = izq + (der - izq) / 2
 SI arr[mid] == x: return mid
 SI arr[mid] < x: izq = mid + 1
 SINO: der = mid - 1
 return -1
```

CÓDIGO C++

---

```
#include <iostream>
#include <vector>
using namespace std;

int binarySearch(const vector<int>& arr, int target) {
 int left = 0;
 int right = arr.size() - 1;

 while (left <= right) {
 // Usamos esta fórmula para evitar overflow de enteros
 int mid = left + (right - left) / 2;

 if (arr[mid] == target) return mid;

 if (arr[mid] < target)
 left = mid + 1; // Buscar en la derecha
 else
```

```

45 right = mid - 1; // Buscar en la izquierda
46 }
47 return -1;
48 }
49
50 int main() {
51 vector<int> sorted_data = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19};
52 int target = 15;
53
54 int res = binarySearch(sorted_data, target);
55 if(res != -1) cout << "Encontrado en idx: " << res << endl;
56
57 return 0;
58 }
59
60 COMPLEJIDAD (Big-O)

```

| Caso    | Tiempo      | Razón                             |
|---------|-------------|-----------------------------------|
| Best    | $O(1)$      | El elemento está justo al medio   |
| Average | $O(\log n)$ | Se divide el problema a la mitad  |
| Worst   | $O(\log n)$ | No existe o queda 1 solo elemento |
| Espacio | $O(1)$      | Versión iterativa                 |

## En entrevistas

Off-by-one errors: El bug más común es usar 'left < right' en lugar de 'left <= right'. ¡Ten cuidado!

Overflow Bug: NUNCA uses 'mid = (left + right) / 2'. Si left y right son muy grandes, su suma puede superar el límite de un 'int'. Usa siempre: 'left + (right - left) / 2'.

Rotated Sorted Array: Un problema clásico de Google es buscar en un array ordenado que fue "rotado" (ej: [4,5,6,1,2,3]). Sigue siendo Binary Search pero con lógica extra.

Binary Search on Answer: No solo sirve para buscar en arrays. Si te preguntan "¿cuál es el mínimo valor de X que cumple esta condición?", puedes hacer Binary Search sobre el RANGO de posibles valores de X.

## Resumen rápido

- Solo funciona con datos ordenados.
- Divide el espacio de búsqueda a la mitad en cada paso.
- Tiempo logarítmico  $O(\log n)$ : Súper veloz.
- Versión iterativa usa  $O(1)$  memoria.
- Previene overflow usando la fórmula correcta de 'mid'.
- Es una de las herramientas más poderosas para optimizar búsquedas.



## Patrones de Entrevista

# Capítulo 43

## Two Pointers (Dos Punteros)

### ¿Qué es?

Dos personas en extremos opuestos de un pasillo ... caminando hacia el centro para encontrarse. O un corredor lento y uno rápido en la misma pista ... .

En lugar de usar un solo índice para recorrer un array (un loop simple), usas DOS. Estos punteros se mueven de forma coordinada para resolver problemas de búsqueda o comparación mucho más rápido que la fuerza bruta.

Es uno de los patrones más útiles para entrevistas técnicas porque suele reducir soluciones de  $O(n^2)$  a  $O(n)$ .

### Dicho de otra forma

El patrón Two Pointers utiliza dos variables de índice para recorrer una estructura de datos lineal (array, string, linked list).

Existen dos variantes principales: 1. Opposite Ends: Los punteros empiezan en los extremos y se mueven hacia el centro. (Ej: Two Sum en array ordenado, Palíndromos). 2. Fast & Slow: Un puntero avanza más rápido que el otro. (Ej: Detectar ciclos en Linked List, encontrar el medio).

### Diagrama

Problema: Encontrar par que sume 10 en array ordenado.

[ 1, 3, 4, 6, 8, 9 ]

L                      R    -> 1 + 9 = 10. ¡ENCONTRADO! 🎉

Si la suma fuera menor a 10, moveríamos L a la derecha.

Si la suma fuera mayor a 10, moveríamos R a la izquierda.

### ¿Cómo funciona?

Opposite Ends: 1. Inicializas 'left = 0' y 'right = n - 1'. 2. Mientras 'left < right': - Evalúas la condición con 'arr[left]' y 'arr[right]'. - Si se cumple la meta, terminas. - Si no, decides qué puntero mover según la lógica del problema.

Fast & Slow (Floyd's Algorithm): 1. Inicializas 'slow = start' y 'fast = start'. 2. En cada paso: 'slow' avanza 1, 'fast' avanza 2. 3. Si 'fast' alcanza el final, no hay ciclo. 4. Si 'slow == fast', ¡hay un ciclo!



## PSEUDOCÓDIGO

```
// Two Sum en Array Ordenado
función tieneSuma(arr, meta):
 izq = 0
 der = arr.tamaño - 1
 MIENTRAS izq < der:
 sumaActual = arr[izq] + arr[der]
 SI sumaActual == meta: return VERDADERO
 SI sumaActual < meta: izq++
 SINO: der--
 return FALSO
```

## CÓDIGO C++

```
1 #include <iostream>
2 #include <vector>
3 #include <string>
4 #include <algorithm>
5 using namespace std;
6
7 // Verificar si un string es palíndromo -> O(n) tiempo, O(1) espacio
8 bool isPalindrome(string s) {
9 int left = 0;
10 int right = s.length() - 1;
11
12 while (left < right) {
13 if (s[left] != s[right]) return false;
14 left++;
15 right--;
16 }
17 return true;
18 }
19
20 int main() {
21 string word = "reconocer";
22 cout << "¿Es palíndromo? " << (isPalindrome(word) ? "SÍ" : "NO") <<
23 endl;
24
25 return 0;
26 }
```

### Complejidad Big-O

| Caso         | Tiempo   | Espacio |
|--------------|----------|---------|
| Two Pointers | $O(n)$   | $O(1)$  |
| Fuerza Bruta | $O(n^2)$ | $O(1)$  |

\* La magia del patrón es que evitas el segundo loop anidado.

### 🔗 En entrevistas

### 📋 Resumen rápido

- Dos índices moviéndose coordinados.
- Reduce  $O(n^2)$  a  $O(n)$ .
- Memoria constante  $O(1)$  casi siempre.
- Variantes: Opuestos (extremos  $\rightarrow$  centro) y Fast/Slow.
- Requisito común: Array ordenado.
- Indispensable para optimizar búsquedas de pares o ciclos.

---

---

# Capítulo 44

## □ Sliding Window (Ventana Deslizante)

### 🧐 ¿Qué es?

Una ventana de tren donde ves paisajes mientras el tren avanza. La ventana tiene un tamaño fijo (por ejemplo, 3 metros de ancho).

Al avanzar, entra nuevo paisaje por un lado y sale el paisaje viejo por el otro. No tienes que redibujar todo el paisaje cada vez, ¡solo actualizas los bordes!

En programación, este patrón sirve para procesar sub-arreglos o sub-cadenas sin tener que recalcular todo desde cero en cada paso, lo cual ahorra muchísimo tiempo.

### 💬 Dicho de otra forma

El patrón Sliding Window convierte un problema de loops anidados  $O(n*k)$  en un problema de un solo loop  $O(n)$ .

Se usa cuando te piden algo sobre elementos CONTIGUOS de un array o string. Existen dos tipos: 1. Fixed Size: La ventana siempre mide K elementos. 2. Variable Size: La ventana crece o se encoge según una condición (usualmente usando dos punteros).

### 🧠 Diagrama

Array: [ 1, 3, 2, 6, -1, 4 ]

Meta: Suma de cada ventana de tamaño 3.

Paso 1: [ 1, 3, 2 ] , 6, -1, 4 -> Suma = 6

Paso 2: 1, [ 3, 2, 6 ] , -1, 4 -> Suma = 11 (6 - 1 + 6)

Paso 3: 1, 3, [ 2, 6, -1 ] , 4 -> Suma = 7 (11 - 3 + (-1))

¡Solo restas el que sale y sumas el que entra! 📌

### ⚙️ ¿Cómo funciona?

Fixed Size Window: 1. Calculas el resultado de la primera ventana (índices 0 a K-1). 2. Empiezas un loop desde el índice K hasta el final. 3. En cada paso: - Sumas el nuevo elemento ('arr[i]'). - Restas el elemento que sale ('arr[i - K]'). -

Actualizas el resultado máximo/mínimo.

Variable Size Window: 1. Usas dos punteros: 'start' y 'end' (ambos en 0). 2. Mueves 'end' para expandir la ventana y cumplir una condición. 3. Si la condición se rompe, mueves 'start' para encoger la ventana.

## PSEUDOCÓDIGO

```
función sumaMaxima(arr, k):
 sumaActual = suma de los primeros k elementos
 maximaSuma = sumaActual
 PARA i desde k hasta arr.tamaño - 1:
 sumaActual = sumaActual + arr[i] - arr[i - k]
 maximaSuma = max(maximaSuma, sumaActual)
 return maximaSuma
```

## CÓDIGO C++

```
1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4 using namespace std;
5
6 // Máxima suma de un subarray de tamaño K -> O(n)
7 int maxSubarraySum(vector<int>& arr, int k) {
8 if (arr.size() < k) return -1;
9
10 int window_sum = 0;
11 for (int i = 0; i < k; i++) window_sum += arr[i];
12
13 int max_sum = window_sum;
14
15 for (int i = k; i < arr.size(); i++) {
16 // Deslizar la ventana: sumar el nuevo, restar el viejo
17 window_sum += arr[i] - arr[i - k];
18 max_sum = max(max_sum, window_sum);
19 }
20
21 return max_sum;
22 }
23
24 int main() {
25 vector<int> data = {2, 1, 5, 1, 3, 2};
26 int k = 3;
27 cout << "Máxima suma (k=3): " << maxSubarraySum(data, k) << endl; //
 9 (5+1+3)
```

```

28 return 0;
29 }

```

## Complejidad Big-O

| Caso           | Tiempo     | Espacio |
|----------------|------------|---------|
| Sliding Window | $O(n)$     | $O(1)$  |
| Fuerza Bruta   | $O(n * k)$ | $O(1)$  |

\* La diferencia es abismal cuando 'n' y 'k' son grandes.

## En entrevistas

## Resumen rápido

- Elementos contiguos.
- Evita recalcular: Actualiza los bordes de la ventana.
- Reduce  $O(n*k)$  a  $O(n)$ .
- Dos tipos: Fija y Variable.
- Ideal para sumas de rango, promedios móviles y búsqueda de sub-strings.

# Capítulo 45

## ✿ Introducción a Dynamic Programming (DP)

### 🤖 ¿Qué es?

Un estudiante inteligente que guarda sus apuntes .

Imagina que en un examen te preguntan 50 veces cuánto es  $123 \times 456$ . En lugar de hacer la multiplicación cada vez (y perder tiempo), la haces la primera vez, anotas el resultado en tu cuaderno, y las siguientes 49 veces ¡solo lo consultas!

Dynamic Programming es simplemente eso: guardar resultados de sub-problemas ya resueltos para no repetir trabajo innecesariamente. Es el arte de intercambiar un poco de memoria por mucho tiempo.

### 💬 Dicho de otra forma

Dynamic Programming (DP) es una técnica de optimización para resolver problemas complejos dividiéndolos en sub-problemas más pequeños.

Para usar DP, el problema debe cumplir dos condiciones: 1. Overlapping Sub-problems: El mismo sub-problema se calcula varias veces (como en Fibonacci). 2. Optimal Substructure: La solución óptima del problema global se puede construir a partir de las soluciones óptimas de sus partes.

### 🧠 Diagrama

Sin DP (Calculas lo mismo muchas veces):

```
 fib(5)
 / \
 fib(4) fib(3)
 / \ / \
fib(3) fib(2) fib(2) fib(1)
 / \
fib(2) fib(1)
```

¡Nota cómo fib(3) y fib(2) se repiten! ☹️

Con DP (Memoization):

Calculas fib(3) una vez, lo guardas, y cuando fib(4) lo necesite, simplemente lo lee de la memoria. 😊

## ⚙️ ¿Cómo funciona?

Existen dos formas de implementar DP:

1. Top-Down (Memoization): - Empiezas con el problema grande. - Lo divides recursivamente. - Guardas el resultado en un Hash Map o Array antes de regresarlo.
2. Bottom-Up (Tabulation): - Empiezas con los casos más pequeños (base cases). - Llenas una tabla (array) paso a paso hasta llegar al resultado. - Generalmente se hace con un loop iterativo.

## PSEUDOCÓDIGO

// Fibonacci Sin DP:  $O(2^n)$  - Pésimo

función fib(n):

    SI  $n \leq 1$ : return n

    return fib(n-1) + fib(n-2)

// Fibonacci Con DP (Memo):  $O(n)$  - Excelente

función fib(n, memo):

    SI n EN memo: return memo[n]

    SI  $n \leq 1$ : return n

    memo[n] = fib(n-1, memo) + fib(n-2, memo)

    return memo[n]

## CÓDIGO C++

```

1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 // Fibonacci sin DP (muy lento para n > 40)
6 long long fibNaive(int n) {
7 if (n <= 1) return n;
8 return fibNaive(n - 1) + fibNaive(n - 2);
9 }
10
11 // Fibonacci con DP (Tabulation - Bottom Up)
12 long long fibDP(int n) {
13 if (n <= 1) return n;
14 vector<long long> dp(n + 1);
15 dp[0] = 0;
16 dp[1] = 1;
17 for (int i = 2; i <= n; i++) {
18 dp[i] = dp[i - 1] + dp[i - 2];
19 }

```

```

20 return dp[n];
21 }
22
23 int main() {
24 int n = 50;
25 cout << "Fibonacci de " << n << " es: " << fibDP(n) << endl;
26 return 0;
27 }

```

### Complejidad Big-O

| Enfoque       | Tiempo   | Espacio        |
|---------------|----------|----------------|
| Fuerza Bruta  | $O(2^n)$ | $O(n)$ (stack) |
| DP (Memo/Tab) | $O(n)$   | $O(n)$ (tabla) |

\* La DP reduce la complejidad de exponencial a lineal en muchos casos.

### En entrevistas

### Resumen rápido

- DP = Recursión + Memoria.
- No calcules lo mismo dos veces.
- Overlapping Subproblems: La clave para usar DP.
- Top-Down: Fácil de pensar (recursivo).
- Bottom-Up: Más eficiente (iterativo).
- Esencial para problemas de optimización.



# Capítulo 46

## DP - Memoization (Top-Down)

### ¿Qué es?

Un chef que anota recetas nuevas en un cuaderno .

Imagina que estás cocinando platillos complejos. La primera vez que haces una salsa secreta, tardas mucho tiempo y esfuerzo. En lugar de olvidar cómo la hiciste, anotas los pasos y el resultado en tu cuaderno.

La siguiente vez que alguien pide ese platillo, no vuelves a experimentar; simplemente abres tu cuaderno, lees la receta y ¡listo! Tienes el resultado al instante.

En programación, Memoization es el cuaderno donde guardas el resultado de una función para ciertos argumentos, para no tener que volver a ejecutar toda la lógica si te vuelven a pedir lo mismo.

### Dicho de otra forma

Memoization es la técnica de DP basada en Recursión (Top-Down).

Empiezas con el problema original (el más grande) y lo vas rompiendo en partes pequeñas. Antes de regresar el valor de una función, lo guardas en una estructura de datos (generalmente un Array o un Hash Map) donde la Key son los argumentos de la función.

Es muy natural de implementar si ya tienes una solución recursiva, ya que solo requiere agregar el paso de "Revisar cuaderno" y "Anotar en cuaderno".

### Diagrama

Llamada:  $f(5)$

1. ¿ $f(5)$  está en el cuaderno?
  - SÍ: Regresa el valor guardado.  $O(1)$  ☑
  - NO:
    - a. Calcula  $f(5)$  haciendo  $f(4) + f(3)$ .
    - b. Guarda el resultado: `cuaderno[5] = resultado`.
    - c. Regresa el resultado.

¡Esto convierte un árbol de recursión gigante en un camino lineal!

### ¿Cómo funciona?

Para cualquier función recursiva: 1. Pasa una estructura 'memo' como parámetro (o úsala global). 2. CASO BASE: Define cuándo detener la recursión. 3. REVISAR MEMO: Si el resultado ya está calculado, regresa-lo. 4. CALCULAR Y GUARDAR: Calcula el valor, guárdalo en 'memo' y regresa-lo.

## PSEUDOCÓDIGO

```
función minMonedas(monto, monedas, memo):
 SI monto == 0: return 0
 SI monto < 0: return INFINITO
 SI monto EN memo: return memo[monto]

 minimo = INFINITO
 PARA cada moneda EN monedas:
 resultado = minMonedas(monto - moneda, monedas, memo)
 SI resultado != INFINITO:
 minimo = min(minimo, resultado + 1)

 memo[monto] = minimo
 return minimo
```

## CÓDIGO C++

```
1 #include <iostream>
2 #include <vector>
3 #include <unordered_map>
4 #include <algorithm>
5 using namespace std;
6
7 // Problema: Coin Change (Mínimo de monedas para una cantidad)
8 // Tiempo: O(monto * n_monedas)
9 // Espacio: O(monto) para el memo
10 int solve(int amount, vector<int>& coins, unordered_map<int, int>& memo)
11 {
12 if (amount == 0) return 0;
13 if (amount < 0) return -1;
14 if (memo.count(amount)) return memo[amount];
15
16 int min_coins = 1e9;
17 bool possible = false;
18
19 for (int coin : coins) {
20 int res = solve(amount - coin, coins, memo);
```

```

20 if (res >= 0) {
21 min_coins = min(min_coins, res + 1);
22 possible = true;
23 }
24 }
25
26 return memo[amount] = (possible ? min_coins : -1);
27 }
28
29 int main() {
30 vector<int> coins = {1, 2, 5};
31 int amount = 11;
32 unordered_map<int, int> memo;
33
34 cout << "Mínimo de monedas para " << amount << ": "
35 << solve(amount, coins, memo) << endl; // 3 (5+5+1)
36
37 return 0;
38 }

```

### Complejidad Big-O

Aspecto | Con Memoization |  
 Tiempo |  $O(\text{Estados Únicos} * \text{Trabajo por Estado})$  Espacio |  $O(\text{Estados Únicos})$   
 + Stack de recursión

\* Para Coin Change:  $O(\text{Amount} * N\_coins)$  tiempo. Mucho mejor que  $O(N\_coins^{\text{Amount}})$  de la fuerza bruta.

### En entrevistas

### Resumen rápido

- Top-Down: Del problema grande a los pequeños.
- Recursión + Diccionario/Array de resultados.
- Anota resultados para no repetir trabajo.
- Transforma complejidad exponencial en lineal/polinomial.
- Ideal para cuando la estructura del problema es un árbol.

# Capítulo 47

## DP - Tabulation (Bottom-Up)

### ¿Qué es?

Construir una pared de ladrillos desde el suelo hacia arriba.

En lugar de empezar por el techo (el problema grande) y tratar de adivinar qué ladrillos necesitas abajo (recursión), empiezas poniendo los primeros ladrillos en la base (los casos base que ya conoces).

Una vez que tienes la primera fila, puedes poner la segunda encima usando la base como apoyo. Vas llenando tu tabla de resultados paso a paso hasta que llegas a la altura que querías (el resultado final).

### Dicho de otra forma

Tabulation es la técnica de DP basada en Iteración (Bottom-Up).

Se llama así porque literalmente llenas una tabla (un array de 1D o 2D) con los resultados de los sub-problemas, empezando por los más pequeños. Cada nueva entrada de la tabla se calcula usando los valores que ya están guardados en las posiciones anteriores.

Es generalmente más eficiente que Memoization porque evita el "overhead" (sobrecosto) de las llamadas recursivas y no corre el riesgo de causar un Stack Overflow.

### Diagrama

Queremos `fib(5)`.

Tabla DP de tamaño 6: [ \\_, \\_, \\_, \\_, \\_, \\_ ]

1. Llenamos bases: `dp[0]=0`, `dp[1]=1`  
[ 0, 1, \\_, \\_, \\_, \\_ ]

2. Llenamos el resto usando `dp[i] = dp[i-1] + dp[i-2]`:  
`i=2`: [ 0, 1, 1, \\_, \\_, \\_ ]  
`i=3`: [ 0, 1, 1, 2, \\_, \\_ ]  
`i=4`: [ 0, 1, 1, 2, 3, \\_ ]  
`i=5`: [ 0, 1, 1, 2, 3, 5 ] → Resultado final

## ⚙️ ¿Cómo funciona?

1. Inicializa una tabla (array o matriz) con valores default. 2. Define los casos base en las primeras posiciones de la tabla. 3. Escribe un loop (o varios) que recorra la tabla. 4. En cada paso, aplica la lógica para llenar la celda actual usando celdas anteriores. 5. Regresa el valor de la última celda relevante.

## PSEUDOCÓDIGO

```
función minMonedasTab(monto, monedas):
 tabla = array de tamaño (monto + 1) con INFINITO
 tabla[0] = 0

 PARA cada moneda EN monedas:
 PARA i desde moneda hasta monto:
 SI tabla[i - moneda] != INFINITO:
 tabla[i] = min(tabla[i], tabla[i - moneda] + 1)

 return tabla[monto]
```

## CÓDIGO C++

```
1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4 using namespace std;
5
6 // Problema: Climbing Stairs (¿De cuántas formas puedes subir N
7 // si puedes dar pasos de 1 o 2?) -> O(n) tiempo, O(n) espacio
8 int climbStairs(int n) {
9 if (n <= 2) return n;
10
11 vector<int> dp(n + 1);
12 dp[1] = 1;
13 dp[2] = 2;
14
15 for (int i = 3; i <= n; i++) {
16 dp[i] = dp[i - 1] + dp[i - 2];
17 }
18
19 return dp[n];
20 }
21
22 // OPTIMIZACIÓN DE ESPACIO: O(n) -> O(1)
23 // Como solo necesitamos los últimos 2 valores, no hace falta el array.
```

```

24 int climbStairsOptimized(int n) {
25 if (n <= 2) return n;
26 int prev2 = 1, prev1 = 2;
27 for (int i = 3; i <= n; i++) {
28 int current = prev1 + prev2;
29 prev2 = prev1;
30 prev1 = current;
31 }
32 return prev1;
33 }
34
35 int main() {
36 int stairs = 5;
37 cout << "Formas de subir " << stairs << " escalones: "
38 << climbStairsOptimized(stairs) << endl; // 8
39 return 0;
40 }

```

### Complejidad Big-O

|         |                                                                       |
|---------|-----------------------------------------------------------------------|
| Aspecto | Con Tabulation                                                        |
| Tiempo  | $O(\text{Tamaño de la Tabla} * \text{Trabajo por Celda})$             |
| Espacio | $O(\text{Tamaño de la Tabla}) \rightarrow$ A veces reducible a $O(1)$ |

\* Al ser iterativo, el tiempo es muy predecible y constante.



### En entrevistas



### Resumen rápido

- Bottom-Up: De lo simple a lo complejo.
- Iterativo: Sin recursión, sin stack overflow.
- Llenas una tabla paso a paso.
- Más eficiente en tiempo y memoria.
- Permite optimizar espacio fácilmente (usando variables en vez de array).
- Es la forma "profesional" de entregar una solución DP.

# Capítulo 48

## DP: Knapsack, Subset Sum y Coin Change

### ¿Qué es?

Una mochila que aguanta cierto peso máximo y un montón de objetos, cada uno con su peso y su valor. ¿Qué combinación METES para maximizar el valor sin pasarte del peso?

No puedes ser goloso (agarrar lo más valioso) porque a veces dos objetos medianos ganan a uno grande. Hay que PROBAR combinaciones, pero de forma inteligente: recordando la mejor respuesta para cada "peso disponible" y reusándola. Eso es Programación Dinámica: no recalcular lo ya resuelto.

### Explicado para un niño de 12

Tienes una mochila que solo aguanta 5 kilos y varios juguetes. Cada juguete pesa algo y te gusta más o menos (su "valor"). Quieres llevarte los juguetes que MÁS te gusten sin que la mochila pese más de 5 kilos.

El truco es hacer una tabla. Vas juguete por juguete preguntando: "si tuviera espacio para 1 kilo, para 2, para 3... ¿qué es lo mejor que puedo llevar?". Para cada juguete decides: ¿lo dejo fuera, o lo meto y sumo su valor? Te quedas con la opción que dé MÁS valor. Como escribes las respuestas en la tabla, nunca tienes que repetir la misma cuenta dos veces. Al final, la esquina de la tabla te dice el mejor botín posible.

### Dicho de otra forma

El **0/1 Knapsack** es el problema base de una familia enorme de DP. Con  $n$  objetos (peso  $w_i$ , valor  $v_i$ ) y capacidad  $W$ , maximiza el valor total eligiendo un subconjunto cuyo peso no supere  $W$ . "0/1" porque cada objeto se toma entero o no se toma.

La recurrencia, con  $dp[i][c]$  = mejor valor usando los primeros  $i$  objetos con capacidad  $c$ :

$$dp[i][c] = \max(dp[i-1][c], v_i + dp[i-1][c - w_i])$$

(no tomar el objeto  $i$ , o tomarlo si cabe).

Variantes que son el MISMO esqueleto:

- **Subset Sum / Partition:** ¿hay un subconjunto que sume  $S$ ? (knapsack booleano).
- **Coin Change:** mínimo de monedas para un monto (unbounded knapsack: cada moneda se repite).

### Diagrama

0/1 KNAPSACK ( $W = 5$ )      objetos: (peso, valor)  
 $o1=(2,3)$     $o2=(3,4)$     $o3=(4,5)$     $o4=(5,6)$

Tabla  $dp[objetos][capacidad]$ , mejor valor:

|      |   |   |   |   |   |   |                                         |
|------|---|---|---|---|---|---|-----------------------------------------|
| cap: | 0 | 1 | 2 | 3 | 4 | 5 |                                         |
| {}   | 0 | 0 | 0 | 0 | 0 | 0 |                                         |
| $o1$ | 0 | 0 | 3 | 3 | 3 | 3 |                                         |
| $o2$ | 0 | 0 | 3 | 4 | 4 | 7 | $\leftarrow o1+o2 = 3+4 = 7$ con peso 5 |
| $o3$ | 0 | 0 | 3 | 4 | 5 | 7 |                                         |
| $o4$ | 0 | 0 | 3 | 4 | 5 | 7 |                                         |

Respuesta =  $dp[n][W] = 7$  (llevar  $o1$  y  $o2$ )

Cada celda:  $\max(\text{de arriba, valor} + \text{celda arriba-desplazada por su peso})$

#### CÓMO FUNCIONA (0/1 knapsack)

- $dp[c]$  = mejor valor con capacidad  $c$  (versión 1D optimizada).
- Por **cada** objeto, recorre la capacidad de  $W$  hacia ABAJO (para no reusar el mismo objeto dos veces).
- $dp[c] = \max(dp[c], \text{valor} + dp[c - \text{peso}])$ .

#### PSEUDOCÓDIGO (0/1 knapsack 1D)

```
función knapsack(pesos, valores, W):
 dp = arreglo de ceros de tamaño W+1
 PARA i desde 0 hasta n-1:
 PARA c desde W hasta pesos[i]: // ¡hacia abajo!
 dp[c] = max(dp[c], valores[i] + dp[c - pesos[i]])
 return dp[W]
```

#### CÓDIGO C++ (0/1 knapsack, 1D)

```
#include <vector>
#include <algorithm>
using namespace std;

int knapsack(vector<int>& peso, vector<int>& valor, int W) {
 vector<int> dp(W + 1, 0);
```



```

26 for (int i = 0; i < (int)peso.size(); i++)
27 for (int c = W; c >= peso[i]; c--) // hacia abajo: 0/1
28 dp[c] = max(dp[c], valor[i] + dp[c - peso[i]]);
29 return dp[W];
30 }

```

31

32 CÓDIGO C++ (Subset Sum: ¿existe subconjunto que sume S?)

---

```

33
34
35 bool subsetSum(vector<int>& nums, int S) {
36 vector<bool> dp(S + 1, false);
37 dp[0] = true; // suma 0 siempre posible
38 for (int x : nums)
39 for (int c = S; c >= x; c--) // hacia abajo
40 dp[c] = dp[c] || dp[c - x];
41 return dp[S];
42 }

```

43

44 CÓDIGO C++ (Coin Change: mínimo de monedas, unbounded)

---

```

45
46
47 int coinChange(vector<int>& coins, int amount) {
48 vector<int> dp(amount + 1, amount + 1); // "infinito"
49 dp[0] = 0;
50 for (int c : coins)
51 for (int a = c; a <= amount; a++) // hacia ARRIBA:
52 // repetible
53 dp[a] = min(dp[a], dp[a - c] + 1);
54 return dp[amount] > amount ? -1 : dp[amount];
55 }

```

56 COMPLEJIDAD (Big-O)

---

| Problema     | Tiempo         | Espacio |
|--------------|----------------|---------|
| 0/1 Knapsack | $O(n \cdot W)$ | $O(W)$  |
| Subset Sum   | $O(n \cdot S)$ | $O(S)$  |
| Coin Change  | $O(n \cdot A)$ | $O(A)$  |

64

65 \* Es "pseudo-polinómico": depende del VALOR W, no solo de n.

### En entrevistas

Dirección del bucle interno = la clave: hacia ABAJO ( $W$ ..peso) para 0/1 (cada objeto una vez); hacia ARRIBA (peso.. $W$ ) para unbounded (objeto repetible, como monedas). Confundirlas cambia el resultado.

"Partition Equal Subset Sum" (LC 416): ¿se puede partir en dos mitades de igual suma? Es Subset Sum con  $S = \text{total}/2$ . Si el total es impar, imposible de una.

Pseudo-polinómico:  $O(n \cdot W)$  parece polinomial pero  $W$  puede ser enorme. Si  $W$  es gigante y  $n$  pequeño, quizá otra técnica convenga.

Reconstruir la solución: si te piden CUÁLES objetos, guarda la tabla 2D y recórrela hacia atrás comparando  $dp[i][c]$  con  $dp[i-1][c]$ .

### Resumen rápido

- 0/1 Knapsack: max valor sin pasar la capacidad; cada objeto entero o no.
- Recurrencia:  $\max(\text{no tomar}, \text{valor} + \text{tomar si cabe})$ .
- Optimización 1D: bucle de capacidad hacia ABAJO para 0/1.
- Subset Sum / Partition: knapsack booleano (¿suma  $S$  posible?).
- Coin Change: unbounded knapsack; bucle hacia ARRIBA (repetible).
- Complejidad pseudo-polinómica  $O(n \cdot W)$ .

---

---

# Capítulo 49



## DP en Cadenas: LCS y Edit Distance



### ¿Qué es?

Comparar dos palabras o textos para ver qué tanto se parecen. Dos preguntas clásicas:

- LCS (Longest Common Subsequence): ¿cuál es la secuencia de letras más larga que aparece en AMBAS, en el mismo orden (aunque no pegadas)?
- Edit Distance: ¿cuántos cambios mínimos (insertar, borrar, sustituir una letra) necesito para convertir una palabra en la otra?

Ambas se resuelven con una tabla 2D donde cada celda compara un prefijo de una cadena contra un prefijo de la otra.



### Explicado para un niño de 12

Tienes dos collares de letras y quieres saber qué tanto se parecen.

Para el LCS: buscas las letras que puedes ir tachando en los dos collares al mismo tiempo, en orden, formando la palabra escondida más larga que comparten. Si las puntas coinciden, esa letra cuenta y avanzas en las dos. Si no, pruebas quitar la punta de un collar o del otro y te quedas con la mejor opción.

Para el Edit Distance: imagina que el corrector de tu celular quiere cambiar una palabra por otra. Cuenta cuántos "arreglos" mínimos hace: meter una letra, quitar una, o cambiar una por otra. Menos arreglos = más parecidas. Escribes todo en una tabla para no repetir cuentas, y la esquina te da la respuesta.



### Dicho de otra forma

Ambos problemas usan una tabla  $dp[i][j]$  que compara el prefijo de largo  $i$  de la cadena  $A$  con el prefijo de largo  $j$  de la cadena  $B$ .

**LCS** (longest common subsequence): si  $A_i = B_j$  entonces  $dp[i][j] = dp[i-1][j-1] + 1$ ; si no,  $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$ .

**Edit Distance** (distancia de Levenshtein): mínimo de inserciones, borrados y sustituciones. Si las letras coinciden, no cuesta nada; si no,  $1 + \min$  de las tres operaciones.

Son la base de: diff de archivos, autocorrector, alineamiento de ADN y detección

de plagio.

## Diagrama

LCS de "ABCBBDAB" y "BDCAB" (respuesta: "BCAB", largo 4)

|   |    | B | D | C | A | B |
|---|----|---|---|---|---|---|
|   | "" | 0 | 0 | 0 | 0 | 0 |
| A | 0  | 0 | 0 | 0 | 1 | 1 |
| B | 0  | 1 | 1 | 1 | 1 | 2 |
| C | 0  | 1 | 1 | 2 | 2 | 2 |
| B | 0  | 1 | 1 | 2 | 2 | 3 |
| D | 0  | 1 | 2 | 2 | 2 | 3 |
| A | 0  | 1 | 2 | 2 | 3 | 3 |
| B | 0  | 1 | 2 | 2 | 3 | 4 |

<- LCS = 4

Diagonal +1 si las letras coinciden; si no, el máximo de izq/arriba.

### CÓMO FUNCIONA

- Tabla (n+1) x (m+1); fila/columna 0 = cadena vacía.
- LCS: si  $A[i-1] == B[j-1]$  -> diagonal+1; si no -> max(arriba, izquierda).
- Edit Distance: si coinciden -> diagonal; si no -> 1 + min(insertar, borrar, sustituir) = 1 + min(dp[i][j-1], dp[i-1][j], dp[i-1][j-1]).

### PSEUDOCÓDIGO (LCS)

```
función lcs(A, B):
 dp = tabla (n+1) x (m+1) de ceros
 PARA i desde 1 hasta n:
 PARA j desde 1 hasta m:
 SI A[i-1] == B[j-1]:
 dp[i][j] = dp[i-1][j-1] + 1
 SINO:
 dp[i][j] = max(dp[i-1][j], dp[i][j-1])
 return dp[n][m]
```

### CÓDIGO C++ (LCS)

```
#include <string>
#include <vector>
#include <algorithm>
using namespace std;

int lcs(const string& A, const string& B) {
 int n = A.size(), m = B.size();
 vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
```

```

31 for (int i = 1; i <= n; i++)
32 for (int j = 1; j <= m; j++)
33 if (A[i-1] == B[j-1]) dp[i][j] = dp[i-1][j-1] + 1;
34 else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
35 return dp[n][m];
36 }
37
38 CÓDIGO C++ (Edit Distance / Levenshtein)
39
40
41 int editDistance(const string& A, const string& B) {
42 int n = A.size(), m = B.size();
43 vector<vector<int>> dp(n + 1, vector<int>(m + 1));
44 for (int i = 0; i <= n; i++) dp[i][0] = i; // borrar todo A
45 for (int j = 0; j <= m; j++) dp[0][j] = j; // insertar todo B
46 for (int i = 1; i <= n; i++)
47 for (int j = 1; j <= m; j++)
48 if (A[i-1] == B[j-1]) dp[i][j] = dp[i-1][j-1]; // sin costo
49 else dp[i][j] = 1 + min({ dp[i][j-1], // insertar
50 dp[i-1][j], // borrar
51 dp[i-1][j-1] }); // sustituir
52 return dp[n][m];
53 }
54
55 COMPLEJIDAD (Big-O)
56
57
58 Problema | Tiempo | Espacio
59
60 LCS | O(n·m) | O(n·m) o O(m) 1D
61 Edit Distance | O(n·m) | O(n·m) o O(m) 1D
62
63 * Se puede reducir a O(m) espacio guardando solo la fila anterior.

```

## En entrevistas

Subsequence vs Substring: LCS permite huecos (subsecuencia); "longest common substring" exige letras contiguas y usa otra recurrencia (reinicia a 0 cuando no coinciden). No los confundas.

"Edit Distance" (LC 72) es de los DP 2D más preguntados. La clave son los tres vecinos: izquierda (insertar), arriba (borrar), diagonal (sustituir).

Casos base primero: la fila 0 y la columna 0 no son ceros en Edit Distance: valen 0,1,2,3... (convertir a/desde la cadena vacía).

Reconstruir la subsecuencia: recorre la tabla desde `dp[n][m]` hacia atrás; cuando las letras coinciden, esa letra va en el LCS y te mueves en diagonal.



## Resumen rápido

- LCS: subsecuencia común más larga (permite huecos, respeta el orden).
- LCS:  $\text{diagonal} + 1$  si coinciden; si no,  $\max(\text{arriba}, \text{izquierda})$ .
- Edit Distance: mínimos insert/borrar/sustituir para igualar cadenas.
- Edit Distance:  $1 + \min(\text{izq}, \text{arriba}, \text{diagonal})$  si no coinciden.
- Ambos  $O(n \cdot m)$ ; reducibles a  $O(m)$  espacio con una fila.
- Usos: diff, autocorrector, ADN, plagio.

# Capítulo 50

## ✓ DP: Longest Increasing Subsequence (LIS)

### 🤖 ¿Qué es?

De una lista de números, encontrar la subsecuencia CRECIENTE más larga: los números van subiendo (no importa si te saltas algunos, pero deben ir en orden y de menor a mayor).

Ejemplo: en  $[10, 9, 2, 5, 3, 7, 101, 18]$  una subida larga es  $[2, 3, 7, 18]$  o  $[2, 5, 7, 101]$ , ambas de largo 4. Hay dos formas de resolverlo: una simple  $O(n^2)$  y una astuta  $O(n \log n)$  con búsqueda binaria.

### 🧒 Explicado para un niño de 12

Imagina que anotas tu estatura cada año, pero en desorden. Quieres encontrar la racha más larga en la que fuiste creciendo, aunque tengas que ignorar algunos años.

La forma fácil: para cada número te preguntas "¿cuál es la subida más larga que TERMINA justo aquí?". Miras todos los números anteriores más pequeños y te quedas con la mejor racha, más uno.

La forma lista: vas armando una "fila de paciencia". Cada número nuevo reemplaza al primer número de la fila que sea igual o más grande que él (así la fila queda lo más chica posible). Al final, el LARGO de esa fila es la respuesta. Es como un solitario de cartas donde apilas de forma ordenada.

### 💬 Dicho de otra forma

El LIS busca la subsecuencia estrictamente creciente más larga de un arreglo (los elementos conservan su orden original, no tienen que ser contiguos).

**Enfoque DP**  $O(n^2)$ :  $dp[i]$  = longitud del LIS que termina en el índice  $i$ . Para cada  $i$ , mira todos los  $j < i$  con  $nums[j] < nums[i]$  y toma  $dp[i] = \max(dp[i], dp[j] + 1)$ .

**Enfoque**  $O(n \log n)$ : mantén un arreglo **tails** donde **tails**[ $k$ ] es el menor valor con el que puede terminar una subsecuencia creciente de longitud  $k + 1$ . Por cada número, usa búsqueda binaria (**lower\_bound**) para colocarlo; el tamaño final de **tails** es la respuesta. Ojo: **tails** NO es el LIS real, solo su longitud.

 Diagrama

```
nums = [10, 9, 2, 5, 3, 7, 101, 18]
```

Método  $O(n \log n)$  construyendo 'tails' (lower\_bound reemplaza):

```
10 -> tails=[10]
9 -> reemplaza 10 -> [9]
2 -> reemplaza 9 -> [2]
5 -> agrega -> [2,5]
3 -> reemplaza 5 -> [2,3]
7 -> agrega -> [2,3,7]
101 -> agrega -> [2,3,7,101]
18 -> reemplaza 101 -> [2,3,7,18]
```

Longitud de tails = 4  $\Rightarrow$  LIS = 4

(tails = [2,3,7,18] NO es la subsecuencia real, pero su LARGO sí es 4)

1 CÓMO FUNCIONA ( $O(n \log n)$ )

2 - tails vacío. Por cada x:

3 \* Busca con lower\_bound la primera posición con valor  $\geq x$ .

4 \* Si no existe (x es mayor que todos) -> agrega x al final.

5 \* Si existe -> reemplaza ese valor por x (mantiene tails "apretado").

6 - La longitud de tails es el LIS.

7 PSEUDOCÓDIGO ( $O(n \log n)$ )

10 función lis(nums):

11 tails = []

12 PARA cada x en nums:

13 pos = lower\_bound(tails, x) // primer  $\geq x$

14 SI pos == fin(tails): tails.push(x)

15 SINO: tails[pos] = x

16 return longitud(tails)

18 CÓDIGO C++ (DP  $O(n^2)$ , fácil de entender)

21 #include <vector>

22 #include <algorithm>

23 using namespace std;

24 int lisDP(vector<int>& nums) {

25 int n = nums.size();

26 vector<int> dp(n, 1);

// cada elemento es un LIS de

1

27 int best = n ? 1 : 0;



```

30 for (int i = 1; i < n; i++)
31 for (int j = 0; j < i; j++)
32 if (nums[j] < nums[i]) {
33 dp[i] = max(dp[i], dp[j] + 1);
34 best = max(best, dp[i]);
35 }
36 return best;
37 }

```

38  
39 CÓDIGO C++ ( $O(n \log n)$  con búsqueda binaria)

```

40
41
42 int lisFast(vector<int>& nums) {
43 vector<int> tails;
44 for (int x : nums) {
45 auto it = lower_bound(tails.begin(), tails.end(), x);
46 if (it == tails.end()) tails.push_back(x); // extiende
47 else *it = x; // reemplaza
48 }
49 return tails.size();
50 }

```

51  
52 COMPLEJIDAD (Big-O)

| Enfoque       | Tiempo        | Espacio |
|---------------|---------------|---------|
| DP clásico    | $O(n^2)$      | $O(n)$  |
| Binary search | $O(n \log n)$ | $O(n)$  |

59  
60 \* Para LIS no decreciente (permite iguales), usa `upper_bound`.

## En entrevistas

"Longest Increasing Subsequence" (LC 300): si te piden solo la LONGITUD, ve directo al  $O(n \log n)$ . Si piden reconstruir la subsecuencia real, el DP  $O(n^2)$  con arreglo de padres es más claro.

Estricto vs no decreciente: `lower_bound` da LIS estrictamente creciente; `upper_bound` permite elementos iguales (no decreciente). Un detalle que cambia la respuesta.

`tails` no es la subsecuencia: mucha gente cree que `tails` ES el LIS. Solo su LONGITUD es correcta; los valores pueden no formar una subsecuencia válida.

Disfraces del LIS: "Russian Doll Envelopes", "Number of LIS", "Longest Chain of Pairs" son LIS con un ordenamiento previo. Reconocer el patrón vale oro.



### Resumen rápido

- LIS: subsecuencia creciente más larga (respeta orden, permite huecos).
- DP  $O(n^2)$ :  $dp[i]$  = mejor LIS que termina en  $i$ ; mira los  $j < i$  menores.
- $O(n \log n)$ : arreglo `tails` + **lower\_bound**; su longitud es el LIS.
- `tails` da la LONGITUD correcta, no la subsecuencia real.
- Estricto -> **lower\_bound**; no decreciente -> **upper\_bound**.
- Muchos problemas se reducen a LIS tras ordenar.

# Capítulo 51

## Greedy (Algoritmos Golosos)

### ¿Qué es?

Tomar SIEMPRE la mejor opción del momento , sin pensar en el futuro, esperando que esas decisiones locales sumen la mejor solución global.

Si quieres dar cambio con la menor cantidad de monedas, tomas la moneda más grande que quepa una y otra vez. Eso es ser "goloso": eliges lo mejor AHORA y no reconsideras. A veces funciona perfecto (monedas normales), a veces falla (monedas raras), y ahí está el arte: saber CUÁNDO un problema se puede resolver golosamente.

### Explicado para un niño de 12

Imagina que tienes una mochila y quieres meter la mayor cantidad de dulces , pero solo puedes agarrar de a uno rapidísimo. La regla golosa dice: "agarra el dulce más grande que veas, no lo pienses". Sigues agarrando el más grande que quede, hasta llenarte.

A veces esa regla te da lo mejor. Pero ojo: si los dulces vinieran pegados de formas raras, agarrar siempre el más grande podría dejarte sin espacio para dos medianos que juntos valían más. Por eso, antes de ser goloso, hay que estar SEGURO de que "lo mejor ahorita" también es "lo mejor al final". Cuando eso es cierto, ser goloso es rapidísimo y fácil.

### Dicho de otra forma

Un algoritmo greedy construye la solución paso a paso, eligiendo en cada etapa la opción que se ve mejor localmente (según un criterio), sin retroceder.

Solo da el óptimo si el problema cumple dos propiedades:

- **Greedy choice property:** una elección local óptima lleva a una solución global óptima.
- **Subestructura óptima:** la solución óptima contiene soluciones óptimas de sus subproblemas.

Si NO se cumplen, greedy falla y hay que usar Programación Dinámica. Ejemplos donde greedy SÍ funciona: activity selection, intervalos, código de Huffman, Dijkstra, MST (Kruskal/Prim), fractional knapsack.

 Diagrama

ACTIVITY SELECTION (máximo de reuniones sin traslape):

Reuniones (inicio, fin):

A(1,3) B(2,5) C(4,7) D(6,8) E(5,9)

Paso 1: ordena por HORA DE FIN: A(3) B(5) C(7) D(8) E(9)

Paso 2: toma A. Última fin = 3.

B empieza en 2 < 3 -> choca, salta.

C empieza en 4 >= 3 -> toma C. Última fin = 7.

D empieza en 6 < 7 -> choca, salta.

E empieza en 5 < 7 -> choca, salta.

Resultado: {A, C} -> 2 reuniones (óptimo).

Criterio goloso ganador: SIEMPRE la que TERMINA más temprano.

PATRÓN GREEDY (3 pasos)

1. Encuentra el criterio de ordenamiento correcto (el "más pequeño", el que "termina antes", el "más valioso por kilo"...).
2. Ordena por ese criterio.
3. Recorre una vez tomando lo que quepa/no choque.

PSEUDOCÓDIGO (activity selection)

---

función maxActividades(intervalos):

ordenar intervalos por hora de FIN ascendente

finAnterior = -infinito; conteo = 0

PARA cada (inicio, fin) en intervalos:

SI inicio >= finAnterior:

conteo++; finAnterior = fin // se toma

return conteo

CÓDIGO C++ (activity selection / non-overlapping intervals)

---

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
int maxActividades(vector<pair<int,int>>& iv) { // {inicio, fin}
```

```
sort(iv.begin(), iv.end(),
```

```
 [](auto& a, auto& b){ return a.second < b.second; }); // por
 fin
```

```
int fin = INT_MIN, cont = 0;
```

```
for (auto& [ini, f] : iv) {
```

```
 if (ini >= fin) { cont++; fin = f; } // no choca -> tómala
```

```

31 }
32 return cont;
33 }
34
35 CÓDIGO C++ (fractional knapsack: greedy SÍ funciona)
36
37
38 // A diferencia del 0/1 knapsack (que es DP), aquí puedes partir objetos
39
40 double fractionalKnapsack(int W, vector<pair<int,int>>& items) {
41 // items = {valor, peso}; ordena por valor/peso DESC
42 sort(items.begin(), items.end(), [](auto& a, auto& b){
43 return (double)a.first/a.second > (double)b.first/b.second;
44 });
45 double total = 0;
46 for (auto& [val, peso] : items) {
47 if (W >= peso) { total += val; W -= peso; } // cabe entero
48 else { total += val * ((double)W / peso); break; } // fracción
49 }
50 return total;
51 }
52
53 COMPLEJIDAD (Big-O)
54
55
56
57
58
59
60
61

```

| Problema            | Tiempo        | Domina     |
|---------------------|---------------|------------|
| Activity selection  | $O(n \log n)$ | el sort    |
| Fractional knapsack | $O(n \log n)$ | el sort    |
| Recorrido greedy    | $O(n)$        | una pasada |

\* Casi siempre el costo es el ORDENAMIENTO inicial.

- 1 
- 2 Problemas greedy clásicos y su criterio ganador:
- 3 - Intervalos sin traslape (LC 435): ordena por FIN, quita los que chocan.
- 4 - Merge Intervals (LC 56): ordena por INICIO, fusiona solapados.
- 5 - Jump Game (LC 55): mantén el alcance máximo; si lo superas, pierdes.
- 6 - Gas Station (LC 134): si el tanque total  $\geq 0$ , hay solución única.
- 7 - Huffman: combina siempre los dos nodos de menor frecuencia (min-heap).

### En entrevistas

El reto es DEMOSTRAR que greedy funciona. Si no puedes justificar por qué la elección local da el óptimo global, probablemente NO sea greedy y necesites DP. No lo uses "de fe".

Contraejemplo de monedas: con monedas 1, 3, 4 y cambio de 6, greedy da  $4+1+1$  (3 monedas) pero lo óptimo es  $3+3$  (2 monedas). Por eso "coin change" general es DP, no greedy.

El criterio de ordenamiento lo es TODO. En activity selection ordenar por duración o por inicio FALLA; solo ordenar por hora de fin da el óptimo. Piensa bien el criterio.

Greedy vs DP: si "elegir lo mejor ahora" nunca te arrepiente  $\rightarrow$  greedy (rápido). Si necesitas explorar combinaciones y recordar subproblemas  $\rightarrow$  DP.

### Resumen rápido

• Greedy: elige lo mejor local en cada paso, sin retroceder. • Solo da el óptimo con greedy-choice + subestructura óptima. • Patrón: encuentra criterio  $\rightarrow$  ordena  $\rightarrow$  recorre tomando lo que quepa. • Activity selection: ordena por hora de FIN. • Fractional knapsack SÍ es greedy; el 0/1 knapsack NO (es DP). • Si no puedes demostrar que funciona, usa DP.

---

---

# Capítulo 52

## Backtracking (Vuelta Atrás)

### ¿Qué es?

Resolver un laberinto con un lápiz.

Sigues un camino con la punta del lápiz hasta que llegas a un callejón sin salida. En lugar de rendirte, borras lo que hiciste (vuelta atrás) hasta la última bifurcación y pruebas el otro camino que dejaste pendiente.

Backtracking es probar TODAS las posibilidades una por una, pero con la inteligencia de "deshacer" tus pasos cuando te das cuenta de que una ruta no lleva a la solución. Es una búsqueda exhaustiva pero controlada.

### Dicho de otra forma

Backtracking es una técnica algorítmica para resolver problemas de forma recursiva, intentando construir una solución incrementalmente.

Se basa en la exploración de un "Árbol de Decisiones". Si en algún punto la solución parcial no cumple con las restricciones del problema (se vuelve inválida), el algoritmo descarta esa rama ("pruning") y regresa al estado anterior para intentar otra opción.

Es la base para resolver problemas de combinatoria, permutaciones y juegos como el Sudoku o el Ajedrez.

### Diagrama

```
[root]
 / \
 Elegir 1? NO elegir 1?
 / \ / \
Elegir 2? NO? Elegir 2? NO?
 | | | |
[1,2] [1] [2] []
```

¡Cada nodo es una decisión! Backtracking recorre este árbol usando DFS.

### ¿Cómo funciona?

Casi todos los problemas de Backtracking siguen este patrón:

1. Caso Base: ¿Ya terminamos de construir la solución? - Sí: Guárdala o imprímela. Regresa.
2. Opciones: Para cada opción disponible en este nivel: - ¿Es válida la opción? (Restricciones). - Sí:
  - a. Hacer Elección: Modifica el estado actual.
  - b. Recursión: Llama a la función para el siguiente nivel.
  - c. Deshacer Elección (BACKTRACK): Regresa el estado como estaba.

## PSEUDOCÓDIGO

```
función backtrack(estado, opciones):
```

```
 SI estado es meta:
 imprimir(estado)
 return
```

```
 PARA cada o EN opciones:
```

```
 SI esValida(o, estado):
 agregar o a estado // "Hacer"
 backtrack(estado, opciones_restantes)
 quitar o de estado // "Deshacer" (La clave 🔑)
```

## CÓDIGO C++

```
1 #include <iostream>
2 #include <vector>
3 #include <string>
4 using namespace std;
5
6 // Problema: Generar todas las permutaciones de un string -> O(n!)
7 void permute(string& s, int l, int r) {
8 // Caso Base
9 if (l == r) {
10 cout << s << endl;
11 return;
12 }
13
14 for (int i = l; i <= r; i++) {
15 // 1. Hacer Elección (Swap)
16 swap(s[l], s[i]);
17
18 // 2. Recursión (Ir al siguiente nivel)
19 permute(s, l + 1, r);
20
21 // 3. Deshacer Elección (Backtrack)
```



```

22 // Regresamos el string a su estado original para la sig.
 iteración
23 swap(s[l], s[i]);
24 }
25 }
26
27 int main() {
28 string str = "ABC";
29 cout << "Permutaciones de ABC:" << endl;
30 permute(str, 0, str.length() - 1);
31 return 0;
32 }

```

## Complejidad Big-O

| Problema      | Tiempo Típico | Razón                         |
|---------------|---------------|-------------------------------|
| Permutaciones | $O(n!)$       | $n * (n-1) * (n-2) \dots$     |
| Subconjuntos  | $O(2^n)$      | Cada elemento se elige o no   |
| N-Queens      | $O(n!)$       | Menos gracias al pruning      |
| Sudoku        | $O(9^{n*n})$  | Fuerza bruta pura (peor caso) |

\* El espacio suele ser  $O(n)$  por la profundidad del stack de recursión.

## En entrevistas

## Resumen rápido

- DFS + Deshacer cambios.
- Explora todas las soluciones posibles.
- Úsalo para permutaciones, combinaciones y laberintos.
- Complejidad alta (Exponencial/Factorial).
- Pruning: Cortar ramas que no sirven para ganar velocidad.
- Esencial dominar el template: Hacer -> Recursión -> Deshacer.

# Capítulo 53

## 12 34 Bit Manipulation (Manipulación de Bits)

### ¿Qué es?

Los interruptores de luz de tu casa.

Cada bit es como un interruptor: 1 significa "encendido" y 0 significa "apagado". Manipular bits es jugar con esos interruptores usando reglas lógicas: - AND: Solo enciende si AMBOS están encendidos. - OR: Enciende si AL MENOS UNO está encendido. - XOR: Solo enciende si EXACTAMENTE UNO está encendido.

Es la forma más profunda y rápida de hablarle a la computadora, directamente en su lenguaje natural de ceros y unos.

### Dicho de otra forma

Bit Manipulation consiste en aplicar operaciones lógicas (AND, OR, XOR, NOT) y de desplazamiento (Shifts) sobre los bits individuales de un número entero.

A nivel de hardware, estas operaciones son las más rápidas que un procesador puede realizar. Usar bits permite ahorrar muchísima memoria y acelerar algoritmos que de otra forma usarían estructuras más pesadas.

### Diagrama

Sean  $A = 5$  (101) y  $B = 3$  (011)

AND ( $\&$ ):  $101 \& 011 = 001$  (1)

OR ( $|$ ):  $101 | 011 = 111$  (7)

XOR ( $\wedge$ ):  $101 \wedge 011 = 110$  (6) <-- ¡Nota:  $1 \wedge 1 = 0$ !

Left Shift ( $\ll$ ):  $5 \ll 1 = 1010$  (10) (Multiplicar por 2)

Right Shift ( $\gg$ ):  $5 \gg 1 = 10$  (2) (Dividir entre 2)

### ¿Cómo funciona?

1.  $x \wedge x = 0$ : Cualquier número XOR consigo mismo da cero. (Base del problema "Single Number"). 2.  $x \wedge 0 = x$ : Cualquier número XOR cero se queda igual. 3.  $n \& (n - 1)$ : ¡Truco de Google! Elimina el bit encendido más a la derecha. Si el

resultado es 0, entonces 'n' era una potencia de 2. 4.  $i \& 1$ : Si da 0 es par, si da 1 es impar. (Más rápido que  $i \% 2$ ). 5.  $n \gg 1$ : Divide entre 2 (truncando). 6.  $1 \ll n$ : Calcula 2 elevado a la potencia 'n'.

## PSEUDOCÓDIGO

```
función contarBits(n):
 contador = 0
 MIENTRAS n > 0:
 n = n & (n - 1) // Eliminar el bit más a la derecha
 contador++
 return contador
```

## CÓDIGO C++

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 // Problema: Encontrar el número que no se repite
6 // en un array donde todos los demás aparecen 2 veces.
7 int findSingleNumber(vector<int>& nums) {
8 int res = 0;
9 for (int x : nums) {
10 res ^= x; // Los duplicados se cancelan: x ^ x = 0
11 }
12 return res;
13 }
14
15 // Verificar si es potencia de 2
16 bool isPowerOfTwo(int n) {
17 return n > 0 && (n & (n - 1)) == 0;
18 }
19
20 int main() {
21 vector<int> data = {4, 1, 2, 1, 2};
22 cout << "Número único: " << findSingleNumber(data) << endl; // 4
23
24 int val = 16;
25 cout << "¿16 es potencia de 2? " << isPowerOfTwo(val) << endl; // 1
26
27 return 0;
28 }
```

 Complejidad Big-O

| Operación            | Tiempo | Espacio |
|----------------------|--------|---------|
| Bitwise (&,  , ^, ~) | O(1)   | O(1)    |
| Shifts («, »)        | O(1)   | O(1)    |

\* Son las operaciones más eficientes posibles en computación.

 En entrevistas Resumen rápido

- Operaciones a nivel de ceros y unos.
- Súper rápidas O(1).
- AND (&), OR (|), XOR (^), NOT (~).
- Shifts: « (x2), » (/2).
- $n \& (n-1)$  = Truco para potencias de 2.
- XOR cancela duplicados ( $x \wedge x = 0$ ).
- Ideal para problemas de eficiencia extrema.

# Capítulo 54

## Recursión y Memoization

### ¿Qué es?

Las muñecas rusas Matryoshka .

Abres una muñeca grande y adentro hay otra igual pero más pequeña. Sigues abriendo muñecas hasta que llegas a la más pequeñita de todas, la cual ya no tiene nada adentro. Ese es tu "punto de parada".

Recursión es una función que se llama a sí misma para resolver una versión más pequeña del mismo problema. Sigues bajando niveles hasta llegar al "Caso Base" (la muñeca más chica), y luego vas regresando hacia arriba con las respuestas de cada nivel.

### Dicho de otra forma

La Recursión es un paradigma de programación donde una función se define en términos de sí misma.

Para que una función recursiva sea correcta, DEBE tener: 1. Caso Base: La condición de salida que detiene las llamadas. 2. Caso Recursivo: La lógica que rompe el problema en una parte más pequeña y hace la auto-llamada.

Si olvidas el Caso Base, la función se llamará infinitamente hasta agotar la memoria del sistema (Stack Overflow).

### Diagrama

Llamada inicial: `fact(3)`

```
[fact(1)] -> retorna 1 (¡Caso Base!)
[fact(2)] -> espera a fact(1)... recibe 1, retorna 2 * 1 = 2
[fact(3)] -> espera a fact(2)... recibe 2, retorna 3 * 2 = 6
```

Resultado Final: 6

### ¿Cómo funciona?

- Recursión: Más elegante, fácil de leer para problemas de árboles o gráficos. Usa memoria en el "Call Stack" para cada llamada. - Iteración (Loops): Más eficiente en memoria ( $O(1)$  espacio extra). A veces es más difícil de pensar para problemas complejos.

Memoization (La mejora): Como vimos en DP, si una función recursiva se llama con los mismos argumentos muchas veces, guardamos el resultado en un caché (Memoization) para que la segunda llamada sea  $O(1)$ .

## PSEUDOCÓDIGO

función factorial(n):

```
SI n <= 1: return 1 // Caso Base
return n * factorial(n - 1) // Caso Recursivo
```

## CÓDIGO C++

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 // Recursión Simple
6 long long factorial(int n) {
7 if (n <= 1) return 1;
8 return n * factorial(n - 1);
9 }
10
11 // Recursión con Memoization (Fibonacci)
12 long long fibMemo(int n, vector<long long>& memo) {
13 if (n <= 1) return n;
14 if (memo[n] != -1) return memo[n]; // Revisar caché
15
16 return memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
17 }
18
19 int main() {
20 cout << "Factorial de 5:" << factorial(5) << endl; // 120
21
22 int n = 40;
23 vector<long long> memo(n + 1, -1);
24 cout << "Fibonacci de " << n << ": " << fibMemo(n, memo) << endl;
25
26 return 0;
27 }
```

### Complejidad Big-O

| Algoritmo | Tiempo | (Normal) | Con | Memoization |
|-----------|--------|----------|-----|-------------|
|           |        |          |     |             |

Factorial |  $O(n)$  |  $O(n)$  Fibonacci |  $O(2^n)$  |  $O(n)$  Espacio |  $O(n)$  (Stack)  
|  $O(n)$  (Stack + Cache)

### En entrevistas

### Resumen rápido

- Función que se llama a sí misma.
- Requiere Caso Base y Caso Recursivo.
- Úsala para árboles, grafos y problemas de "Divide y Vencerás".
- Memoization: El truco para que la recursión sea ultra rápida.
- Cuidado con el Stack Overflow en inputs grandes.
- Cada nivel de recursión =  $O(1)$  de espacio extra en el stack.

---

---



# Cadenas y Matemáticas



## Capítulo 55

# Búsqueda de Patrones en Texto: KMP y Rabin-Karp

### ¿Qué es?

Buscar una palabra (patrón) dentro de un texto largo , como el Ctrl+F de tu navegador. La forma ingenua compara letra por letra desde cada posición: lenta ( $O(n \cdot m)$ ) porque repite comparaciones.

Dos formas inteligentes de hacerlo en  $O(n + m)$ :

- KMP: precalcula cuánto puede "saltar" el patrón cuando falla, sin retroceder en el texto.
- Rabin-Karp: convierte cada ventana del texto en un número (hash) y compara números en vez de letras.

### Explicado para un niño de 12

Quieres encontrar la palabra "ABABC" dentro de un texto larguísimo .

La forma tonta: la comparas empezando en cada letra, y si falla, vuelves a empezar todo desde la siguiente letra. Repites trabajo que ya hiciste.

KMP es más listo: cuando ya comparaste "ABAB" y falla la siguiente letra, KMP sabe que "AB" del inicio del patrón ya coincide, así que NO vuelve hasta el principio; salta directo y sigue. Es como cuando ya memorizaste parte de una canción y no empiezas desde cero cada vez que te equivocas.

Rabin-Karp usa otro truco: le pone un "número de identidad" (huella) a cada pedazo del texto del tamaño de la palabra, y compara huellas. Si dos huellas son iguales, probablemente encontró la palabra.

### Dicho de otra forma

**KMP (Knuth-Morris-Pratt)** evita retroceder en el texto usando un arreglo de "prefijos-sufijos" (`lps`, longest proper prefix which is also suffix). `lps[i]` dice, para el prefijo del patrón hasta  $i$ , cuántos caracteres del inicio coinciden con el final. Cuando una comparación falla, el patrón salta a `lps[j-1]` en vez de reiniciar. Total:  $O(n + m)$ .

**Rabin-Karp** calcula un hash rodante (rolling hash) de cada ventana del texto de longitud  $m$  y lo compara con el hash del patrón. El rolling hash actualiza en

$O(1)$  al deslizar la ventana (quita la letra que sale, agrega la que entra). Promedio  $O(n+m)$ ; peor caso  $O(n \cdot m)$  si hay muchas colisiones de hash. Brilla para buscar MÚLTIPLES patrones a la vez.

## 🧠 Diagrama

KMP: patrón = "ABABCABAB"

Construir lps (prefijo propio = sufijo más largo):

|         |   |   |   |   |   |   |   |   |   |
|---------|---|---|---|---|---|---|---|---|---|
| índice: | A | B | A | B | C | A | B | A | B |
| lps:    | 0 | 0 | 1 | 2 | 0 | 1 | 2 | 3 | 4 |

Significado: si fallas en la posición 8 (ya casaste "ABABCABA"), saltas a lps[7]=3, o sea reusas "ABA" ya emparejado. No retrocedes en el texto.

RABIN-KARP: rolling hash

texto: "ABCDE", patrón "CDE" (largo 3)

hash("ABC") -> desliza -> hash("BCD") -> hash("CDE") == hash patrón!  
(al deslizar: quita 'A', agrega 'D', todo en  $O(1)$ )

1 KMP EN 2 FASES

2 1. Precalcula lps[] del patrón:  $O(m)$ .

3 2. Recorre el texto con dos índices (i en texto, j en patrón). Si  
4 coinciden, avanza ambos. Si falla y  $j > 0$ ,  $j = \text{lps}[j-1]$  (salta). Si  
5 j llega a m, encontraste una ocurrencia.

6  
7 PSEUDOCÓDIGO (construir lps)

8  
9  
10 función buildLPS(patrón):

11 lps[0] = 0; len = 0; i = 1

12 MIENTRAS i < m:

13 SI patrón[i] == patrón[len]:

14 len++; lps[i] = len; i++

15 SINO SI len > 0:

16 len = lps[len-1] // retrocede en el patrón

17 SINO:

18 lps[i] = 0; i++

19 return lps

20  
21 CÓDIGO C++ (KMP)

22  
23  
24 #include <string>

25 #include <vector>

26 using namespace std;

```

27
28 vector<int> buildLPS(const string& p) {
29 int m = p.size(); vector<int> lps(m, 0);
30 for (int i = 1, len = 0; i < m;) {
31 if (p[i] == p[len]) lps[i++] = ++len;
32 else if (len) len = lps[len-1];
33 else lps[i++] = 0;
34 }
35 return lps;
36 }
37
38 int kmpSearch(const string& text, const string& pat) {
39 vector<int> lps = buildLPS(pat);
40 int n = text.size(), m = pat.size();
41 for (int i = 0, j = 0; i < n;) {
42 if (text[i] == pat[j]) { i++; j++;
43 if (j == m) return i - m; // ocurrencia encontrada
44 } else if (j) j = lps[j-1]; // salta usando lps
45 else i++;
46 }
47 return -1;
48 }
49
50 CÓDIGO C++ (Rabin-Karp, rolling hash)

51
52
53 int rabinKarp(const string& text, const string& pat) {
54 int n = text.size(), m = pat.size();
55 if (m > n) return -1;
56 const long long B = 256, MOD = 1e9 + 7;
57 long long hp = 0, ht = 0, pow = 1;
58 for (int i = 0; i < m; i++) {
59 hp = (hp * B + pat[i]) % MOD;
60 ht = (ht * B + text[i]) % MOD;
61 if (i) pow = pow * B % MOD;
62 }
63 for (int i = 0; i + m <= n; i++) {
64 if (hp == ht && text.compare(i, m, pat) == 0) return i; //
 verifica
65 if (i + m < n) // desliza
 ventana
66 ht = ((ht - text[i]*pow % MOD + MOD) * B + text[i+m]) % MOD;
67 }
68 return -1;
69 }
70
71 COMPLEJIDAD (Big-O)

72
73

```

| 74 | Algoritmo                                                 | Tiempo           | Nota                             |
|----|-----------------------------------------------------------|------------------|----------------------------------|
| 75 |                                                           |                  |                                  |
| 76 | Naive                                                     | $O(n \cdot m)$   | compara desde <b>cada</b> pos    |
| 77 | KMP                                                       | $O(n + m)$       | nunca retrocede en texto         |
| 78 | Rabin-Karp                                                | $O(n + m)$ prom. | $O(n \cdot m)$ peor (colisiones) |
| 79 |                                                           |                  |                                  |
| 80 | * Rabin-Karp brilla buscando múltiples patrones a la vez. |                  |                                  |

### 🔗 En entrevistas

El corazón de KMP es el lps: si sabes construir el arreglo de prefijos-sufijos, KMP sale solo. Practica el lps hasta que sea automático.

"Implement strStr()" (LC 28) y "Repeated String Match" son KMP directos. "Longest Happy Prefix" (LC 1392) es literalmente **lps[m-1]**.

Rabin-Karp necesita verificación: dos hashes iguales NO garantizan igualdad (colisión). Siempre compara los caracteres reales al hacer match de hash.

Rolling hash reutilizable: la técnica de hash rodante de Rabin-Karp sirve para detectar substrings duplicados, comparar rangos de texto en  $O(1)$  y problemas de "longest duplicate substring".

### 📋 Resumen rápido

- Buscar un patrón de largo  $m$  en un texto de largo  $n$ .
- Naive  $O(n \cdot m)$ ; KMP y Rabin-Karp  $O(n + m)$ .
- KMP: arreglo lps (prefijo propio = sufijo) para saltar sin retroceder.
- Rabin-Karp: rolling hash de cada ventana; compara números.
- Rabin-Karp exige verificar el match real (colisiones de hash).
- Rabin-Karp es ideal para múltiples patrones a la vez.

# Capítulo 56

## Algoritmos Matemáticos Esenciales

### ¿Qué es?

Un puñado de trucos matemáticos que aparecen escondidos en muchísimos problemas :

- GCD (Euclides): el máximo común divisor de dos números, en un parpadeo.
- Criba de Eratóstenes: encontrar TODOS los números primos hasta  $N$  rapidísimo.
- Exponenciación rápida: calcular  $a^b$  en  $O(\log b)$  en vez de multiplicar  $b$  veces.
- Aritmética modular: trabajar con residuos para no desbordar con números gigantes.

### Explicado para un niño de 12

Cuatro herramientas de matemáticas mágicas :

GCD: para saber el pedazo más grande con el que puedes medir dos cuerdas de distinto largo sin que sobre. Vas restando la chica de la grande (o usando el residuo) hasta que quedan iguales: ese es el GCD.

Criba: quieres una lista de números primos (los que solo se dividen entre 1 y ellos mismos). Tachas todos los múltiplos de 2, luego de 3, luego de 5...y los que quedan sin tachar son primos.

Exponenciación rápida: para calcular 2 elevado a 16, en vez de multiplicar 2 dieciséis veces, elevas al cuadrado: 2, 4, 16, 256...¡en 4 pasos! Cada vez duplicas el exponente. Menos trabajo, mismo resultado.

### Dicho de otra forma

**GCD (algoritmo de Euclides):**  $\gcd(a, b) = \gcd(b, a \bmod b)$  hasta que  $b = 0$ .  $O(\log \min(a, b))$ . De ahí sale el LCM:  $\text{lcm}(a, b) = a / \gcd(a, b) \cdot b$ .

**Criba de Eratóstenes:** marca como no-primos los múltiplos de cada primo desde 2. Genera todos los primos hasta  $N$  en  $O(N \log \log N)$ .

**Exponenciación rápida (binary exponentiation):** usa la representación binaria del exponente. Si  $b$  es par,  $a^b = (a^{b/2})^2$ ; si es impar,  $a^b = a \cdot a^{b-1}$ .  $O(\log b)$ . Combinada con módulo evita overflow: base de la exponenciación modular usada

en criptografía y hashing.

**Aritmética modular:**  $(x + y) \bmod m$ ,  $(x \cdot y) \bmod m$  mantienen los números acotados. Regla clave: aplica el módulo tras cada operación para no desbordar.

## Diagrama

EUCLIDES  $\gcd(48, 18)$ :

$\gcd(48, 18) \rightarrow 48 \bmod 18 = 12 \rightarrow \gcd(18, 12)$

$\gcd(18, 12) \rightarrow 18 \bmod 12 = 6 \rightarrow \gcd(12, 6)$

$\gcd(12, 6) \rightarrow 12 \bmod 6 = 0 \rightarrow \gcd(6, 0) = 6$

GCD = 6

CRIBA hasta 20 (X = tachado, no primo):

2 3 4X 5 6X 7 8X 9X 10X 11 12X 13 14X 15X 16X 17 18X 19 20X

Primos: 2 3 5 7 11 13 17 19

EXP RÁPIDA  $2^{10}$ :

$10 = 1010$  binario

$2^{10} = 2^8 * 2^2 = 256 * 4 = 1024$  (4 cuadrados, no 10 multiplic.)

1 GCD (Euclides)

2 -  $\gcd(a, b)$ : si  $b == 0$  **return**  $a$ ; si no  $\gcd(b, a \% b)$ .

3  
4 EXPONENCIACIÓN RÁPIDA (con módulo)

5 - Mientras el exponente  $> 0$ : si es impar, multiplica el resultado por la  
6 base; eleva la base al cuadrado; divide el exponente entre 2.

7  
8 PSEUDOCÓDIGO (exponenciación modular)

9  
10  
11 función  $\text{powMod}(a, b, m)$ :

12 resultado = 1;  $a = a \bmod m$

13 **MIENTRAS**  $b > 0$ :

14 **SI**  $b$  es impar: resultado = resultado \*  $a \bmod m$

15  $a = a * a \bmod m$

16  $b = b / 2$  // desplaza un bit

17 **return** resultado

18  
19 CÓDIGO C++

20  
21  
22 **#include** <vector>

23 **using namespace** std;

24  
25 // GCD y LCM

26 **long long**  $\gcd(\text{long long } a, \text{long long } b)$  {

27 **return**  $b == 0 ? a : \gcd(b, a \% b)$ ;

28 }

```

29 long long lcm(long long a, long long b) {
30 return a / gcd(a, b) * b; // divide primero: evita overflow
31 }
32
33 // Exponenciación modular rápida -> O(log b)
34 long long powMod(long long a, long long b, long long m) {
35 long long res = 1; a %= m;
36 while (b > 0) {
37 if (b & 1) res = res * a % m; // bit impar
38 a = a * a % m; // eleva al cuadrado
39 b >>= 1; // siguiente bit
40 }
41 return res;
42 }
43
44 // Criba de Eratóstenes -> primos hasta n
45 vector<bool> criba(int n) {
46 vector<bool> esPrimo(n + 1, true);
47 esPrimo[0] = esPrimo[1] = false;
48 for (long long i = 2; i * i <= n; i++)
49 if (esPrimo[i])
50 for (long long j = i * i; j <= n; j += i) // desde i*i
51 esPrimo[j] = false;
52 return esPrimo;
53 }
54
55 COMPLEJIDAD (Big-O)
56
57
58
59
60
61
62
63
64

```

| Algoritmo      | Tiempo              |
|----------------|---------------------|
| GCD (Euclides) | $O(\log \min(a,b))$ |
| Exp. rápida    | $O(\log b)$         |
| Criba          | $O(n \log \log n)$  |

\* La criba usa  $O(n)$  memoria (un `bool` por número hasta  $n$ ).



### En entrevistas

powMod es la base de todo: hashing polinómico, inverso modular (por Fermat:  $a^{m-2} \bmod m$  cuando  $m$  es primo), y criptografía. Tenlo memorizado.

Criba desde  $i$ : al marcar múltiplos de  $i$ , empieza en  $i^2$  (los menores ya fueron marcados por primos más chicos). Pequeña optimización que preguntan.

Overflow: en `a * a % m` con `long long` y  $m$  grande puedes desbordar. Si  $m \sim 10^{18}$ , usa `__int128` o multiplicación modular segura.

GCD para simplificar: muchos problemas de fracciones, rotaciones de arreglos ("rotate array by  $k$ " usa `gcd(n,k)` ciclos) y geometría de rejilla se apoyan en el

GCD.



### Resumen rápido

- GCD (Euclides):  $\text{gcd}(a,b) = \text{gcd}(b, a \bmod b)$ ;  $O(\log \min)$ .
- LCM =  $a / \text{gcd}(a,b) \cdot b$  (divide primero para no desbordar).
- Criba: tacha múltiplos desde  $i \cdot i$ ; primos hasta  $n$  en  $O(n \log \log n)$ .
- Exponenciación rápida: eleva al cuadrado;  $O(\log b)$ .
- powMod evita overflow y es base de hashing e inverso modular.
- Aritmética modular: aplica el módulo tras cada operación.